

基于风险校准与库存价值控制的微网购电调度

摘要

针对计划购电、储能运行与日内调单的耦合关系，本文以实际总购电费用最小为目标，构建“库存状态—可见信息—合同费用”统一调度框架。

针对问题一，建立**多期线性规划与连续状态动态规划模型**，证明自由弃电条件下充放电互斥的等价线性化，并以凸分段线性价值函数实现无 SOC 网格递推。两种方法均得最优日费用 35,126.85 元，购电 59,482.70 kWh，较无储能节省 26.90%；九个初始库存的 DP 值与混合整数规划一致。

针对问题二，建立**岭回归预测、分位数风险校准与历史路径价值控制模型**。70%误差余量控制欠购风险，过去 28 日配对残差刻画后续缺口费用。2—12 月 334 天总费用为 13,991,392.60 元，较相同计划生成规则的即时平衡执行器节省 141,219.56 元，主要来自库存向高价缺口时段转移。

针对问题三，建立**非对称结算滚动 LP 与可调单库存价值模型**，结合**负载收缩回归及月度模型选择**，在 6、12、18 点更新信息。计入下次调单机会后，总费用为 13,479,283.32 元，较仅加入月度负载选择的策略再省 9,844.62 元；固定合同价值函数直接移用于滚动问题反而增费，说明续期价值必须匹配合同权限。

针对问题四，比较**同期、岭回归与残差 GRU 价格预测模型**。GRU 价格 MAE 为 0.04366 元/kWh，较岭回归降低 8.09%；按验证费用选择价格模型并加入库存价值控制，日前与滚动费用分别为 14,765,492.68 元和 14,210,881.01 元，较对应基础执行器节省 115,042.37 元和 5,444.97 元。共同名义可行域下的价格改善空间为 1.002%，与实际运行收益分别评价。

11 条全年候选轨迹通过物理约束与合同账单复核，四组节费对照均保持相同初末库存。结果表明，风险校准确定合理购电规模，滚动预报降低供需偏差，库存价值控制将有限电量配置给更昂贵的后续缺口，三者共同提升微网运行经济性。

关键词：微网调度；线性规划；连续动态规划；库存价值；滚动优化

一 问题重述

1.1 研究背景

光伏出力与小区负载在时间上存在差异。并网微网通过外部电网补充电力，并利用储能能在低价或光伏富余时充电、高价或供电不足时放电，实现电量在时段间的转移。储能的容量、功率和充放电损耗共同限制了电量转移规模，因此购电计划必须与储能运行协同制定。

实际运行还受到信息与交易规则的双重约束。运营者需在每天零点申报全天计划，当天负载、光伏和波动电价尚未完全获知；计划不足会触发高价紧急购电，计划过量则形成额外支出。日内预报提供了修正供需判断的机会，但调整购电同样产生费用。如何在保障供电的前提下，协调峰谷套利、预测风险与计划调整成本，是本题的核心问题。

1.2 问题的提出

题目给出单日分时电价及供需曲线、全年负载与光伏实测数据、日内滚动光伏预报和波动电价四类附件[1]。微网储能额定容量为 12,000 kWh，最大充放电功率为 5,000 kW，储电量须保持在 1,200—10,800 kWh 之间，初始储电量为 6,000 kWh；充放电效率按题设取值，具体解释见第 3.1 节。以十分钟为决策间隔，需依次解决以下问题。

问题一：确定性条件下的购电与储能调度。 在电价、负载逐日重复且当日光伏预测已给定时，制定全天 144 个时段的购电与充放电计划，使日初、日末储电量相等且总购电费用最小，给出指定时段购电量、全天费用及四小时储能汇总。

问题二：供需不确定条件下的日前购电。 利用决策时已掌握的历史数据，在每日零点制定购电计划；实时供电缺口按交易电价的五倍紧急补购，普通购电按计划量结算。求解 2025 年 2 月 1 日至 12 月 31 日的策略，并报告 3 月 20 日、6 月 21 日、9 月 23 日和 12 月 21 日的指定结果。

问题三：滚动预报下的计划调整。 利用 0、6、12、18 点发布的未来 24 小时整点光伏预报，调整尚未执行的购电计划；相对午夜计划，取消部分按半价、新增部分按 1.5 倍交易电价结算。制定全年滚动策略，并比较仅用午夜预报与日内更新的总费用，判断增加预报更新是否必要。

问题四：波动电价下的策略扩展。 在当天电价未知且实时波动的条件下，建立电价预测与购电调度模型，分别重新求解问题二的前日策略和问题三的滚动策略，按实际交易电价计算费用并给出相应结果。

二 问题分析

2.1 问题一的分析

本问的供需与电价均已给定，关键在于利用储能将低价电量转移至高价时段。各时段通过储电量递推相互关联，不能独立优化。以购电费用最小为目标，联立能量平衡、容量、功率和日周期约束，建立多期规划；再利用无成本弃电条件消除循环充放电，将互斥模型等价化为线性规划。所得模型构成后续三问的共同物理基础。

2.2 问题二的分析

本问新增供需预测误差，欠购五倍补购与多购计划支出形成非对称风险。先以岭回归预测、历史误差分位数校准日前计划，再利用过去 28 日配对残差构造后续费用函数，按当前储电量选择实时放电。预测层控制缺口规模，执行层分配有限库存，两层共同以实际账单评价。

2.3 问题三的分析

本问允许修订未执行合同。以保留、取消和新增量列式，随光伏预报和负载偏差更新滚动求解；后续库存价值须计入下次调单机会，不能沿用固定合同下的五倍缺口成本。由此协调当前放电、后续补购与调单费用。

2.4 问题四的分析

本问进一步引入电价不确定性。比较同期、岭回归和 GRU 的价格预测，在相同信息条件下区分预测精度与调度收益；日前执行采用固定合同价值估计，滚动执行采用可调单价值估计。当前交易使用已揭示电价，后续价值仅使用当前价格预测。

四问按“确定性基准—供需风险—日内修正—价格不确定性”逐步扩展，整体建模思路如图 1 所示。各问共用能量平衡与储能约束，新增模型仅处理对应的信息与结算机制。

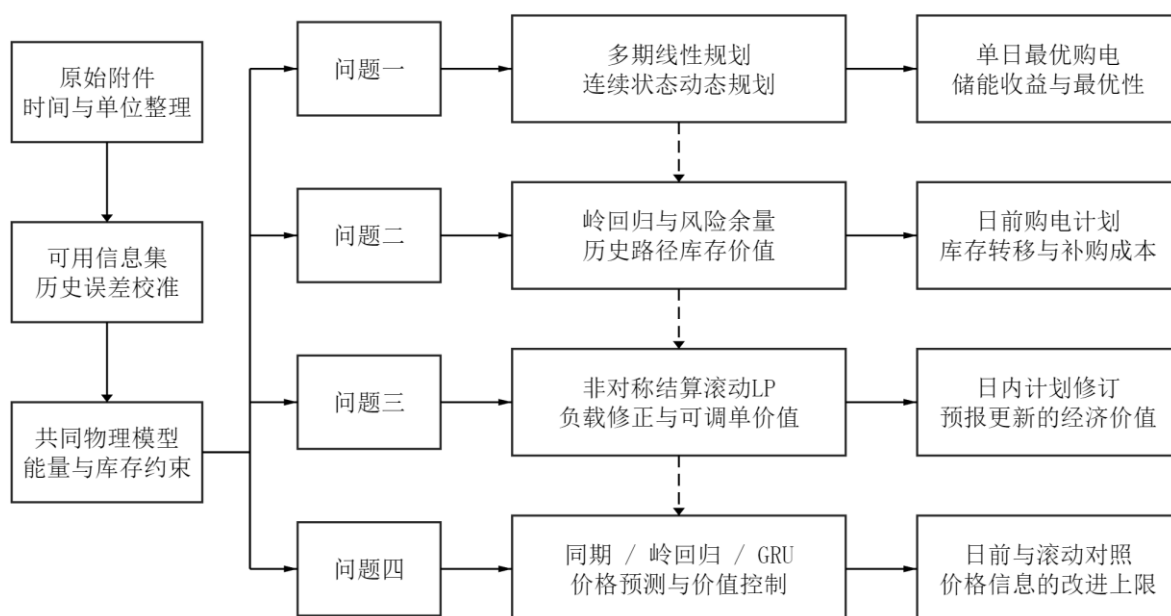


图 1 整体建模思路与四问递进关系

三 模型假设与数据处理

3.1 参数及工作假设

储能与时间参数见表 1。功率上限定义在微网母线侧，所有决策电量统一使用 kWh。基础模型将题设 90%的充放电效率解释为两侧各 90%，对应往返效率 81%；第 5.3 节另以往返效率 90%检验结果对效率解释的敏感性。

表 1 储能参数与时间离散

参数	数值	性质
额定容量	12000 kWh	题面给定
允许储电量	1200—10800 kWh	题面给定
最大充放电功率	5000 kW	题面给定
初始储电量	6000 kWh	1 月 1 日 00:00 给定
充电 / 放电效率	0.9 / 0.9	主实验解释
时段长度	1/6 h	十分钟离散

(1) 第一问按模板起点采样，周期曲线以 24:00 值补齐 00:00，功率在十分钟区间内取常值；全年历史序列按记录终点对应十分钟区间均值建模。状态统一定义在区间边界，功率乘 1/6 小时换算为电量。

(2) 微网仅从外网购电，不计售电收入；无法消纳的电量允许无成本弃用，电池退化成本不纳入目标。第一、二问普通购电按计划量结算，第三、四问主动调整按保留量、取消量和新增量分项结算。

(3) 实时运行按十分钟区间内功率恒定建模，以当前可观测供需完成充放电与紧急购电；日前及日内计划仅使用决策时已经获得的信息。

(4) 第二至四问的实际储电量连续跨日。日前名义轨迹的日末储电量下限为 6,000 kWh，用于保留次日运行所需电量；实际日末状态由实时运行决定，并作为次日初始状态。

3.2 输入审计与时间边界

附件 1 含 144 个时段，附件 2 的负载与光伏各含 $365 \times 144 = 52,560$ 个观测。以 2025 年完整日期和十分钟时段为索引对齐两类序列，将附件 1 混合存储的时间类型与字符串统一为分钟，核对序列为 10、20、…、1440。功率与电量按下式换算：

$$L_{d,t} = P_{d,t}^{\text{load}} \Delta t, \quad V_{d,t} = P_{d,t}^{\text{PV}} \Delta t, \quad \Delta t = \frac{1}{6} \text{ h}. \quad (1)$$

数据核查表明：附件 1 含 60 个时间类型单元格和 84 个字符串，统一转换后得到完整时间序列；附件 2 两类序列均无缺失、非有限值、负数、重复日期或完全重复日曲线。全局四分位距检验未检出超界观测，原始样本全部保留。

预处理仅规范时间格式与计量单位，无须填补、删点、缩尾或平滑。光伏序列中的 23,540 个零值按零发电记录保留。训练集标准化和预测值非负载断分别作用于特征及模型输出，不改变实际观测。

原始供需和调度轨迹保留十分钟分辨率。图中的月均值、分位范围和累计费用均由原始记录直接汇总，分别刻画季节结构、价格分布与策略收益。

3.3 单位、富余电量与计划计费核验

附件 1—4 的五张工作表均无公式、隐藏行列或隐藏工作表。以另一读取程序逐值核对附件 2 的 105,120 个负载与光伏观测，换算后与建模输入一致；附件 3 的 1,095 处空日期是四行一组的结构记录，只补齐日期标识，不插补预测值。实际光伏零值和 0.0076 元/kWh 的低电价均保留。

全篇采用电量平衡：功率乘 1/6 小时得到 kWh，5,000 kW 对应单时段充放电上限 833.33 kWh。平衡式中的非负弃电变量允许供给富余；将其移项后即为“购电+光伏+放电+紧急购电 $\geq$ 负载+充电”，因此等式与供给不小于需求的约束等价。取消弃电变量而强行取等号才会错误地要求全部消纳。

普通费用按合同量核算。针对性测试取电池已满、计划购电 1,000 kWh、负载 100 kWh、无光伏、电价 2 元/kWh，实际弃电 900 kWh，普通购电费仍为 2,000 元。第三、四问则按最终合同的保留、取消和新增量结算，未把实际消纳量代替计划量。

四 符号说明

下文沿用表 2 的符号。单日模型省略日期下标，日内区间用 1—144，状态包含 145 个边界点；跨日计算另带日期下标。

表 2 主要符号及单位

符号	含义	单位
d, t	日期与十分钟时段	—

符号	含义	单位
$L_{d,t}, V_{d,t}$	负载与光伏电量	kWh
$\hat{L}_{d,t}, \hat{V}_{d,t}$	日前负载与光伏预测	kWh
$g_{d,t}, e_{d,t}$	计划购电与紧急购电	kWh
$c_{d,t}, d_{d,t}$	母线侧充电与放电	kWh
$w_{d,t}$	不能利用的富余电量	kWh
$S_{d,t}$	区间起点的储电量	kWh
p_t	固定分时交易电价	元/kWh
η_c, η_d	充电与放电效率	—
z_t	充电状态二元变量	—
λ, q	岭惩罚强度与余量分位数	—
$r_{d,t}(q)$	非负净负载预测余量	kWh

日期下标与放电变量均使用字母 d ，前者仅出现在下标位置，后者作为决策变量出现；两者由位置及上下文区分。

五 第一问的模型建立与求解

5.1 供需结构与决策目标

图 2 给出单日供需曲线与分时电价。光伏集中于白天，负载贯穿全天；储能既可吸收午间富余光伏，也可利用外网峰谷价差转移购电时段。两种作用共同决定购电与充放电计划。

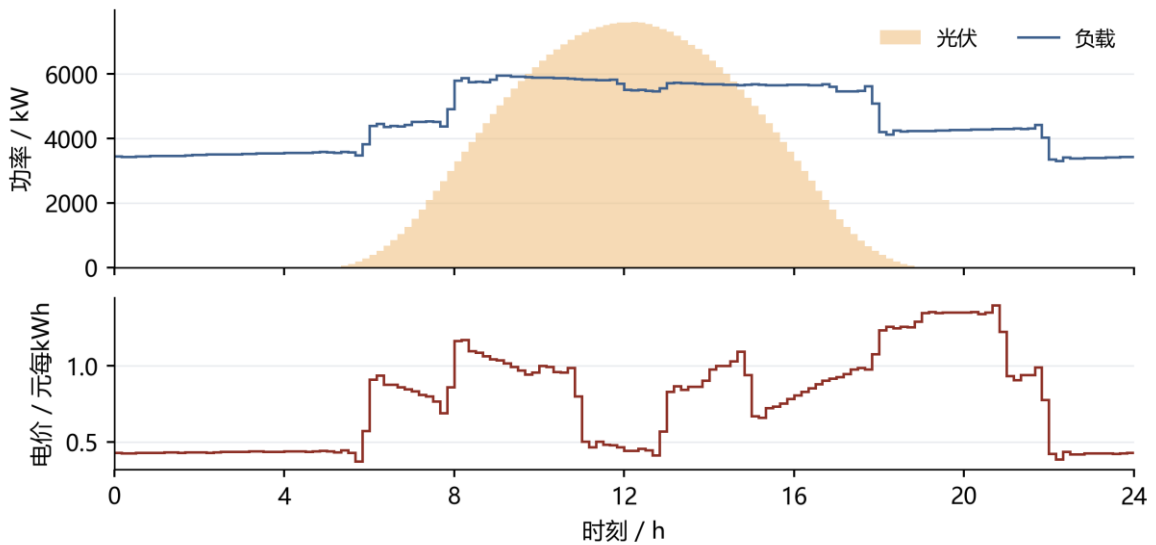


图 2 第一问供需曲线与分时电价

图 2 的每个阶梯对应一个十分钟区间，功率与电价分别使用独立坐标，光伏填充区域表示其出力过程。

目标是最小化全天计划购电费：

$$\min C_1 = \sum_{t=1}^{144} p_t g_t. \quad (2)$$

以母线侧收支写能量守恒，将允许的富余供电显式记为未利用电量：

$$g_t + V_t + d_t = L_t + c_t + w_t, \quad g_t, w_t \geq 0. \quad (3)$$

充电损耗发生在进入储能时，放电损耗发生在从储能输出至母线时，因而状态方程为：

$$S_{t+1} = S_t + \eta_c c_t - \frac{d_t}{\eta_d}, \quad 1200 \leq S_t \leq 10800. \quad (4)$$

SOC 上下界施加于全部 145 个状态点，状态递推与其他时段约束对应 $t=1, \dots, 144$ 。令每时段最大母线侧充放电量 $M=5000/6$ kWh，用二元变量直接限制互斥：

$$0 \leq c_t \leq M z_t, \quad 0 \leq d_t \leq M(1 - z_t), \quad z_t \in \{0,1\}. \quad (5)$$

第一问满足以下初始和终端条件，避免利用电池初始存量净放电造成虚假节省：

$$S_1 = S_{145} = 6000. \quad (6)$$

将上述关系集中写成一般日模型，决策变量为购电、充放电、未利用电量、储电量 and 状态变量。给定不同供需输入与初末状态条件时，可复用以下结构：

$$\left\{ \begin{array}{l} \min_{g,c,d,w,S,z} \quad \sum_{t=1}^{144} p_t g_t \\ \text{s. t.} \quad g_t + V_t + d_t = L_t + c_t + w_t, \\ S_{t+1} = S_t + \eta_c c_t - d_t / \eta_d, \\ 1200 \leq S_t \leq 10800, \quad g_t, w_t \geq 0, \\ 0 \leq c_t \leq M z_t, \quad 0 \leq d_t \leq M(1 - z_t), \\ z_t \in \{0,1\}, \quad S_1 = S_{145} = 6000. \end{array} \right. \quad (7)$$

完整互斥模型含 865 个变量，其中 144 个为二元变量，使用 SciPy 的 milp 接口调用 HiGHS[2]。设置相对最优间隙 10^{-7} 、单次求解上限 30 秒，求得最优整数解。第 5.4 节进一步证明等价线性化，并用于后续滚动模型求解。

5.2 调度结果与指定时段

计算得到全天购电量 59,482.70 kWh、费用 35,126.85 元。购电与储能轨迹见图 3，指定时段购电和四小时充放电分别见表 3、表 4。

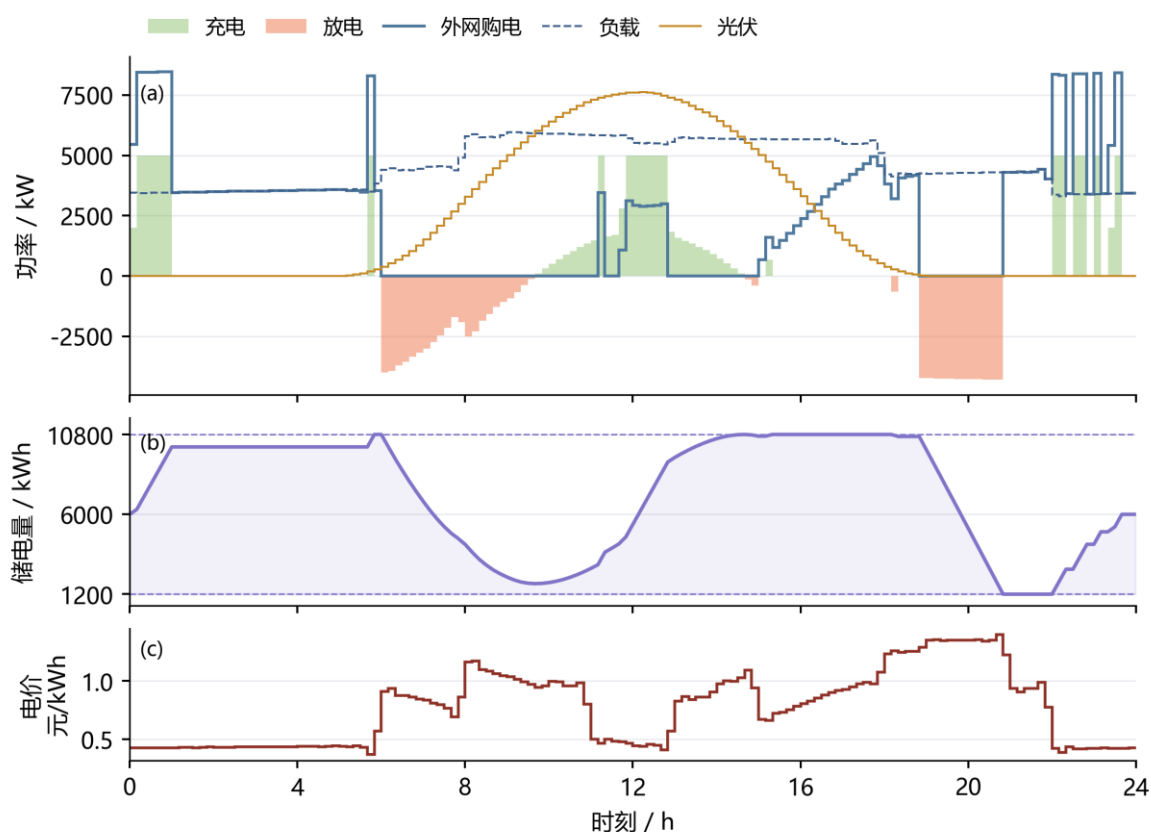


图 3 第一间电价驱动的购电与储能运行

图 3 的上面板给出供需、购电与充放电功率，中面板给出储电量及边界，下方面板给出电价。三者共用时间轴，可逐时核对价格、动作和储电状态之间的关系。

表 3 列出指定十分钟区间的购电量及全天汇总。

表 3 第一间指定时段购电量及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	486.40	14:00-14:10	0.00
16:00-16:10	394.93	18:00-18:10	636.98	20:00-20:10	0.00
全天购电量 59,482.70 kWh		全天购电费 35,126.85 元			

表 4 按四小时汇总充放电电量，并列出各块边界储电量。

表 4 第一间四小时充放电及边界储电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,500.00	0.00	04:00-08:00	833.33	5,947.42
08:00-12:00	3,954.63	2,121.42	12:00-16:00	6,119.37	91.10
16:00-20:00	0.00	5,068.69	20:00-24:00	5,333.33	3,571.31
00:00 储电量	6,000.00		24:00 储电量	6,000.00	

低价时段的外网购电同时满足负载与充电，形成集中购电峰值；高价时段由储能放电补充供电，压低外网购电量。图 3 中购电峰值与充电动作对应，反映储能的跨时段套

利。表 3、表 4 分别按十分钟和四小时汇总；四小时块内可先充后放，各十分钟区间仍保持互斥。

5.3 最优性检验与效率解释

删除整数限制得到线性松弛下界 35,126.84858929 元，与整数解目标之差为 0.00000000 元，求解器相对间隙为零。整数解与松弛下界一致，达到该模型的全局最优费用。能量平衡最大残差约 2.27e-13 kWh，SOC 递推与容量、功率边界均通过复核。

以每时段正净负载直接购电构造无储能基线：

$$g_t^{(0)} = \max(L_t - V_t, 0), \quad C_1^{(0)} = \sum_t p_t g_t^{(0)}. \quad (8)$$

该基线费用为 48,052.05 元，基础模型节省 26.90%。效率解释的对照见图 4：若总往返效率为 0.9，并对称取两侧效率为平方根 0.9，费用为 33,801.48 元。提高往返效率可进一步降低套利损耗。

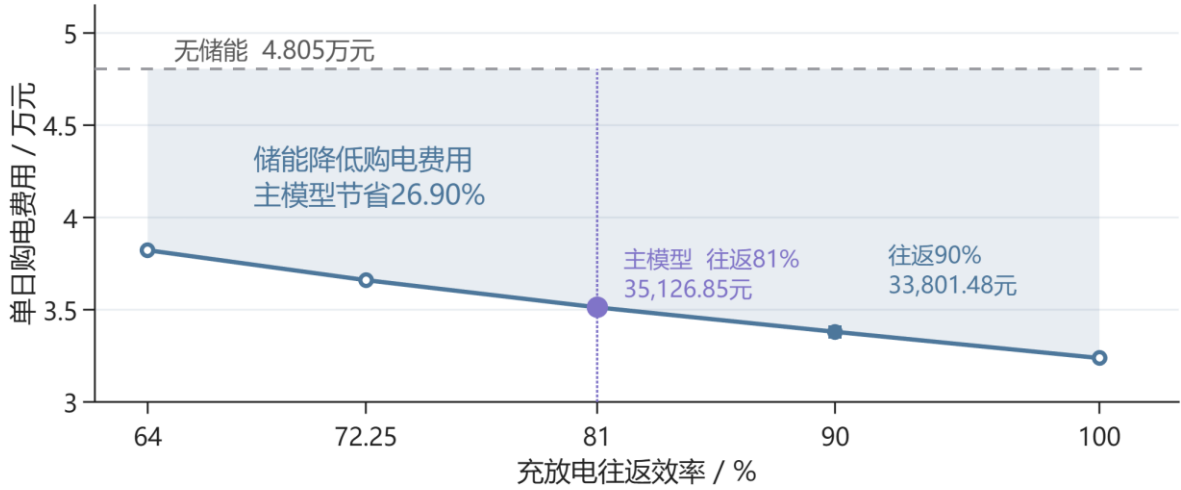


图 4 储能往返效率与最优购电费用的关系

图 4 给出五组往返效率下的最优费用，并以无储能费用为参照。储能降低购电支出，效率提升进一步减少能量转移损耗；主结果采用两侧各 90%、即往返 81% 的效率。

5.4 互斥约束的等价线性化

为降低多次滚动求解的规模，考察互斥约束的结构性质。在允许无成本且无上界弃电的条件下，可将任意线性松弛可行解转化为同费用的互斥可行解。令 $\rho = \eta c \eta d \leq 1$ ，对任一时段作如下变换：

$$x_t = \min\{c_t, d_t/\rho\}, \quad c'_t = c_t - x_t, \quad d'_t = d_t - \rho x_t, \quad w'_t = w_t + (1 - \rho)x_t. \quad (9)$$

充电量和放电量均不增加，且至少一个变为零。由于 $\eta c x = \rho x / \eta d$ ，储能增量完全不变；由于 $d' - c' - w' = d - c - w$ ，母线平衡也保持不变。购电、紧急购电和所有计划增减量不变，因此本文各类费用均不变。逐槽处理后得到符合互斥约束的物理理解：

$$C_{LP}^* \leq C_{MILP}^* \leq C_{transformed LP} = C_{LP}^*. \quad (10)$$

由上下界相等可知，线性规划与混合整数规划的最优费用相同。第一问由 865 个变量降至 721 个变量，消去全部 144 个二元变量；滚动调整模型采用相同变换。求解后逐时消除循环充放电并复核约束，即得到可执行的互斥解。

上述等价性以自由弃电、线性购电成本及所列储能约束为前提。引入切换成本或弃电上限后，需重新判断变换的可行性与费用保持性。

5.5 连续状态价值函数与精确递推

线性规划给出指定初始状态下的最优轨迹；为刻画其他库存水平的后续费用，以储电量 s 为状态，令 $v=s-s'$ ， $n=L-V$ 。单时段净放电 $v \geq 0$ 时向母线提供 $\eta_d v$ ，净充电 $v < 0$ 时从母线吸收 $-v/\eta_c$ 。最小普通购电费及递推为：

$$h_t(v) = p_t \max\{n_t - \phi(v), 0\}, \quad \phi(v) = \begin{cases} \eta_d v, & v \geq 0, \\ v/\eta_c, & v < 0, \end{cases} \quad J_t(s) = \min_{s' \in \mathcal{S}_t(s)} \{h_t(s-s') + J_{t+1}(s')\}. \quad (11)$$

集合 $\mathcal{S}$ 限定储电量 1,200—10,800 kWh、单时段充放电不超过 833.33 kWh；终端函数在 $s=6,000$ 处为 0，其余为正无穷。由于价格非负且 $\eta_c \eta_d \leq 1$ ， h 为凸分段线性函数。凸函数的下确界卷积与区间限制保持凸性[7]，故各 J 均可由有限折点精确表示。实现将两函数线段按斜率归并，再截取可行库存区间，无需离散 SOC 网格。凸性意味着库存边际节费 $-J$ 的斜率随库存增加而不增；严格期末等式下 J 未必处处递减，不把全部斜率解释为正收益。

初始价值函数含 14 个折点。在 1,200 至 10,800 kWh 的九个初始库存下，其函数值均与独立混合整数规划一致； $s=6,000$ 时费用为 35,126.84858929 元，与前述 LP 完全相同。DP 的新增作用是给出状态变化对应的费用函数，并未降低已达全局最优的确定性费用。参数化 LP 亦可描述全状态价值，二者是同一优化结构的不同表达。

六 第二问的预测与日前策略

6.1 训练与评价的时间顺序

采用按时间推进的训练、验证与评价划分：1 月 22—31 日选择参数与执行器，2 月 1 日至 12 月 31 日开展顺序回测。模型每七天更新，使用最近至多 60 个已结束日期。此次模型修订属于同年度数据上的开发复核；各时点决策遵守历史可见性，评价区间不作为从未接触的外部测试集。

各候选策略从 1 月 1 日 6,000 kWh 储电量进行共同热身。首日以附件 1 作为冷启动参考，历史不足七天时使用近三日均值，随后转为日 / 周同期预测。验证期初与评价期初储电量分别为 6,244.256891 kWh 和 9,902.811288 kWh，保证各策略具有相同起点。附件 1 仅用于初始化。

6.2 同期基线与岭回归预测

对负载或光伏电量序列 y ，同期基线取昨日与上周同日相同时间槽的均值：

$$\hat{y}_{d,t}^{\text{seasonal}} = \frac{y_{d-1,t} + y_{d-7,t}}{2}. \quad (12)$$

岭回归采用 21 个特征：昨日、前日、上周同日的同槽值，三日和七日同槽均值、七日同槽标准差、昨日全天均值和近三日全天均值共八项；前三阶日内正余弦六项；星期指示七项。日期及时间属于决策时已知的日历信息。标准化均值与尺度仅在当次训练集上估计，常量特征尺度置 1。

$$\min_{\beta, b} \sum_{i \in \mathcal{T}_d} (y_i - b - \tilde{x}_i^T \beta)^2 + \lambda \|\beta\|_2^2. \quad (13)$$

截距不惩罚，负载和光伏分别拟合。按验证 MAE 比较 $\lambda = 1, 10, 100$ ，均选 1；实现采用 NumPy 线性方程求解，与带截距岭目标一致 [3]。从 1 月 15 日开始具备拟合条件，之后每七日重新拟合。预测截断为非负；某槽过去七日光伏全零时，该槽预测置零，日出日落季节变化可能使此规则产生滞后。

评价指标采用功率单位的 MAE 与 RMSE；不使用夜间分母为零的普通 MAPE：

$$\text{MAE} = \frac{1}{n} \sum_i |P_i - \hat{P}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_i (P_i - \hat{P}_i)^2}. \quad (14)$$

6.3 净负载误差余量与名义优化

定义历史净负载误差及只包含过去信息的误差样本池：

$$\varepsilon_{j,s} = (L_{j,s} - V_{j,s}) - (\hat{L}_{j,s} - \hat{V}_{j,s}). \quad (15)$$

$$\mathcal{E}_{d,t} = \{\varepsilon_{j,s} : \max(2, d-28) \leq j < d, 1 \leq s \leq 144, |s-t| \leq 3\}. \quad (16)$$

日期下标以 1 月 1 日为 $d=1$ ；排除首日冷启动误差，最多池化 28 个已结束日期和目标槽前后三槽。跨日边界不绕回，不足 28 天时仅取已有历史。余量规则为：

$$r_{d,t}(q) = \max\{0, Q_q(\mathcal{E}_{d,t})\}. \quad (17)$$

无余量候选直接设 $r=0$ ，与“取零分位数”区分。对 $q=0.7, 0.8, 0.9$ ，分位数使用样本排序后的线性插值。将预测负载加余量作为名义需求，求解第五节同一物理模型：

$$L_t^{\text{nom}} = \hat{L}_{d,t} + r_{d,t}(q), \quad V_t^{\text{nom}} = \hat{V}_{d,t}, \quad S_{d,145}^{\text{nom}} \geq 6000. \quad (18)$$

风险校准通过历史误差分位数提高名义需求，在一月验证中按计划费与五倍紧急费之和选择余量。70%表示误差样本池的分位水平；模型属于分位数校准的确定性日前优化，其供电风险由顺序回测评价。

6.4 实时执行与跨日状态

当天普通购电计划固定后，定义当前供需剩余。对非负剩余尽量充电，对缺口尽量放电，并分别受可用空间、储电量和功率限制：

$$u_t = g_t + V_t - L_t. \quad (19)$$

$$c_t = \min \left\{ \max(u_t, 0), M, \frac{10800 - S_t}{\eta_c} \right\}. \quad (20)$$

$$d_t = \min \{ \max(-u_t, 0), M, \eta_d(S_t - 1200) \}. \quad (21)$$

$$e_t = \max(-u_t - d_t, 0), \quad w_t = \max(u_t - c_t, 0). \quad (22)$$

以上规则使同一时段仅有充电或放电，SOC 仍按状态方程更新，实际日末传给次日：

$$S_{d+1,1} = S_{d,145}. \quad (23)$$

所有供电缺口由紧急购电补足。全年评价费用定义为：

$$C_2 = \sum_{d,t} p_t g_{d,t} + 5 \sum_{d,t} p_t e_{d,t}. \quad (24)$$

将第二问策略概括如下。记 Ω 为第一问中除初末等式之外的物理可行域，输入替换为带余量预测，日初状态取真实已知值；G 表示本节的贪心实时平衡规则：

$$\left\{ \begin{array}{l} (g^{\text{plan}}, \dots) \in \arg \min_{(g,c,d,w,S,z) \in \Omega} \sum_t p_t g_t, \\ L^{\text{nom}} = \hat{L} + r(q), \quad V^{\text{nom}} = \hat{V}, \\ S_1^{\text{nom}} = S_1^{\text{actual}}, \quad S_{145}^{\text{nom}} \geq 6000, \\ (c_t, d_t, e_t, w_t) = \mathcal{G}(g_t^{\text{plan}}, L_t, V_t, S_t), \\ C_2 = \sum_{d,t} p_t (g_{d,t}^{\text{plan}} + 5e_{d,t}). \end{array} \right. \quad (25)$$

Ω 中各时段的平衡约束采用带余量的名义供需。计划层最小化普通购电费，实际缺口在执行时揭示并计入评价费用；余量 q 以一月验证总费用确定，随后固定。

上述即时平衡规则作为基础执行器：出现缺口便尽量放电，余下部分紧急补购。该规则未区分库存用于当前与后续时段的价值，改进执行器见第 6.5 节；普通购电仍按计划量结算。

6.5 历史路径价值驱动的实时放电

普通计划锁定后，计划费为执行阶段的已付成本。每次发布用此前至多 28 个完整日的负载、光伏配对残差，分别叠加至当前预测并截断非负，形成保持日内关联的净缺口路径。对每条历史路径按第 5.5 节递推计算紧急补购费用函数，终端附加价值取零；名义计划仍要求日末储电量不少于 6,000 kWh。以这些路径函数的均值估计后续费用，在当前真实缺口 $n = L - V - g > 0$ 时选择下一储电状态：

$$s_{t+1}^* \in \arg \min_{z \in [\max\{S_{\min}, S_t - \min(n_t, M)/\eta_d\}, S_t]} \left\{ 5p_t[n_t - \eta_d(S_t - z)] + \frac{1}{K} \sum_{k=1}^K J_{t+1}^{(k)}(z) \right\}. \quad (26)$$

M 为单时段放电上限；最优解只需比较可达区间端点和路径价值折点。富余时优先充电，超出容量或功率上限的部分弃电；紧急购电仅补当前负载缺口，不用于给电池充电。当库存的预计后续节费超过当前放电节费时，保留部分库存并紧急补购是可行且有经济意义的动作。题面要求及时补足缺口，并未规定必须先耗尽可放电量。

各历史路径的后续费用经递推后取均值，形成当前库存的续期价值估计；执行动作由当前真实供需和这一价值函数共同决定。1 月验证中，零终端价值与低谷价终端价值均得到 509,140.53 元，较即时平衡节省 616.94 元，按预设候选顺序选取零终端价值。

七 第二问的结果与检验

7.1 验证期选择与紧急购电权衡

表 5 与图 5 比较相同初始储电量下的验证费用及期末状态。随余量增加，紧急补购支出下降，普通购电支出上升；两者的折中决定余量选择。

表 5 一月份十天验证结果

预测器	余量	总费/元	紧急费/元	期末 SOC/kWh
同期	无	661,834.09	173,098.38	9,902.81
同期	70%	634,179.58	110,989.47	10,595.98
同期	80%	666,226.97	64,910.51	10,800.00
同期	90%	680,257.72	456.98	10,800.00
岭回归	无	532,888.41	61,836.07	5,924.08
岭回归	70%	509,757.47	8,765.44	6,623.17
岭回归	80%	529,846.45	4.16	7,110.16
岭回归	90%	633,664.05	0.00	9,039.38

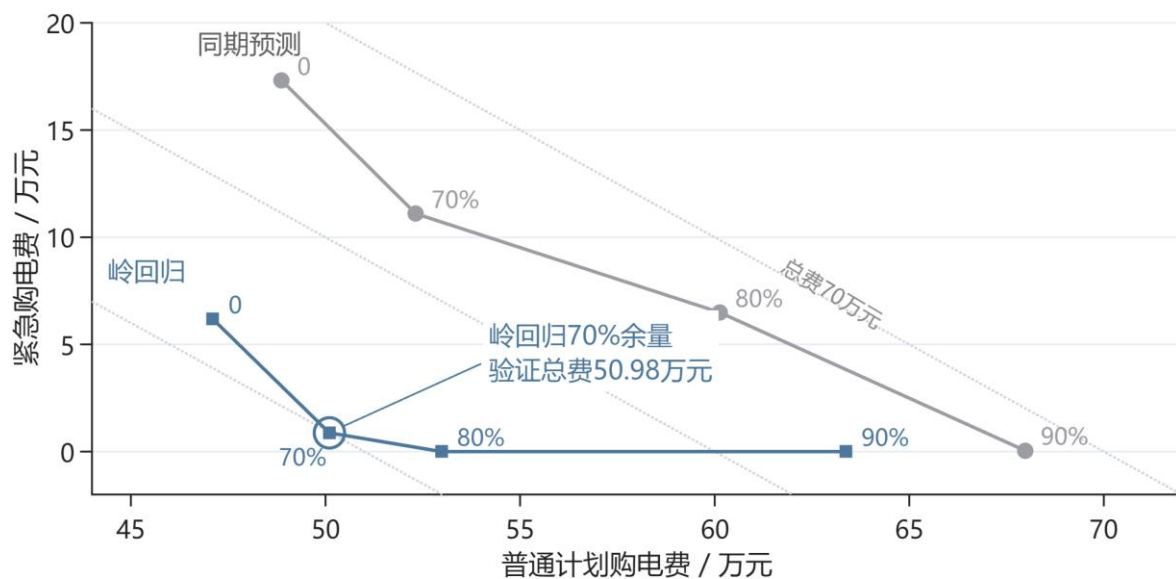


图 5 验证期风险余量与费用的权衡

图 5 将普通计划费与紧急费置于同一费用平面，虚线为总费用等值线，点旁数字为误差余量分位数。沿各预测器的四个候选增加余量，紧急支出下降而普通支出上升；圈出的岭回归 70%余量方案取得最低验证总费。

岭回归 70%余量的验证费为 509,757.47 元，其中紧急费 8,765.44 元；90%余量虽消除了验证期紧急购电，总费却升至 633,664.05 元。按总购电费用选择 70%余量，说明额外多购支出已超过进一步压缩紧急购电的收益。两方案期末储电量相差约 2,416 kWh，相关存量同时列于表 5；费用按题设结算，不计期末残值。

7.2 全期预测误差与季节变化

按表 6 比较 334 天共 48,096 个十分钟区间的功率预测误差，岭回归在负载预测上的改善大于光伏。四个指定日期的曲线对照见图 6，整年误差仍以全期统计为准。

表 6 第二问全期预测误差

预测器	负载 MAE	负载 RMSE	光伏 MAE	光伏 RMSE
同期	372.26	585.91	171.98	343.38
岭回归	179.86	234.59	155.28	306.74

表 6 的误差单位为 kW，光伏统计覆盖全天并包含夜间零值，表示全时段平均预测水平。

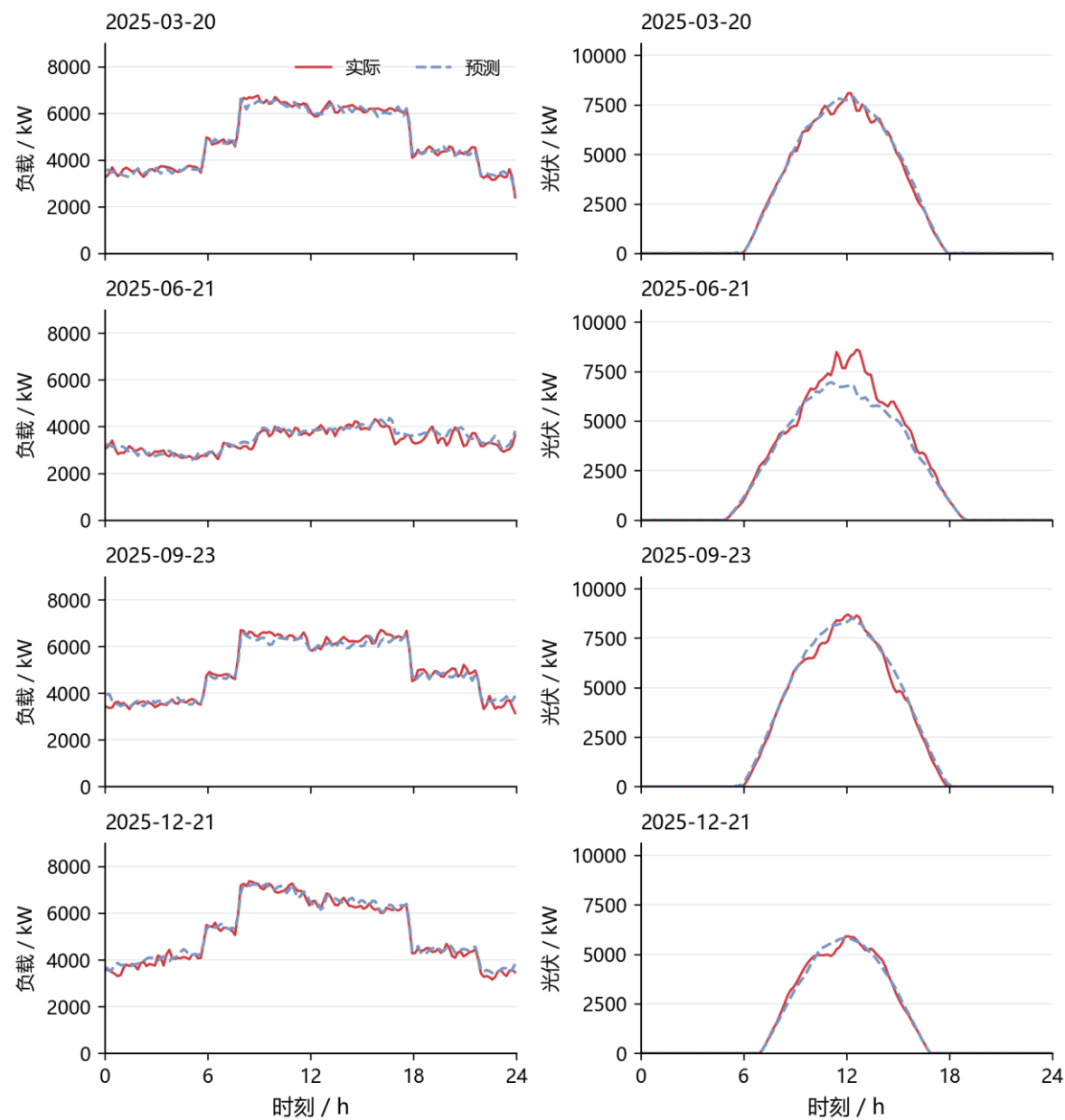


图 6 四个指定日期的负载与光伏日前预测

各列共用纵轴尺度，日期由题目指定。实线为实际观测，虚线为岭回归预测，展示十分钟分辨率下的预测偏差。

7.3 全期购电费用与运行约束

表 7 列出基础策略，图 7 按实际替换顺序展示从同期无余量到岭回归预测、70%风险余量及库存价值控制的费用变化。每一步均与前一步配对，直接给出各层改进的费用贡献。

表 7 第二问全期费用及期末储电量

策略	计划费/万元	紧急费/万元	总费/万元	期末 SOC/kWh
同期 无余量	1,199.19	723.59	1,922.78	5,825.99
岭回归 无余量	1,215.92	354.62	1,570.54	5,903.65
同期 70%余量	1,297.86	452.54	1,750.40	6,217.35
岭回归 70%余量	1,293.50	119.76	1,413.26	5,903.65

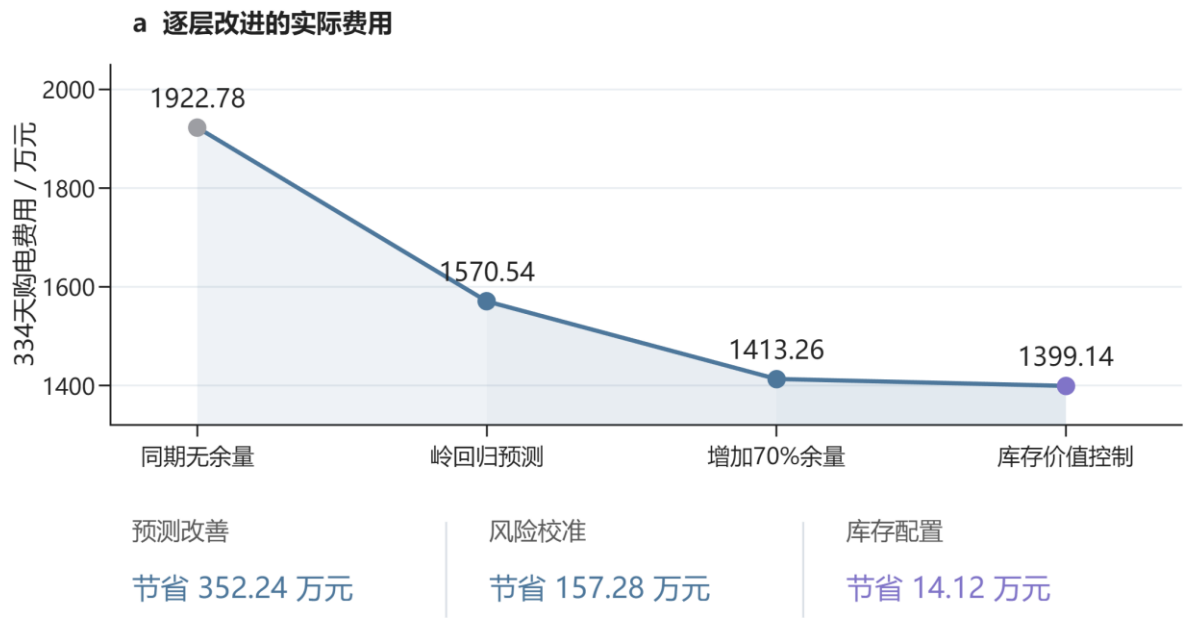


图 7 第二问逐层优化的费用变化与节费来源

图 7 的折线按模型实际替换顺序连接四个策略，末端为最终库存价值控制方案；下方分别列出预测改善、风险校准和库存配置带来的节省。各组件的费用贡献由相邻策略的同条件比较确定，体现预测、风险与控制的协同作用。

基础策略总费用为 14,132,612.16 元，较同期无余量降低 26.50%，较岭回归无余量降低 10.01%。岭回归两组的期末储电量同为 5,903.65 kWh，在相同预测器及初末状态下，10.01%的费用降幅直接反映风险余量的价值。

基础策略紧急购电累计 197,465.10 kWh，分布于 1,132 个十分钟区间；连续区间合并后统计为紧急购电事件。未利用电量为 2,211,229.49 kWh，由未消纳光伏与富余计划购电共同构成。风险余量通过增加部分普通购电，减少高价缺口补购。

四个指定日期的购电、充放电和紧急购电结果见附录 A。

7.4 库存价值控制的增益

在相同岭回归、70%余量、费用规则和初始储电量下，历史路径价值控制使 334 天费用由 14,132,612.16 元降至 13,991,392.60 元，节省 141,219.56 元（0.999%）。其中紧急购电费减少 141,170.39 元，而紧急电量由 197,465.10 增至 198,137.46 kWh。由此可见，收益来自补购与放电时点的重新分配：以较低价格补足当前缺口，将库存留给更高价格的后续缺口。两方案期末均为 5,903.65 kWh，库存转移的经济收益得到同状态对照支持。

负载同槽七日相关系数为 0.9855，日总量七日相关为 0.9880，周周期特征具有明确的数据依据。28、56、60、84 日训练窗的固定控制器敏感性费用分别为 14,211,345.15、14,152,541.05、14,132,612.16、14,033,154.46 元，说明窗口长度通过历史信息量与时效性共同影响预测。四种窗口在一月短历史下的验证结果相同，主方案保留 60 日窗，其余结果用于检验窗口敏感性。

八 第三问 日内预报的滚动价值

8.1 预报信息与决策时域

附件 3 含 365 天、每日 4 个版本、每版 24 个整点光伏预报，共 35,040 个值。按四行组还原 1,095 个结构性空日期后，未发现缺失、非有限数或负值。预报发布时间分别为 0、6、12、18 点；时刻 τ 的新预报只服务于 τ 之后的调度。小时预报端点之间采用分段线性函数，并在十分钟区间上积分为预测电量；发布点以最近完成区间的光伏观测衔接。

这一处理保留了预报更新带来的信息差异。每次求解均以当时真实储电量为起点，固定已经执行的购电与充放电量，重新配置其余区间。该时域推进方式与储能模型预测控制的基本框架一致[4]。

图 8 给出指定日期的实际光伏及各版预报，各预报从对应发布时间起展示。日内预报对剩余时段的光伏判断持续更新，为重配购电和储能提供信息。

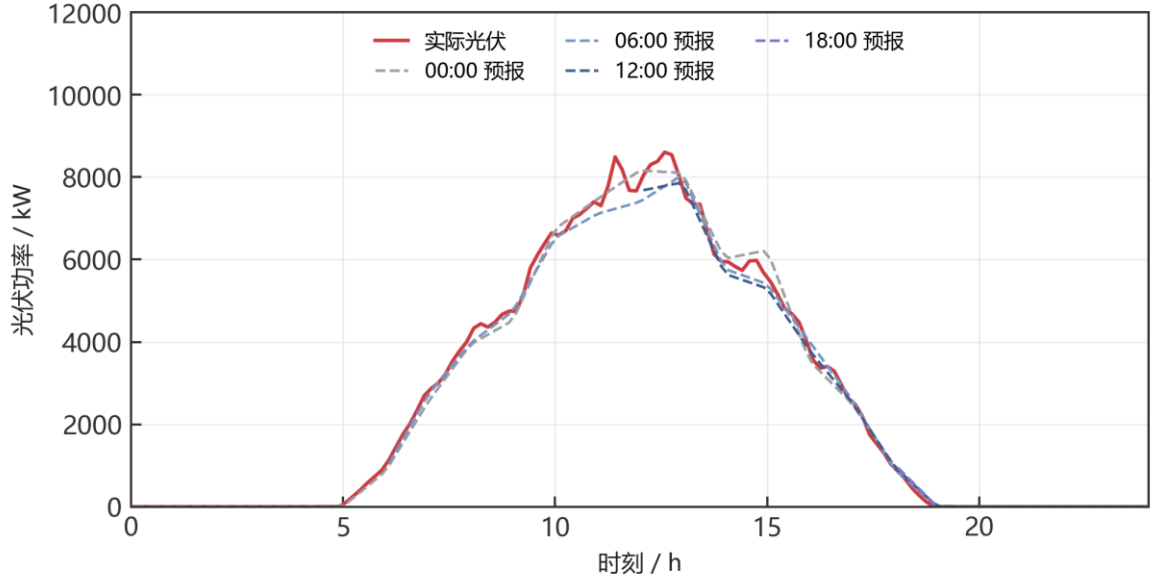


图 8 四个发布版本的光伏预报

8.2 保留量、取消量与新增量的结算

记午夜计划为 x_t ，时段执行前最后一次有效调整量为 y_t ，紧急购电为 e_t 。取消量 u_t 与新增量 v_t 分别定义为：

$$u_t = (x_t - y_t)^+, \quad v_t = (y_t - x_t)^+, \quad y_t = x_t + v_t - u_t. \quad (27)$$

保留购电按原价结算，取消部分按半价支付违约金，新增部分按 1.5 倍价格购买。因此每个区间的完整费用为：

$$F_t = p_t \min(x_t, y_t) + 0.5p_t u_t + 1.5p_t v_t + 5p_t e_t = p_t x_t - 0.5p_t u_t + 1.5p_t v_t + 5p_t e_t. \quad (28)$$

原计划 100 kWh、调整为 80 kWh、电价 1 元/kWh 时，总费用为 $80 + 0.5 \times 20 = 90$ 元；调整为 120 kWh 时，费用为 $100 + 1.5 \times 20 = 130$ 元。前式按保留量计普通费用，后式按原计划记账，两种表达完全等价。多次预报更新均相对同一午夜计划确定偏差；中途计划用于运行与追溯，不重复累计尚未交付区间的修改费用。

8.3 滚动优化模型与风险校准

记 T_τ 为当前尚未执行的区间集合， Ω 为第一问去除首末状态等式后的物理可行域。以更新后的光伏预测、负载预测和对应误差余量作为供需输入，求解：

$$\begin{cases} \min_{y, c, d, w, S, u, v} & \sum_{t \in T_\tau} \hat{p}_{\tau, t} (1.5v_t - 0.5u_t) \\ \text{s. t.} & y_t = x_t + v_t - u_t, \quad 0 \leq u_t \leq x_t, \quad v_t \geq 0, \\ & y_t + \hat{V}_{\tau, t} + d_t = \hat{L}_{\tau, t} + r_{\tau, t} + c_t + w_t, \\ & (y, c, d, w, S) \in \Omega, \quad S_\tau = S_\tau^{\text{actual}}, \quad S_{145} \geq 6000. \end{cases} \quad (29)$$

目标函数省略了与当前决策无关的午夜合同常数项。第一至三问的 p 由附件 1 给定；第四问替换为当时可获得的价格预测。名义规划满足供电需求，基础执行按第二问即时平衡规则运行，改进执行按第 8.6 节估计后续库存价值。

风险余量取此前 28 个完整日期、相同发布版本及目标前后三个区间的净负载误差。将无余量、50%分位余量和 70%分位余量分别与 8 种更新时间组合配对，共比较 24 组方案。1 月 22—31 日验证选定 50%分位余量及 6、12、18 点更新，随后在 2—12 月评价。负载预测器和名义日末目标保持一致，以分离预报更新与调整机制的影响。

图 9 以一月验证数据比较风险余量与更新时间。左图给出 24 组候选的费用矩阵，黑框标出 50%余量、三次日内更新的选定方案；右图沿 0、6、12、18 点的顺序增加预报，展示信息更新对验证费用的影响。

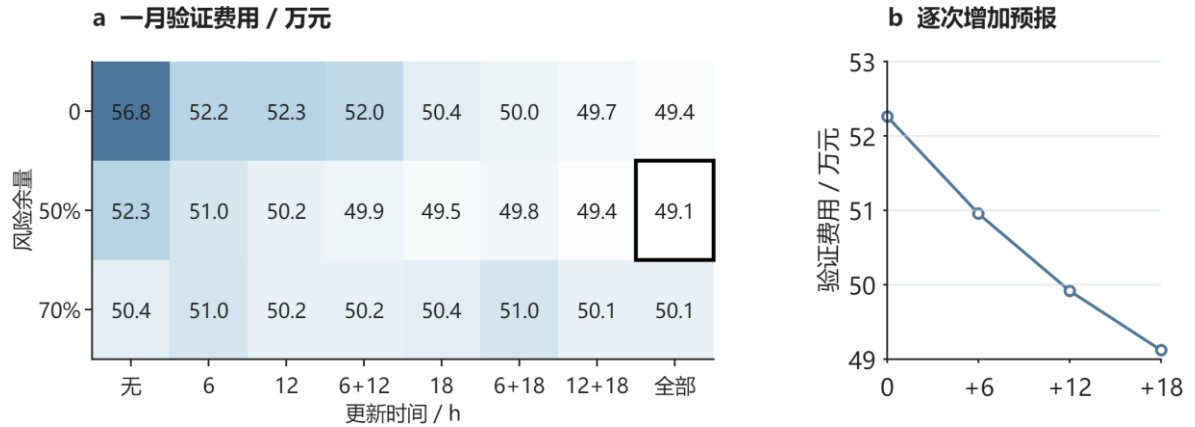


图 9 风险余量与预报更新频次的验证比较

8.4 基础滚动策略的更新时间收益

图 10 在同一 50%余量下比较八种更新时间组合。左侧蓝格表示启用该时刻预报，右侧条形给出相对仅午夜预报的全年节费；下方汇总基础滚动、负载选择与库存价值控制的最终费用。

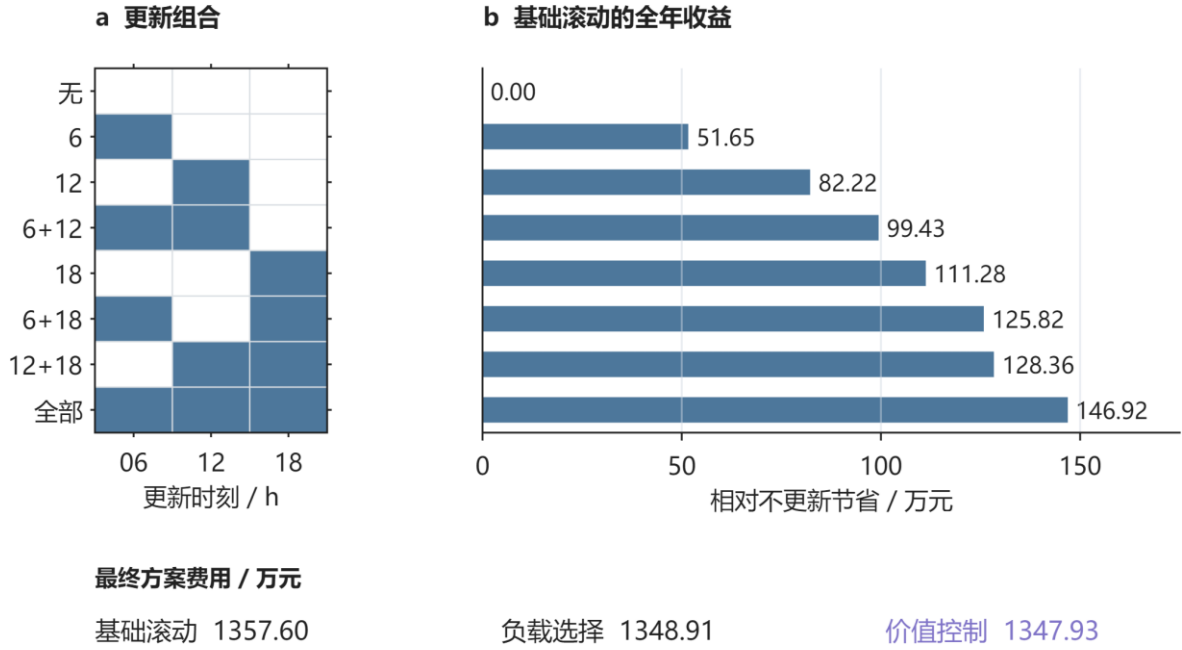


图 10 预报更新组合的全年收益与第三问最终费用

表 8 将两方案费用分解为午夜计划、净调整及紧急补购三部分。

表 8 第三问基础策略与仅午夜预报的费用分解

费用或运行量	仅午夜预报	日内滚动
原计划费/元	12,530,137.77	12,521,239.63
新增购电费/元	0.00	678,211.30
取消量净费用/元	0.00	-203,600.86
紧急购电费/元	2,514,989.16	580,126.71
总费用/元	15,045,126.93	13,575,976.78
紧急电量/kWh	407,324.31	101,019.32
期末储电量/kWh	5,903.65	5,903.65

在相同 50%余量下，三次更新将总费用由 15,045,126.93 元降至 13,575,976.78 元，净节省 1,469,150.15 元，降幅 9.76%；实际紧急购电量降至 101,019.32 kWh。新预报使部分供电缺口提前通过计划调整解决，紧急补购费用的下降足以覆盖调整支出。因此，本题有必要引入零点之外的预报。两方案初末储电量相同，节费来自运行策略改善。

8.5 面向提前时长的负载修正与月度模型选择

仅在零点预测负载，会忽略日内已观测偏差的持续性。设零点预测误差为实际负载减零点预测，以每次发布前 18 个十分钟区间的平均误差作为当前偏差 x ；历史目标小时的六个区间平均误差作为响应 y 。分别对 6、12、18 点及各未来目标小时拟合非负收缩回归：

$$\hat{\beta}_{d,v,h} = \text{clip}_{[0,1.5]} \left(\frac{\sum_{i \in H_d} x_{i,v} y_{i,v,h}}{1.2 \sum_{i \in H_d} x_{i,v}^2} \right), \quad H_d = \{\max(7, d-28), \dots, d-1\}. \quad (30)$$

其中日期从 0 编号；自第 14 日起启用，至少使用 7 个完整历史日；分母为零时系数取零。1.2 倍分母对应 20% 的平方误差尺度惩罚，系数上限 1.5 限制短样本放大。修正后的预测为：

$$\hat{L}_{d,v,t}^A = \max\{0, \hat{L}_{d,t}^0 + \hat{\beta}_{d,v,h(t)} x_{d,v}\}. \quad (31)$$

同一目标小时共享修正量，零点预测不变。50% 风险余量仍从该预测方案此前 28 天的历史误差计算，避免沿用旧误差分布而重复补偿。表 9 比较全期各发布时刻的负载预测误差，单位均为 kWh/十分钟。

表 9 负载自适应修正的同目标误差对照

发布时刻	基础 MAE	修正 MAE	降幅
06:00	30.26	27.96	7.61%
12:00	30.98	28.85	6.85%
18:00	31.58	27.68	12.34%

固定启用修正在 1 月 22—31 日使第三、四问滚动验证费分别增加 972.85 元和 1,149.73 元，因此不能依据后续全年收益回选一月方案。本文在验证结束后固定一项月度更新规则：2 月使用一月选定的基础预测；3 月起，每月首日分别回放此前 28 天的基础与修正方案，两者从该窗口起点的实际储电量出发，按历史真实费用选择下月方案，平局保留基础方案。候选模型、窗口、收缩系数与费用口径在全年评价前固定，月底真实数据仅在次月选择时可见。

两类电价情景均在 3、6、7、8、9、10 月启用修正，在其余月份使用基础预测。实际储电量连续跨月传递，历史回放的末状态不覆盖实时状态。全年固定启用修正的费用另作消融结果保存，其收益不作为回选规则。该比较属于同一年度数据上的顺序开发验证，并非未接触过的外部测试集。

事后对月度验证末端库存作敏感性检查：将每 kWh 储电的价值从 0 变化至样本最高紧急电价折算值，20 次模型选择均保持不变，费用排序不依赖低估候选方案的剩余储电量。

8.6 计入下次调单机会的库存价值

直接把第 6.5 节固定合同价值函数用于滚动执行，会将后续全部缺口都按五倍紧急价计值，忽略下次发布后可新增普通合同。该方法在一月验证中增加第三问费用 4,210.28 元。为修正这一结构偏差，在当前至下次发布的区间仍按五倍价估计缺口，下一发布之后则以 1.5 倍预测价近似可新增计划的边际费用；每次发布重新构造价值函数并只执行随后 36 个时段。1.5 倍仅用于后续价值估计，实际紧急账单始终按五倍价支付。

修正方案一月费用 491,182.82 元，较即时平衡低 29.11 元；据此选定后，全年费用由 13,489,127.94 元降至 13,479,283.32 元，再省 9,844.62 元。固定合同价值法全年反而增加 149,493.07 元，验证了调单机会必须进入价值估计。该近似未显式求解未来调单、取消与库存的完整联合场景问题，其作用由同口径顺序账单检验。

为单独识别执行器贡献，两执行方案共享第 8.5 节因果负载选择信号；该信号由基础执行器的历史影子回放生成，实际库存分别连续传递，不用改进方案的未来费用重选预测模型。

九 第四问 GRU 预测与价格信息的优化空间

9.1 波动电价与共同预测信息

附件 4 给出 365 天、每天 144 个电价，共 52,560 点，范围为 0.0076—1.7936 元/kWh。所有价格保留原值，不对峰谷截尾。各预测器使用发布前的价格历史以及已知日历信息，输出未来 144 个十分钟目标；当日调度只读取其中尚未执行的区间。

图 11 展示全年电价的日内结构与月度分布。峰谷时段决定储能转移方向，价格幅度及季节变化影响各时段购电规模。

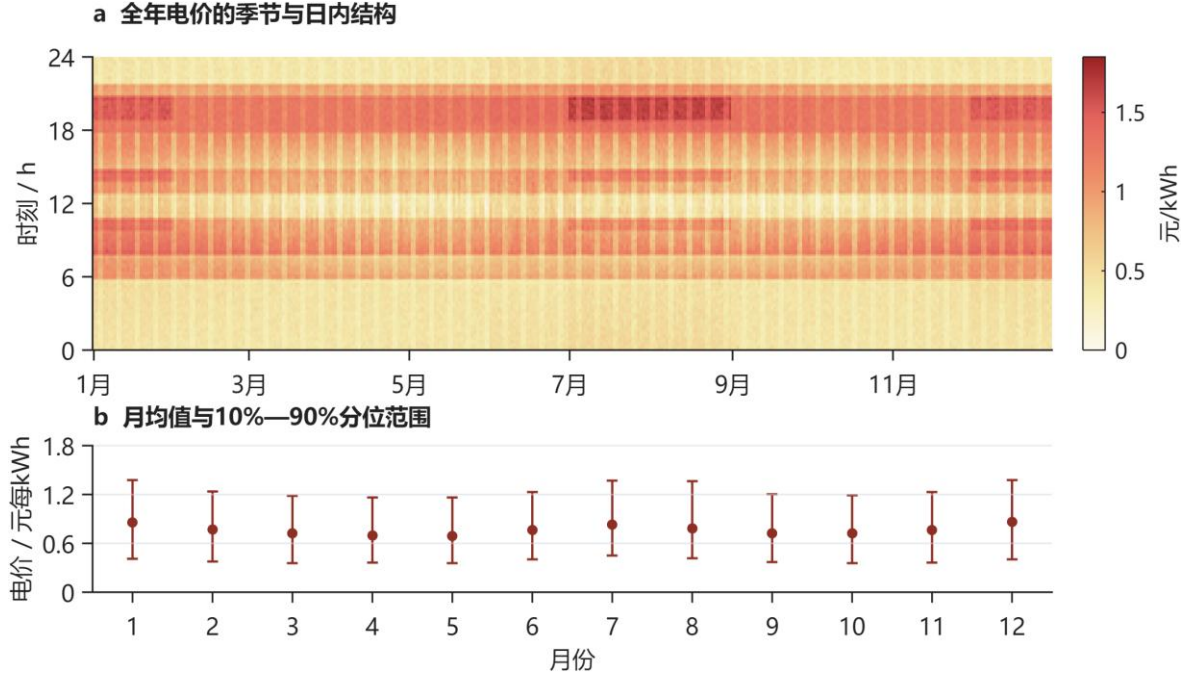


图 11 全年电价的时刻结构与月度分布

图 11a 以十分钟原始记录展示电价的日内峰谷及季节差异，颜色表示电价，范围覆盖全部观测。图 11b 的点为各月均价，线段为该月全部十分钟电价的 10%—90%分位范围，表示分布离散程度。同期预测取昨日与上周同目标区间价格的平均值。岭回归在这两组历史价格、日历周期特征及滞后统计量上构造 12 维输入。残差 GRU 使用 6 维输入：昨日价、上周价、日内正余弦和星期正余弦。三类预测器共享历史可见范围，分别表达周期基线、线性修正与非线性时序依赖。

$$\hat{p}_{\tau,j}^{\text{seasonal}} = \frac{p_{\tau+j-144} + p_{\tau+j-1008}}{2}, \quad j = 0, \dots, 143. \quad (32)$$

9.2 残差 GRU 的结构与训练

GRU 以门控结构选择保留的历史信息[6]。本文采用单层 32 维隐藏状态，在同期基线之上预测残差，输出头从零初始化，使初始预测与同期方法一致。输入维度较小、序列固定为 144 步，有利于在有限历史数据上控制训练规模。网络按 PyTorch 的 GRU 形式实现[5]：

$$r_j = \sigma(W_{ir}x_j + b_{ir} + W_{hr}h_{j-1} + b_{hr}), \quad z_j = \sigma(W_{iz}x_j + b_{iz} + W_{hz}h_{j-1} + b_{hz}). \quad (33)$$

$$n_j = \tanh(W_{in}x_j + b_{in} + r_j \odot (W_{hn}h_{j-1} + b_{hn})), \quad h_j = (1 - z_j) \odot n_j + z_j \odot h_{j-1}. \quad (34)$$

$$\hat{p}_{\tau,j} = \max\{0.001, \hat{p}_{\tau,j}^{\text{seasonal}} + s_y(w^T h_j + b)\}. \quad (35)$$

前两个价格特征的均值和标准差仅从训练集估计， s_y 为训练价格标准差。网络采用 SmoothL1 损失与 Adam 优化器，学习率 0.003，批量 32，梯度范数截断为 1；最多训练 50 轮，内部验证连续 6 轮无改进即早停。输入窗口每 6 小时取样，只有全部预测目标已发生的窗口进入训练。

1 月 15 日、22 日和之后每月首日重新训练，每次最多使用此前 60 天数据；内部验证为最近 3 个完整日期，跨越训练—验证边界的目标窗口不进入训练。以 17、42、2026 为三个基种子，实际随机种子为基种子加拟合日编号，分别训练，最终等权平均，共 39 次拟合，原实验 CPU 训练约 126.56 秒。39 份权重重新加载后的预测与缓存逐值一致；同期预测由原始价格重建，岭回归 13 次重新拟合亦与缓存一致。图中的训练曲线展示一次月初拟合的内部验证过程。

图 12 展示同一月初拟合中三个网络随训练轮数变化的内部验证 MAE；早停分别保留各自最优权重。

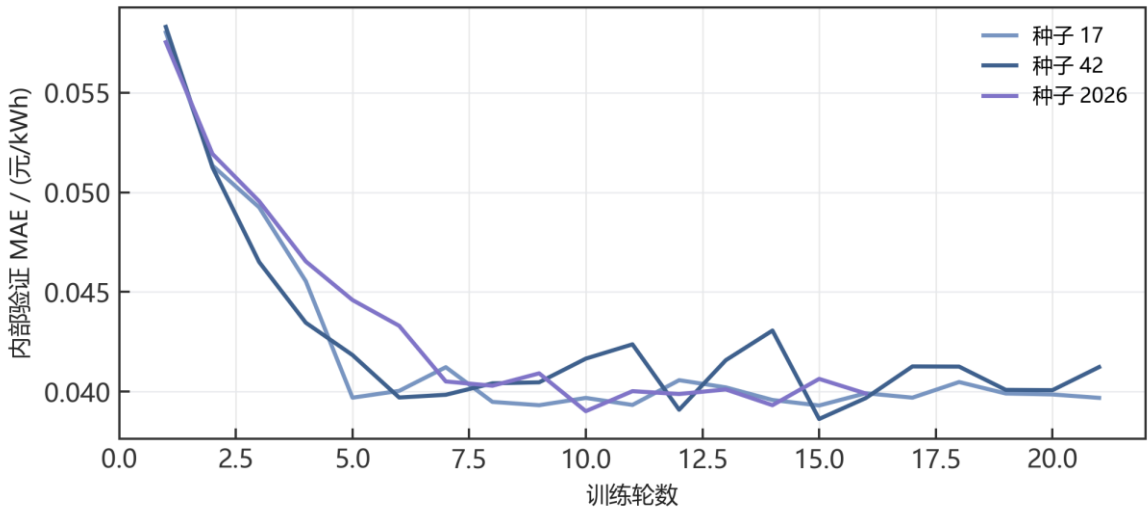


图 12 三个随机种子的 GRU 验证误差

9.3 四个发布时刻的预测优势

图 13 展示各发布时刻相同目标范围上的 MAE。GRU 在四组比较中均取得最低误差。

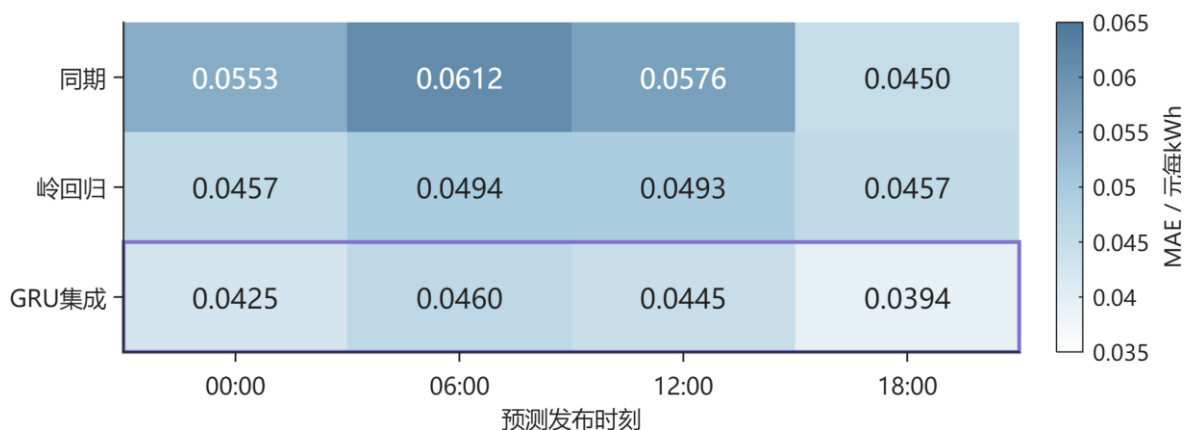


图 13 四个发布时刻的价格预测 MAE

表 10 按发布时刻列出价格 MAE 及 GRU 相对岭回归的降幅。

表 10 各发布时刻价格 MAE 及 GRU 相对岭回归的改进

发布时刻	同期	岭回归	GRU 集成	改进比例
00:00	0.05527	0.04567	0.04253	6.88%
06:00	0.06117	0.04938	0.04603	6.78%
12:00	0.05761	0.04928	0.04448	9.74%
18:00	0.04499	0.04566	0.03941	13.68%

按全部发布一目标配对加权，GRU 集成 MAE 为 0.04366 元/kWh，较同期预测降低 22.70%，较岭回归降低 8.09%。共 120,240 个配对，其中同一目标可由不同发布时刻预测；分时评价保证各模型比较的是相同信息时点。

图 14 比较实际电价与岭回归、GRU 的逐时预测。曲线显示两类模型对日内变化的拟合差异；未被预测捕捉的价格变化最终反映在实际结算费用中。

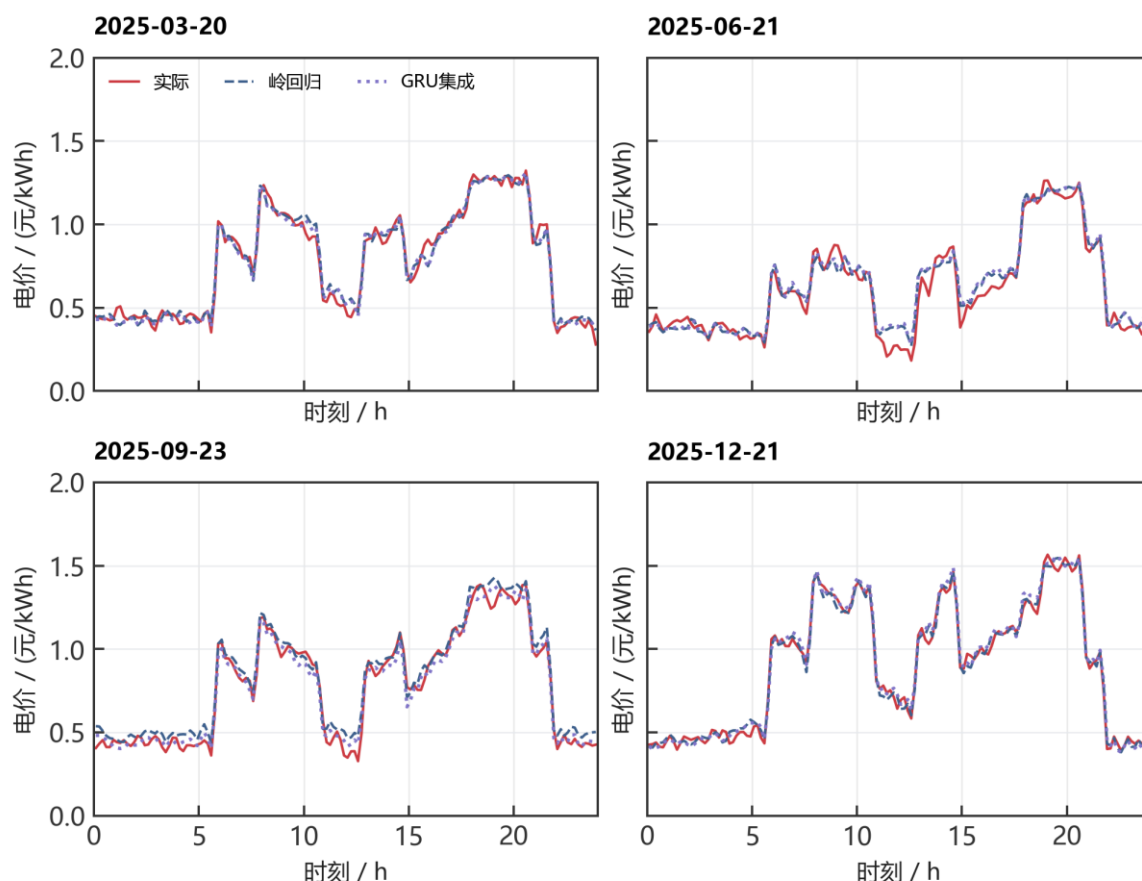


图 14 四个指定日期的午夜价格预测

9.4 基础调度下的价格预测收益

表 11 在日前与滚动两类策略内分别比较各价格预测器的实际费用。

表 11 波动电价下两类策略的全年费用

价格预测器	日前总费/元	滚动总费/元
同期预测	14,867,653.20	14,314,442.48
岭回归	14,880,535.05	14,320,233.59
GRU 集成	14,877,217.55	14,312,347.70
真实电价对照	14,818,406.37	14,212,407.27

按一月验证总费用，日前基础策略选取岭回归电价预测，滚动基础策略选取同期预测，2—12 月费用分别为 14,880,535.05 元和 14,314,442.48 元，后者减少 566,092.57 元（3.80%）。固定滚动调度规则后，GRU 费用为 14,312,347.70 元，较同期预测节省 2,094.78 元，较岭回归节省 7,885.90 元。两类基础策略比较反映整体方案差异，同一滚动规则下的比较则分离出价格预测器的贡献。

图 15a、b 在相同基础执行器内比较价格预测器的节费贡献，零点右侧为节省、左侧为增费。图 15c、d 进一步给出库存价值控制相对各自即时平衡基线的月度节费及全年区间，将预测模块和执行模块的经济作用分别呈现。

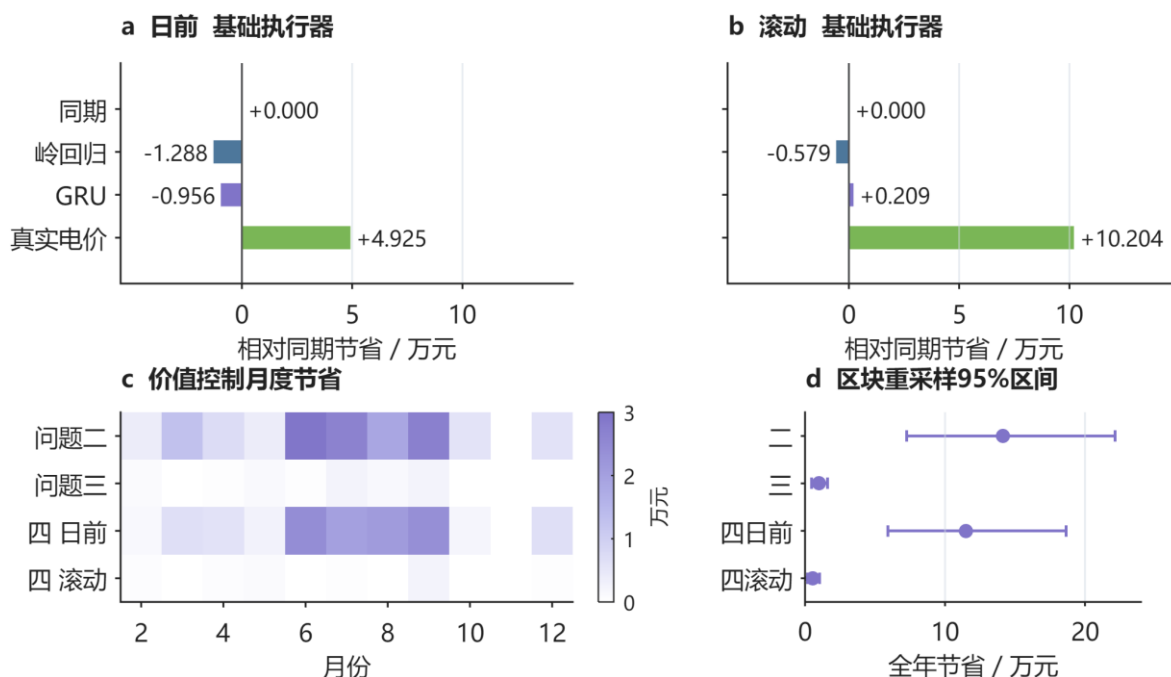


图 15 价格预测与库存价值控制的费用贡献

GRU 改善了四个发布时刻的价格预测精度，其费用贡献取决于预测修正是否改变可行调度动作。固定基础滚动执行器后，GRU 相对同期节省 2,094.78 元，配对区块重采样区间跨零；因此按一月验证费用保留原价格模型，并在执行层引入库存价值控制。图 15d 采用 2,000 次、14 天配对区块重采样，点为全年节费，线为 95%区间；四个分支分别比较，不叠加为同一系统收益。最终费用见表 13。

9.5 名义最优下界与剩余改进上限

为直接回答价格预测还有多大优化空间，对每个日期固定相同的负载与光伏预测、风险余量、日初状态和日末目标，得到共同可行域 $\mathcal{F}_d$ 。各预测器只改变目标函数中的价格；事后将真实电价代入同一可行域求解线性规划[7]：

$$J_d^* = \min_{g \in \mathcal{F}_d} p_d^T g, \quad J_{m,d} = p_d^T g_{m,d}, \quad g_{m,d} \in \mathcal{F}_d. \quad (36)$$

由于真实电价解在相同可行域内最优，任一预测器的可行计划都满足 $J_{m,d} \geq J_d^*$ 。因此模型 m 仅通过改善价格预测能够获得的名义费用节省 Δm 受到下式约束：

$$0 \leq \Delta_m \leq R_m = \sum_d (J_{m,d} - J_d^*), \quad \gamma_m = \frac{R_m}{\sum_d J_{m,d}} \times 100\%. \quad (37)$$

表 12 列出共同名义可行域内的真实电价最优费用及各预测器的费用差距。

表 12 相同名义可行域中的真实价格下界

预测器	名义费用/元	真实价格下界/元	剩余差额/元
同期	13,650,909.99	13,519,514.66	131,395.33
岭回归	13,663,504.91	13,519,514.66	143,990.25
GRU 集成	13,656,357.29	13,519,514.66	136,842.64

图 16a 累计 334 个共同名义可行域上的逐日最优值差距，图 16b 汇总全年剩余改进上限。GRU 的名义费用距真实电价最优值 13.68 万元，占其名义费用 1.002%；在既定供需预测和储能约束下，进一步提高价格预测精度的费用改善空间已接近这一量级。

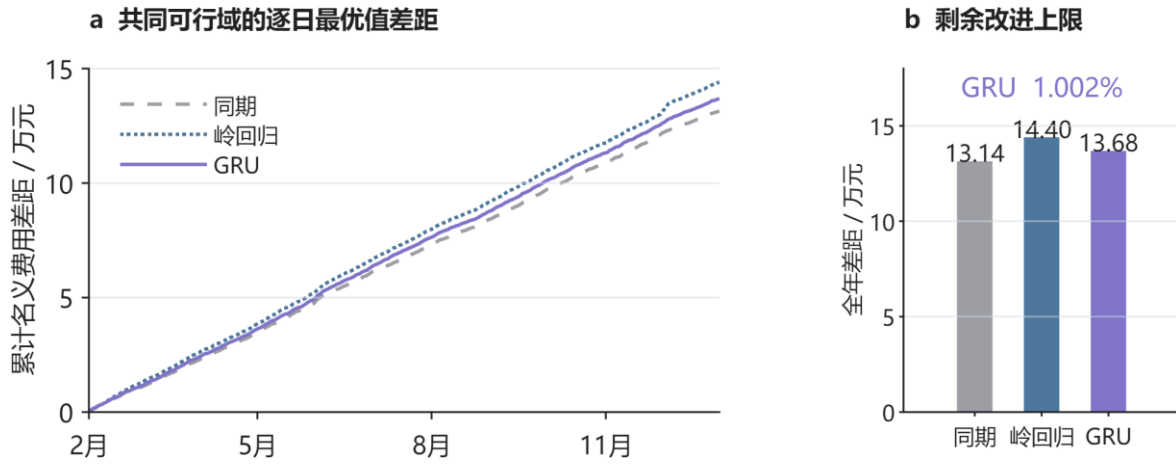


图 16 价格预测的累计名义差距与剩余改进上限

GRU 的名义计划费用为 13,656,357.29 元，真实电价最优下界为 13,519,514.66 元；由未舍入值计算，两者相差 136,842.64 元，占 GRU 名义费用的 1.002%。在固定供需预测、余量及储能条件下，即使完全消除电价预测误差，名义费用的进一步下降也受此差额限制。同期与岭回归处于相近量级，表明三类预测器形成的名义购电计划均已接近该共同可行域的最优值。

在固定实时执行规则下，使用真实电价进行事后调度的滚动费用为 14,212,407.27 元，较同期预测降低 0.713%。这一结果包含供需误差与紧急购电，衡量特定执行规则下的反事实收益；1.002% 的上限则仅约束共同名义可行域内的计划费用。结合风险余量与滚动更新的对照，当前优化应优先改进供需预测和储能控制，再细化电价预测。

9.6 波动电价下的库存价值与最终费用

日前策略在午夜以岭回归预测价格构造固定合同后续价值，执行时仅揭示当前交易电价；滚动策略采用同期价格预测并按第 8.6 节计入下次调单机会。一月验证分别节省 593.25 元和 99.55 元，随后保持执行器规则，全年日前费用为 14,765,492.68 元，滚动费用为 14,210,881.01 元，较仅含月度负载选择的对应方案分别节省 115,042.37 元和 5,444.97 元。

因此，四问共享“库存状态—可见信息—合同费用”的决策结构：第一问精确求值，第二问引入供需路径，第三问计入后续调单，第四问增加价格预测。库存价值随合同权限改变，统一结构不要求各问使用相同的近似函数。

十 模型检验与评价

10.1 数值验证与结论稳健性

本次逐时复核 11 条 334 天候选轨迹，共 529,056 个十分钟时段，原始供需、功率电量换算、跨日库存、互斥动作及合同费用均一致。另以消去状态变量的累计库存 LP 独立重建 24 个指定日滚动实例，最优值与对偶证书均通过；连同基础实验 109 例独立列式，支持名义优化层的最优性。连续 DP 经九个初始库存的 MILP 及随机算例交叉核对；扰动未来实际供需或电价不改变当前动作。

四组改进均保持相同初末库存。以日节费作 2,000 次 14 天配对区块重采样，第二问、第三问、第四问日前及滚动的 95% 区间依次为 [72,476.63, 221,616.33]、[4,558.94, 15,996.50]、[59,151.20, 186,368.40]、[1,953.22, 10,500.59] 元，7 与 28 天区块下也均为正。这些是同年固定轨迹的条件重采样结果，不校正反复开发与模型选择造成的偏差；跨年推广仍需独立数据。

十分钟平均功率与线性效率忽略区间内快速波动、退化和爬坡。确定性 DP 精确求解所列模型，历史路径均值及滚动 1.5 倍续期成本则是后续价值近似；日末名义 6,000 kWh 目标仍为工作假设。后续应以跨年数据检验策略，并在多阶段场景树中联合处理未来调单与跨日终端价值。

十一 结论

表 13 汇总四问交付结果及库存价值控制增益。基础列与改进列采用相同费用规则和初末库存；图 7、图 10 及图 15 共同展示最终方案、收益来源和月份分布，指定日期的购电及储能结果见附录 A。

表 13 四问交付结果与库存价值增益

问题	交付策略	基础费用/元	改进费用/元	节省/元
一	LP / 连续 DP	35,126.85	35,126.85	0.00
2	配对路径价值	14,132,612.16	13,991,392.60	141,219.56
3	可调单价值+负载选择	13,489,127.94	13,479,283.32	9,844.62
4-2	岭价格+配对路径价值	14,880,535.05	14,765,492.68	115,042.37
4-3	同期价格+可调单价值	14,216,325.98	14,210,881.01	5,444.97

第一问通过 LP 与连续 DP 得到一致的最优费用 35,126.85 元，确立确定性调度基准。第二问结合风险校准与库存价值控制，将全年费用降至 13,991,392.60 元；第三问进一步利用日内预报、负载选择与调单机会，费用降至 13,479,283.32 元。波动电价下，日前与滚动费用分别为 14,765,492.68 元和 14,210,881.01 元。四问结果共同表明，微网优化的关键在于根据后续缺口价格和合同调整机会配置库存，并以完整购电账单评价信息与控制的价值。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 C 题 微网与外部电网电力调控策略及附件.
- [2] SciPy Developers. `scipy.optimize.linprog` and `scipy.optimize.milp`.
<https://docs.scipy.org/doc/scipy/reference/optimize.html>. 访问日期 2026-09-11.
- [3] scikit-learn Developers. Ridge. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html. 访问日期 2026-09-11.
- [4] Morstyn T, Hredzak B, Aguilera R P, Agelidis V G. Model Predictive Control for Distributed Microgrid Battery Energy Storage Systems. *IEEE Transactions on Control Systems Technology*, 2018, 26(3):1107–1114. DOI:10.1109/TCST.2017.2699159.
- [5] PyTorch Contributors. GRU. <https://docs.pytorch.org/docs/2.14/generated/torch.nn.GRU.html>. 访问日期 2026-09-11.
- [6] Cho K, van Merriënboer B, Gulcehre C, et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *EMNLP*, 2014. arXiv:1406.1078.
- [7] Boyd S, Vandenberghe L. *Convex Optimization*. Cambridge University Press, 2004.
<https://web.stanford.edu/~boyd/cvxbook/>.

附录 A 四问指定日期完整结果

电量单位为 kWh，费用单位为元。普通购电与紧急购电分列。滚动问题的指定时段和全天购电量指最终有效普通计划；午夜计划及每次更新完整保存在 Excel 中。所有表由实际轨迹自动生成。

A.1 问题 1

对应结果见表 14。

表 14 问题 1（单日）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	486.40	14:00-14:10	0.00
16:00-16:10	394.93	18:00-18:10	636.98	20:00-20:10	0.00
全天购电量 59,482.70 kWh			全天购电费 35,126.85 元		

对应结果见表 15。

表 15 问题 1（单日）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,500.00	0.00	04:00-08:00	833.33	5,947.42
08:00-12:00	3,954.63	2,121.42	12:00-16:00	6,119.37	91.10
16:00-20:00	0.00	5,068.69	20:00-24:00	5,333.33	3,571.31
00:00 储电	6,000.00		24:00 储电	6,000.00	

A.2 问题 2

对应结果见表 16。

表 16 问题 2（2025-03-20）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	574.14	14:00-14:10	0.00
16:00-16:10	488.93	18:00-18:10	720.76	20:00-20:10	0.00
全天购电量 68,036.47 kWh			全天购电费 41,302.44 元		

对应结果见表 17。

表 17 问题 2（2025-03-20）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,071.86	209.79	04:00-08:00	1,038.99	5,515.86
08:00-12:00	6,186.28	2,777.38	12:00-16:00	4,162.63	444.57
16:00-20:00	266.98	5,654.71	20:00-24:00	5,687.88	2,954.47
00:00 储电	6,617.05		24:00 储电	6,382.68	

对应结果见表 18。

表 18 问题 2（2025-03-20）紧急购电

时段	电量
17:20-17:30	5.19

对应结果见表 19。

表 19 问题 2（2025-06-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	210.92	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 36,315.40 kWh		全天购电费 21,120.39 元			

对应结果见表 20。

表 20 问题 2（2025-06-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	526.76	524.94	04:00-08:00	413.77	4,402.74
08:00-12:00	9,426.35	0.00	12:00-16:00	0.00	0.00
16:00-20:00	151.96	5,493.83	20:00-24:00	6,132.83	2,497.81
00:00 储电	6,945.01		24:00 储电	7,576.71	

对应结果见表 21。

表 21 问题 2（2025-06-21）紧急购电

时段	电量
无	0.00

对应结果见表 22。

表 22 问题 2（2025-09-23）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	491.40	14:00-14:10	0.00
16:00-16:10	549.29	18:00-18:10	788.73	20:00-20:10	0.00
全天购电量 71,561.09 kWh		全天购电费 44,415.98 元			

对应结果见表 23。

表 23 问题 2（2025-09-23）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,252.67	0.00	04:00-08:00	218.14	5,830.96
08:00-12:00	5,558.85	1,896.62	12:00-16:00	4,033.40	575.17
16:00-20:00	231.66	5,642.21	20:00-24:00	6,050.86	2,841.96
00:00 储电	6,072.60		24:00 储电	6,631.50	

对应结果见表 24。

表 24 问题 2（2025-09-23） 紧急购电

时段	电量
20:20-20:30	0.33

对应结果见表 25。

表 25 问题 2（2025-12-21） 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	967.61	14:00-14:10	0.00
16:00-16:10	812.86	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 97,145.51 kWh			全天购电费 62,434.14 元		

对应结果见表 26。

表 26 问题 2（2025-12-21） 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,629.25	143.74	04:00-08:00	1,398.62	1,538.91
08:00-12:00	5,704.82	7,168.67	12:00-16:00	6,940.07	2,243.11
16:00-20:00	24.16	5,708.86	20:00-24:00	5,695.76	2,842.53
00:00 储电	6,318.60		24:00 储电	6,443.34	

对应结果见表 27。

表 27 问题 2（2025-12-21） 紧急购电

时段	电量
07:50-08:00	23.78

A.3 问题 3

对应结果见表 28。

表 28 问题 3（2025-03-20） 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	520.67	14:00-14:10	0.00
16:00-16:10	638.18	18:00-18:10	714.94	20:00-20:10	0.00
全天购电量 66,401.94 kWh			全天购电费 42,938.13 元		

对应结果见表 29。

表 29 问题 3（2025-03-20） 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,249.82	348.06	04:00-08:00	1,010.91	5,813.27
08:00-12:00	3,254.96	2,698.33	12:00-16:00	6,483.14	254.72
16:00-20:00	646.16	5,206.99	20:00-24:00	5,530.12	3,018.64
00:00 储电	6,309.41		24:00 储电	6,100.33	

对应结果见表 30。

表 30 问题 3（2025-03-20） 紧急购电

时段	电量
09:40-10:00	79.05
21:40-21:50	4.66

对应结果见表 31。

表 31 问题 3（2025-06-21） 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	120.43	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 34,874.69 kWh			全天购电费 19,723.09 元		

对应结果见表 32。

表 32 问题 3（2025-06-21） 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	275.76	271.67	04:00-08:00	449.87	3,308.89
08:00-12:00	9,888.78	0.00	12:00-16:00	0.00	0.00
16:00-20:00	41.37	6,074.66	20:00-24:00	5,603.72	2,654.17
00:00 储电	5,225.44		24:00 储电	6,181.87	

对应结果见表 33。

表 33 问题 3（2025-06-21） 紧急购电

时段	电量
无	0.00

对应结果见表 34。

表 34 问题 3（2025-09-23） 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	347.56	14:00-14:10	0.00
16:00-16:10	522.47	18:00-18:10	784.63	20:00-20:10	0.00
全天购电量 67,918.94 kWh			全天购电费 44,580.19 元		

对应结果见表 35。

表 35 问题 3（2025-09-23） 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,885.98	16.12	04:00-08:00	353.76	6,967.82
08:00-12:00	4,885.84	1,675.86	12:00-16:00	6,129.08	508.84
16:00-20:00	118.44	5,841.36	20:00-24:00	5,814.52	2,695.27
00:00 储电	6,102.47		24:00 储电	6,398.80	

对应结果见表 36。

表 36 问题 3（2025-09-23） 紧急购电

时段	电量
09:00-09:50	220.76
20:20-20:30	6.37

对应结果见表 37。

表 37 问题 3（2025-12-21） 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	924.13	14:00-14:10	0.00
16:00-16:10	821.56	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 95,721.86 kWh			全天购电费 62,219.35 元		

对应结果见表 38。

表 38 问题 3（2025-12-21） 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,657.30	179.36	04:00-08:00	1,565.97	1,572.10
08:00-12:00	5,310.40	7,513.79	12:00-16:00	7,773.62	2,250.94
16:00-20:00	57.98	5,766.67	20:00-24:00	5,676.93	2,842.53
00:00 储电	6,219.20		24:00 储电	6,395.64	

对应结果见表 39。

表 39 问题 3（2025-12-21） 紧急购电

时段	电量
07:50-08:00	33.89

A.4-2 问题 4-2

对应结果见表 40。

表 40 问题 4-2（2025-03-20） 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	574.14	14:00-14:10	0.00
16:00-16:10	488.93	18:00-18:10	310.56	20:00-20:10	756.24
全天购电量 67,791.08 kWh			全天购电费 42,355.14 元		

对应结果见表 41。

表 41 问题 4-2（2025-03-20）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	3,655.77	152.63	04:00-08:00	1,195.22	5,572.84
08:00-12:00	6,186.28	2,777.38	12:00-16:00	4,162.63	444.57
16:00-20:00	214.58	7,108.02	20:00-24:00	5,717.23	1,458.76
00:00 储电	6,850.72		24:00 储电	6,409.05	

对应结果见表 42。

表 42 问题 4-2（2025-03-20）紧急购电

时段	电量
无	0.00

对应结果见表 43。

表 43 问题 4-2（2025-06-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	210.92	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 36,422.89 kWh			全天购电费 18,518.97 元		

对应结果见表 44。

表 44 问题 4-2（2025-06-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	438.52	3,694.59	04:00-08:00	1,239.76	1,877.96
08:00-12:00	9,421.97	0.00	12:00-16:00	0.00	0.00
16:00-20:00	163.47	5,505.34	20:00-24:00	6,196.26	2,497.81
00:00 储电	7,001.50		24:00 储电	7,631.37	

对应结果见表 45。

表 45 问题 4-2（2025-06-21）紧急购电

时段	电量
无	0.00

对应结果见表 46。

表 46 问题 4-2（2025-09-23）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	491.40	14:00-14:10	0.00
16:00-16:10	549.29	18:00-18:10	416.70	20:00-20:10	0.00
全天购电量 71,436.45 kWh			全天购电费 46,451.24 元		

对应结果见表 47。

表 47 问题 4-2（2025-09-23）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,585.50	0.00	04:00-08:00	760.67	5,830.96
08:00-12:00	5,687.53	1,896.62	12:00-16:00	3,858.02	528.46
16:00-20:00	253.12	6,023.40	20:00-24:00	6,035.33	2,472.31
00:00 储电	6,184.77		24:00 储电	6,633.87	

对应结果见表 48。

表 48 问题 4-2（2025-09-23）紧急购电

时段	电量
20:10-20:20	10.23

对应结果见表 49。

表 49 问题 4-2（2025-12-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	967.61	14:00-14:10	0.00
16:00-16:10	812.86	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 97,314.24 kWh			全天购电费 73,263.64 元		

对应结果见表 50。

表 50 问题 4-2（2025-12-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,056.07	143.74	04:00-08:00	1,743.90	2,155.89
08:00-12:00	5,755.95	7,219.80	12:00-16:00	6,952.06	2,243.11
16:00-20:00	24.16	5,708.86	20:00-24:00	5,755.02	2,845.49
00:00 储电	6,309.24		24:00 储电	6,493.37	

对应结果见表 51。

表 51 问题 4-2（2025-12-21）紧急购电

时段	电量
07:50-08:00	23.78

A.4-3 问题 4-3

对应结果见表 52。

表 52 问题 4-3（2025-03-20）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	520.67	14:00-14:10	0.00
16:00-16:10	638.18	18:00-18:10	714.94	20:00-20:10	741.44
全天购电量 66,091.31 kWh			全天购电费 43,656.33 元		

对应结果见表 53。

表 53 问题 4-3（2025-03-20）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	3,255.66	249.34	04:00-08:00	1,840.75	5,937.64
08:00-12:00	3,254.96	2,698.33	12:00-16:00	6,863.61	254.72
16:00-20:00	291.94	6,696.45	20:00-24:00	5,561.08	1,554.74
00:00 储电	6,485.79		24:00 储电	6,123.41	

对应结果见表 54。

表 54 问题 4-3（2025-03-20）紧急购电

时段	电量
09:40-10:00	79.05

对应结果见表 55。

表 55 问题 4-3（2025-06-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	120.43	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 34,998.69 kWh			全天购电费 17,375.08 元		

对应结果见表 56。

表 56 问题 4-3（2025-06-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	275.76	2,705.06	04:00-08:00	1,497.17	1,779.01
08:00-12:00	9,921.65	0.00	12:00-16:00	0.00	0.00
16:00-20:00	41.37	6,074.66	20:00-24:00	5,613.02	2,623.77
00:00 储电	5,257.18		24:00 储电	6,224.03	

对应结果见表 57。

表 57 问题 4-3（2025-06-21）紧急购电

时段	电量
无	0.00

对应结果见表 58。

表 58 问题 4-3（2025-09-23）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	347.56	14:00-14:10	0.00
16:00-16:10	522.47	18:00-18:10	628.48	20:00-20:10	0.00
全天购电量 67,886.23 kWh			全天购电费 46,606.18 元		

对应结果见表 59。

表 59 问题 4-3（2025-09-23）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,249.37	22.22	04:00-08:00	984.97	6,967.82
08:00-12:00	4,885.84	1,675.86	12:00-16:00	6,081.90	461.66
16:00-20:00	110.65	5,980.56	20:00-24:00	5,668.05	2,563.12
00:00 储电	6,114.10		24:00 储电	6,262.09	

对应结果见表 60。

表 60 问题 4-3（2025-09-23）紧急购电

时段	电量
09:00-09:50	220.76
18:20-18:30	1.08
20:10-20:20	10.23

对应结果见表 61。

表 61 问题 4-3（2025-12-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	924.13	14:00-14:10	0.00
16:00-16:10	821.56	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 95,641.71 kWh			全天购电费 72,890.05 元		

对应结果见表 62。

表 62 问题 4-3（2025-12-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,169.62	178.51	04:00-08:00	232.98	904.57
08:00-12:00	5,347.35	7,538.30	12:00-16:00	8,080.06	2,504.57
16:00-20:00	57.98	5,766.67	20:00-24:00	5,708.45	2,845.49
00:00 储电	6,215.16		24:00 储电	6,420.72	

对应结果见表 63。

表 63 问题 4-3（2025-12-21）紧急购电

时段	电量
07:50-08:00	33.89

附录 B 支撑文件与完整程序

五份结果工作簿、全部候选轨迹、各次计划版本和独立核验记录保存在 `artifacts/v5b`；基础证据保留于 `artifacts/v5` 及更早目录。结果工作簿保持原题模板。截图思路对照与逐项节费解释另见 `docs/V5_SCREENSHOT_COMPARISON.md` 和 `docs/V5_SAVINGS_EXPLAINED.md`。

以下列出原始数据读取、预测、计划优化、在线选择和库存价值控制的核心计算程序。程序不修改原始附件，所有实际费用由原始电价与执行账本重建。

src/q12.py

"""Causal forecasts, MILP day-ahead dispatch and independent physical audits.

Run from the repository: `python -m src.q12 --data-root <C 題 directory> --stage full`
All power observations are interpreted as interval means at right-end labels.

"""

```
from __future__ import annotations
```

```
import argparse
from dataclasses import asdict, dataclass
import hashlib
import json
import os
from pathlib import Path
import platform
import subprocess
import time
```

```
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
import numpy as np
import pandas as pd
import scipy
from scipy.optimize import Bounds, LinearConstraint, milp
from scipy.sparse import lil_matrix
```

```
@dataclass(frozen=True)
```

```
class Battery:
    minimum: float = 1200.0
    maximum: float = 10800.0
    initial: float = 6000.0
    max_power: float = 5000.0
    eta_c: float = 0.9
    eta_d: float = 0.9
    dt: float = 1 / 6
```

```
    @property
    def limit(self):
        return self.max_power * self.dt
```

```
B = Battery()
N = 144
TOL = 1e-5
```

```
def dump_json(path, obj):
    Path(path).write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")
```

```
def interval_label(i):
    def clock(m):
        return f"{m // 60:02d}:{m % 60:02d}"
    return f"{clock(i * 10)}-{clock((i + 1) * 10)}"
```

```
def time_minutes(value):
    if hasattr(value, "hour"):
        return value.hour*60 + value.minute
    text = str(value).strip()
    next_day = "+1" in text
    hour, minute, *_ = text.replace("+1", "").split(":")
    return int(hour)*60 + int(minute) + 1440*int(next_day)
```

```
def read_inputs(root):
    root = Path(root)
```

```

day = pd.read_excel(root / "附件/附件 1.xlsx", sheet_name=0, header=0)
load = pd.read_excel(root / "附件/附件 2.xlsx", sheet_name="小区负载", header=0)
pv = pd.read_excel(root / "附件/附件 2.xlsx", sheet_name="光伏发电实际功率", header=0)
dates = pd.to_datetime(load.iloc[:, 0])
if day.shape != (144, 4) or load.shape != (365, 145) or pv.shape != (365, 145):
    raise ValueError("Unexpected input dimensions")
if not dates.equals(pd.to_datetime(pv.iloc[:, 0])):
    raise ValueError("Load/PV dates differ")
if list(dates) != list(pd.date_range("2025-01-01", "2025-12-31")):
    raise ValueError("Dates must cover the complete ordered year")
expected_minutes = list(range(10, 1441, 10))
if any([time_minutes(v) for v in labels] != expected_minutes
        for labels in (load.columns[1:], pv.columns[1:], day.iloc[:, 0])):
    raise ValueError("Input time labels do not match")
data = {
    "dates": pd.DatetimeIndex(dates), "price": day.iloc[:, 1].to_numpy(float),
    "q1_load": day.iloc[:, 2].to_numpy(float) * B.dt,
    "q1_pv": day.iloc[:, 3].to_numpy(float) * B.dt,
    "load": load.iloc[:, 1:].to_numpy(float) * B.dt,
    "pv": pv.iloc[:, 1:].to_numpy(float) * B.dt,
}
audit = {"shapes": {"attachment1": list(day.shape), "load": list(load.shape), "pv": list(pv.shape)},
        "date_start": str(dates.iloc[0].date()), "date_end": str(dates.iloc[-1].date()),
        "duplicates": int(dates.duplicated().sum()), "input_files": [], "variables": {}}
for key in ["price", "q1_load", "q1_pv", "load", "pv"]:
    a = data[key]
    if not np.isfinite(a).all() or (a < 0).any():
        raise ValueError(f"Nonfinite or negative data in {key}; investigate, do not silently impute")
    audit["variables"][key] = {"count": int(a.size), "missing": 0, "min": float(a.min()),
                              "max": float(a.max()), "zero_count": int((a == 0).sum())}
if (data["price"] <= 0).any():
    raise ValueError("This implementation assumes strictly positive fixed prices")
for relative in ["C 题.pdf", "附件/附件 1.xlsx", "附件/附件 2.xlsx"]:
    raw = (root / relative).read_bytes()
    audit["input_files"].append({"path": relative, "bytes": len(raw), "sha256":
hashlib.sha256(raw).hexdigest()})
    audit["data_treatment"] = "No missing/negative values found; no observations deleted or imputed. Unusual
peaks retained."
return data, audit

```

```

def optimize_day(load, pv, price, initial, terminal=6000.0, battery=B, integer=True):
    """Nominal minimum-cost schedule, g/c/d/w/S/z in kWh; terminal is a lower bound.

```

For Q1 initial=terminal and terminal_equal=True is obtained by the explicit upper bound below when requested by a (value, 'equal') tuple.

```

"""
n = len(price)
eq_terminal = isinstance(terminal, tuple)
target = terminal[0] if eq_terminal else terminal
size = 6 * n + 1
gi, ci, di, wi, si, zi = 0, n, 2*n, 3*n, 4*n, 5*n+1
objective = np.zeros(size)
objective[:n] = price
lb, ub = np.zeros(size), np.full(size, np.inf)
ub[ci:wi] = battery.limit
lb[si:zi], ub[si:zi] = battery.minimum, battery.maximum
lb[si] = ub[si] = initial
lb[si+n] = target
if eq_terminal:
    ub[si+n] = target
ub[zi:] = 1
integrality = np.zeros(size)
integrality[zi:] = int(integer)
a = lil_matrix((4*n, size))
lower, upper = np.zeros(4*n), np.zeros(4*n)
for t in range(n):
    a[t, gi+t], a[t, ci+t], a[t, di+t], a[t, wi+t] = 1, -1, 1, -1
    lower[t] = upper[t] = load[t] - pv[t]
    a[n+t, si+t+1], a[n+t, si+t] = 1, -1
    a[n+t, ci+t], a[n+t, di+t] = -battery.eta_c, 1 / battery.eta_d
    a[2*n+t, ci+t], a[2*n+t, zi+t] = 1, -battery.limit
    lower[2*n+t], upper[2*n+t] = -np.inf, 0
    a[3*n+t, di+t], a[3*n+t, zi+t] = 1, battery.limit
    lower[3*n+t], upper[3*n+t] = -np.inf, battery.limit
start = time.perf_counter()
result = milp(objective, integrality=integrality, bounds=Bounds(lb, ub),

```



```

        constraints=LinearConstraint(a.tocsc(), lower, upper),
        options={"time_limit": 30.0, "mip_rel_gap": 1e-7})
elapsed = time.perf_counter() - start
if not result.success or result.x is None:
    raise RuntimeError(f"MILP failed: status={result.status}, {result.message}")
x = result.x
sol = {"plan_kwh": x[:n], "charge_kwh": x[ci:di], "discharge_kwh": x[di:wi],
      "spill_kwh": x[wi:si], "soc_start_kwh": x[si:si+n], "soc_end_kwh": x[si+1:zi],
      "emergency_kwh": np.zeros(n), "load_kwh": np.asarray(load), "pv_kwh": np.asarray(pv)}
frame = pd.DataFrame(sol)
frame["price"] = price
meta = {"status": int(result.status), "cost": float(np.dot(price, x[:n])),
      "seconds": elapsed, "mip_gap": float(getattr(result, "mip_gap", 0) or 0),
      "dual_bound": float(getattr(result, "mip_dual_bound", result.fun) or result.fun)}
if integer:
    audit_trace(frame, battery=battery, initial=initial)
return frame, meta

def execute_day(plan, load, pv, initial, price, battery=B):
    """Causal real-time balancing; does not read future observations.

    Assumes instantaneous regulation within a ten-minute constant-power interval.
    Surplus charges first; deficit exhausts feasible discharge before emergency.
    """
    s = float(initial)
    records = []
    for t, (g, l, v) in enumerate(zip(plan, load, pv)):
        surplus = float(g + v - l)
        c = min(max(surplus, 0), battery.limit, max(0, (battery.maximum-s)/battery.eta_c))
        d = min(max(-surplus, 0), battery.limit, max(0, (s-battery.minimum)*battery.eta_d))
        e = max(0, -surplus-d)
        w = max(0, surplus-c)
        end = s + battery.eta_c*c - d/battery.eta_d
        records.append((g, l, v, c, d, e, w, s, end, price[t]))
        s = end
    frame = pd.DataFrame(records, columns=["plan_kwh", "load_kwh", "pv_kwh", "charge_kwh", "discharge_kwh",
      "emergency_kwh", "spill_kwh", "soc_start_kwh", "soc_end_kwh",
"price"], dtype=float)
    audit_trace(frame, battery=battery, initial=initial)
    return frame

def audit_trace(frame, battery=B, initial=None):
    """Independent vectorized equations, also applicable to saved full-year traces."""
    f = frame
    balance = f.plan_kwh + f.pv_kwh + f.discharge_kwh + f.emergency_kwh - f.load_kwh - f.charge_kwh -
f.spill_kwh
    evolution = f.soc_end_kwh - f.soc_start_kwh - battery.eta_c*f.charge_kwh + f.discharge_kwh/battery.eta_d
    continuity = f.soc_start_kwh.to_numpy()[1:] - f.soc_end_kwh.to_numpy()[:-1]
    metrics = {"max_balance_residual_kwh": float(np.max(np.abs(balance))),
      "max_soc_residual_kwh": float(np.max(np.abs(evolution))),
      "max_soc_discontinuity_kwh": float(np.max(np.abs(continuity))) if len(f)>1 else 0,
      "simultaneous_intervals": int(((f.charge_kwh > TOL) & (f.discharge_kwh > TOL)).sum()),
      "min_soc_kwh": float(min(f.soc_start_kwh.min(), f.soc_end_kwh.min())),
      "max_soc_kwh": float(max(f.soc_start_kwh.max(), f.soc_end_kwh.max()))}
    checks = [np.isfinite(f.select_dtypes(include="number").to_numpy()).all(),
      (f[["plan_kwh", "charge_kwh", "discharge_kwh", "emergency_kwh", "spill_kwh"]].to_numpy() >= -
TOL).all(),
      metrics["max_balance_residual_kwh"] < TOL, metrics["max_soc_residual_kwh"] < TOL,
      metrics["max_soc_discontinuity_kwh"] < TOL, metrics["simultaneous_intervals"] == 0,
      metrics["min_soc_kwh"] >= battery.minimum-TOL, metrics["max_soc_kwh"] <= battery.maximum+TOL,
      f.charge_kwh.max() <= battery.limit+TOL, f.discharge_kwh.max() <= battery.limit+TOL]
    if initial is not None:
        checks.append(abs(f.soc_start_kwh.iloc[0]-initial) < TOL)
    if not all(checks):
        raise AssertionError(f"Physical audit failed: {metrics}")
    return {**metrics, "pass": True}

def features(history, day, dates):
    """Features for all 144 targets on a day, using completed days only."""
    if day < 7:
        raise ValueError("Seven completed days required")
    past = history[day-7:day]

```

```

h = (np.arange(N)+0.5)/N
dow = dates[day].dayofweek
cols = [past[-1], past[-2], past[0], past[-3:].mean(0), past.mean(0), past.std(0),
        np.full(N, past[-1].mean()), np.full(N, past[-3:].mean())]
for k in (1, 2, 3):
    cols.extend([np.sin(2*np.pi*k*h), np.cos(2*np.pi*k*h)])
cols.extend([np.full(N, float(dow == k)) for k in range(7)])
return np.column_stack(cols)

def ridge_fit(x, y, alpha):
    """Centered ridge via linear solve; intercept unpenalized, no sklearn required."""
    mean, scale = x.mean(0), x.std(0)
    scale[scale < 1e-10] = 1
    z = (x-mean)/scale
    intercept = float(y.mean())
    coef = np.linalg.solve(z.T@z + alpha*np.eye(z.shape[1]), z.T@(y-intercept))
    return mean, scale, coef, intercept

def forecasts(data, method, alphas=(10.0, 10.0), stop=365):
    pred, fit_dates = {}, []
    for k, alpha in zip(("load", "pv"), alphas):
        values = data[k]
        out = np.zeros((stop, N))
        model, fitted_on = None, -1
        all_x = {d: features(values, d, data["dates"]) for d in range(7, stop)}
        for d in range(stop):
            if d == 0:
                out[d] = data[f"q1_{k}"]
            elif d < 7:
                out[d] = values[max(0,d-3):d].mean(0)
            elif method == "seasonal" or d < 14:
                out[d] = 0.5*values[d-1] + 0.5*values[d-7]
            else:
                if model is None or d-fitted_on >= 7:
                    training_days = range(max(7,d-60), d)
                    x = np.concatenate([all_x[j] for j in training_days])
                    y = values[max(7,d-60):d].ravel()
                    model = ridge_fit(x, y, alpha)
                    fitted_on = d
                fit_dates.append({"variable": k, "decision_date": str(data["dates"][d].date()),
                                "fit_until": str(data["dates"][d-1].date()), "alpha": alpha,
                                "training_rows": len(y)})
                mean, scale, coef, intercept = model
                out[d] = ((all_x[d]-mean)/scale)@coef+intercept
            out[d] = np.maximum(out[d], 0)
            if k == "pv" and d:
                out[d, values[max(0,d-7):d].max(0) == 0] = 0
        pred[k] = out
    return pred, fit_dates

def reserve(data, forecast, day, quantile):
    if quantile == 0 or day < 7:
        return np.zeros(N)
    start = max(1, day-28)
    errors = (data["load"][start:day]-data["pv"][start:day]
              -forecast["load"][start:day]+forecast["pv"][start:day])
    # Pool +/- three slots; no wrap at the day boundary, no future residuals.
    return np.array([max(0.0, float(np.quantile(errors[:,max(0,t-3):min(N,t+4)], quantile))) for t in
                    range(N)])

def summarize_day(f, date, name, seconds, gap=0.0):
    normal = float(np.dot(f.plan_kwh, f.price))
    emergency = float(5*np.dot(f.emergency_kwh, f.price))
    return {"date": str(date.date()), "strategy": name, "normal_cost_yuan": normal,
            "emergency_cost_yuan": emergency, "total_cost_yuan": normal+emergency,
            "plan_kwh": float(f.plan_kwh.sum()), "emergency_kwh": float(f.emergency_kwh.sum()),
            "emergency_intervals": int((f.emergency_kwh>TOL).sum()), "spill_kwh": float(f.spill_kwh.sum()),
            "charge_kwh": float(f.charge_kwh.sum()), "discharge_kwh": float(f.discharge_kwh.sum()),
            "soc_start_kwh": float(f.soc_start_kwh.iloc[0]), "soc_end_kwh": float(f.soc_end_kwh.iloc[-1]),
            "solve_seconds": seconds, "mip_gap": gap}

```

```

def backtest(data, forecast, start, stop, initial, quantile, name, retain=True):
    s, daily, frames = initial, [], []
    for d in range(start, stop):
        buffer = reserve(data, forecast, d, quantile)
        nominal, meta = optimize_day(forecast["load"][d]+buffer, forecast["pv"][d], data["price"], s)
        f = execute_day(nominal.plan_kwh.to_numpy(), data["load"][d], data["pv"][d], s, data["price"])
        daily.append(summarize_day(f, data["dates"][d], name, meta["seconds"], meta["mip_gap"]))
        if retain:
            f.insert(0, "interval_start", pd.date_range(data["dates"][d], periods=N, freq="10min"))
            f.insert(1, "date", str(data["dates"][d].date()))
            f.insert(2, "slot", np.arange(N))
            f["forecast_load_kwh"], f["forecast_pv_kwh"], f["reserve_kwh"] = forecast["load"][d],
forecast["pv"][d], buffer
            frames.append(f)
            s = float(f.soc_end_kwh.iloc[-1])
            if retain and (d == start or data["dates"][d].day == 1):
                print(f"{name}: {data['dates'][d].date()}, cumulative cost={sum(x['total_cost_yuan'] for x in
daily):.2f}", flush=True)
            combined = pd.concat(frames, ignore_index=True) if frames else None
            if combined is not None:
                audit_trace(combined, initial=initial)
            return pd.DataFrame(daily), combined, s

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--data-root", type=Path, required=True)
    parser.add_argument("--out", type=Path, default=Path("artifacts/q12"))
    parser.add_argument("--stage", choices=["q1", "smoke", "full"], default="full")
    args = parser.parse_args()
    args.out.mkdir(parents=True, exist_ok=True)
    start = time.perf_counter()
    data, input_audit = read_inputs(args.data_root)
    dump_json(args.out/"input_audit.json", input_audit)
    q1, q1_meta = optimize_day(data["q1_load"], data["q1_pv"], data["price"], B.initial, (B.initial, "equal"))
    lp, lp_meta = optimize_day(data["q1_load"], data["q1_pv"], data["price"], B.initial, (B.initial, "equal"),
integer=False)
    q1.insert(0, "slot", np.arange(N)); q1.insert(1, "interval", [interval_label(i) for i in range(N)])
    q1.to_csv(args.out/"q1_intervals.csv", index=False)
    no_storage_cost = float(np.dot(np.maximum(data["q1_load"]-data["q1_pv"], 0), data["price"]))
    q1_meta.update({"no_storage_cost": no_storage_cost, "saving_percent": 100*(no_storage_cost-
q1_meta["cost"])/no_storage_cost,
                    "lp_cost_lower_bound": lp_meta["cost"], "lp_gap_yuan": q1_meta["cost"]-lp_meta["cost"],
                    "audit": audit_trace(q1, initial=B.initial, "plan_kwh": float(q1.plan_kwh.sum()))})
    dump_json(args.out/"q1_summary.json", q1_meta)
    print("Q1", json.dumps(q1_meta, ensure_ascii=False), flush=True)
    if args.stage == "q1":
        return
    seasonal, _ = forecasts(data, "seasonal")
    tuning = []
    for alpha in (1.0, 10.0, 100.0):
        pred, _ = forecasts(data, "ridge", (alpha, alpha), stop=31)
        row = {"alpha": alpha}
        for key in ("load", "pv"):
            row[key+"_mae_kw"] = float(np.abs(pred[key][21:31]-data[key][21:31]).mean())/B.dt)
        tuning.append(row)
    alpha_load = min(tuning, key=lambda r: r["load_mae_kw"])["alpha"]
    alpha_pv = min(tuning, key=lambda r: r["pv_mae_kw"])["alpha"]
    ridge, fit_log = forecasts(data, "ridge", (alpha_load, alpha_pv))
    dump_json(args.out/"forecast_fit_log.json", fit_log)
    pd.DataFrame(tuning).to_csv(args.out/"ridge_validation.csv", index=False)
    _, _, validation_soc = backtest(data, seasonal, 0, 21, B.initial, 0, "warmup", retain=False)
    warmup, _, evaluation_soc = backtest(data, seasonal, 21, 31, validation_soc, 0, "warmup", retain=False)
    warmup.to_csv(args.out/"warmup_last10days.csv", index=False)
    validation = []
    for method, pred in (("seasonal", seasonal), ("ridge", ridge)):
        for q in (0.0, 0.7, 0.8, 0.9):
            daily, _, _ = backtest(data, pred, 21, 31, validation_soc, q, method, retain=False)
            validation.append({"forecast": method, "quantile": q, "cost_yuan": float(daily.total_cost_yuan.sum()),
                              "emergency_cost_yuan": float(daily.emergency_cost_yuan.sum()),
                              "end_soc_kwh": float(daily.soc_end_kwh.iloc[-1])})
    selected = min(validation, key=lambda r: r["cost_yuan"])
    pd.DataFrame(validation).to_csv(args.out/"policy_validation.csv", index=False)
    stop = 38 if args.stage == "smoke" else 365
    strategies = [("seasonal_q0", "seasonal", 0.0), ("ridge_q0", "ridge", 0.0)]

```

```

for method in ("seasonal", "ridge"):
    best = min((v for v in validation if v["forecast"]==method), key=lambda r: r["cost_yuan"])
    if best["quantile"]:
        strategies.append((f"{method}_q{best['quantile']:g}", method, best["quantile"]))
selected_name = f"{selected['forecast']}_q{selected['quantile']:g}"
all_daily, metrics = [], []
for name, method, q in strategies:
    pred = {"seasonal": seasonal, "ridge": ridge}[method]
    daily, trace, _ = backtest(data, pred, 31, stop, evaluation_soc, q, name)
    all_daily.append(daily)
    trace.to_csv(args.out/f"{name}_intervals.csv.gz", index=False, float_format="%.17g")
    audit = audit_trace(trace, initial=evaluation_soc)
    row = {"strategy": name, "forecast": method, "quantile": q, "selected_on_january": name==selected_name,
          "days": len(daily), "total_cost_yuan": float(daily.total_cost_yuan.sum()),
          "normal_cost_yuan": float(daily.normal_cost_yuan.sum()),
          "emergency_cost_yuan": float(daily.emergency_cost_yuan.sum()),
          "emergency_kwh": float(daily.emergency_kwh.sum()), "emergency_intervals": int(daily.emergency_intervals.sum()),
          "plan_kwh": float(daily.plan_kwh.sum()), "spill_kwh": float(daily.spill_kwh.sum()),
          "solve_seconds": float(daily.solve_seconds.sum()), "initial_soc_kwh": evaluation_soc,
          "final_soc_kwh": float(daily.soc_end_kwh.iloc[-1]), "audit": audit}
    for key in ("load", "pv"):
        error = (pred[key][31:stop]-data[key][31:stop])/B.dt
        row[key+"_mae_kw"], row[key+"_rmse_kw"] = float(np.abs(error).mean()), float(np.sqrt(np.mean(error**2)))
    metrics.append(row)
    daily = pd.concat(all_daily, ignore_index=True)
    daily.to_csv(args.out/f"q2_daily_metrics.csv", index=False)

dump_json(args.out/"q2_summary.json", {"selection": selected, "selected_strategy": selected_name, "strategies": metrics})
command = f'python -m src.q12 --data-root "{args.data_root}" --out "{args.out}" --stage {args.stage}'
manifest = {"stage": args.stage, "battery": asdict(B), "time_interpretation": "interval mean power, right-end labels",
           "alpha_load": alpha_load, "alpha_pv": alpha_pv, "validation_start": "2025-01-22", "validation_end": "2025-01-31",
           "evaluation_start": "2025-02-01", "evaluation_end": str(data["dates"][stop-1].date()),
           "validation_initial_soc": validation_soc, "evaluation_initial_soc": evaluation_soc,
           "terminal_nominal_soc_minimum": 6000, "refit_period_days": 7, "training_window_days": 60,
           "residual_window_days": 28, "pool_halfwidth_slots": 3, "seed": 2026, "deterministic": True,
           "selected_strategy": selected_name, "command": command, "total_seconds": time.perf_counter()-start,
           "python": platform.python_version(), "numpy": np.__version__, "scipy": scipy.__version__, "pandas": pd.__version__,
           "platform": platform.platform(), "git_head_before_run": subprocess.check_output(["git", "rev-parse", "HEAD"], text=True).strip(),
           "source_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(), "input_files": input_audit["input_files"]}
dump_json(args.out/"run_manifest.json", manifest)
print("COMPLETE", json.dumps({"selected": selected_name, "seconds": manifest["total_seconds"], "days": stop-31}), flush=True)

if __name__ == "__main__":
    main()

```

src/q34_data.py

```
"""Strict attachment 3/4 parsing and causal hourly forecast alignment."""
from pathlib import Path
import hashlib
import numpy as np
import pandas as pd
import openpyxl
from src.q12 import read_inputs, time_minutes, dump_json

def read_extended(root, audit_path=None):
    root=Path(root); data, base_audit=read_inputs(root)
    wb=openpyxl.load_workbook(root/'附件/附件 3.xlsx', read_only=True, data_only=True)
    rows=list(wb.active.values); wb.close()
    assert len(rows)==1461 and len(rows[0])==26
    assert list(rows[0][2:]) == [f'预报{i}小时' for i in range(1,25)]
    hourly=np.empty((365,4,24)); filled=0
    for day in range(365):
        date=data['dates'][day]
        for version in range(4):
            row=rows[1+day*4+version]
            if version==0:
                assert pd.Timestamp(row[0])==date
            elif row[0] in (None,''):
                filled+=1
            else: assert pd.Timestamp(row[0])==date
            assert time_minutes(row[1])==version*360
            hourly[day,version]=np.asarray(row[2:],float)
    wb=openpyxl.load_workbook(root/'附件/附件 4.xlsx', read_only=True, data_only=True)
    rows=list(wb.active.values); wb.close()
    assert len(rows)==366 and len(rows[0])==145
    assert [time_minutes(v) for v in rows[0][1:]]==list(range(10,1441,10))
    assert pd.DatetimeIndex([r[0] for r in rows[1:]]).equals(data['dates'])
    prices=np.asarray([r[1:] for r in rows[1:]],float)
    checks={}
    for name, values in [(('hourly_pv_kw', hourly), ('actual_price', prices))]:
        assert np.isfinite(values).all(), f'{name}: missing/nonfinite'
        assert (values==0).all(), f'{name}: negative values need investigation'
        if name=='actual_price': assert (values>0).all()
        q1,q3=np.quantile(values, [.25, .75]); iqr=q3-q1
        checks[name]={ 'shape':list(values.shape), 'count':values.size, 'missing_nonfinite':0,
            'negative':0, 'zero':int((values==0).sum()), 'min':float(values.min()), 'max':float(values.max()),
            'iqr_flags_retained':int(((values<q1-1.5*iqr)|(values>q3+1.5*iqr)).sum()),
            'deleted':0, 'imputed_numeric':0, 'winsorized':0, 'smoothed_observations':0}
    # An hourly point is an instantaneous forecast. Integrate its piecewise-linear
    # curve exactly over 10-minute cells. At lead 0 use the newest completed PV
    # interval (causally known) as a boundary proxy; midnight Jan 1 uses zero.
    pv=np.full((365,4,144), np.nan)
    for day in range(365):
        for version in range(4):
            start=version*36
            anchor=(data['pv'][day,start-1]*6 if start else
                data['pv'][day-1,-1]*6 if day else 0.0)
            vals=np.r_[anchor, hourly[day,version]]
            edges=np.arange(145-start)/6
            powers=np.interp(edges, np.arange(25), vals)
            pv[day,version,start:]=(powers[:-1]+powers[1:])/12
    data.update(hourly_pv=hourly, pv_issued=pv, actual_price=prices)
    audit={'pass':True, 'checks':checks, 'date_forward_fills_structural_only':filled,
        'dates':365, 'unique_issues':1460, 'issue_hours':[0,6,12,18],
        'forecast_horizons_hours':list(range(1,25)),
        'alignment':'Hourly lead endpoints; piecewise-linear integration over 10-minute cells; lead-zero
        anchor from last completed observed PV interval; only current-day suffix traded.',
        'input_files':base_audit['input_files']+[
            {'path':name, 'sha256':hashlib.sha256((root/name).read_bytes()).hexdigest()}
            for name in ['附件/附件 3.xlsx', '附件/附件 4.xlsx']],
        'unusable_past_slots':'NaN intentionally marks targets before each issue; never supplied to a
        solver',
        'auditor_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
    if audit_path: dump_json(audit_path, audit)
    return data, audit

if __name__=='__main__':
```

```
import argparse, json
p=argparse.ArgumentParser();p.add_argument('--data-root',required=True)
p.add_argument('--out',default='artifacts/q34/input_audit.json');args=p.parse_args()
Path(args.out).parent.mkdir(parents=True,exist_ok=True)
_,audit=read_extended(args.data_root,args.out)
print(json.dumps(audit,ensure_ascii=False,indent=2))
```

src/q4_forecast.py

"""Direct 24-hour price forecasts at four issue times, with monthly causal fits.

Seasonal, ridge and GRU see the same day/week lag curves and calendar. GRU predicts a residual around the seasonal curve, not future observed prices.

```
"""
from pathlib import Path
import argparse
import copy
import hashlib
import json
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from src.q12 import ridge_fit, dump_json
from src.q34_data import read_extended

torch.set_num_threads(1)

def calendar(indices):
    hours=(indices%144+.5)/144
    dow=(indices//144+2)%7 # 2025-01-01 Wednesday
    return np.stack([np.sin(2*np.pi*hours),np.cos(2*np.pi*hours),
                    np.sin(2*np.pi*dow/7),np.cos(2*np.pi*dow/7)],axis=-1)

def features_at(flat,origin):
    assert origin>=1008
    target=np.arange(origin,origin+144)
    return np.column_stack([flat[target-144],flat[target-1008],calendar(target)]).astype('float32')

def ridge_features(x):
    a,b=x[...,0],x[...,1]
    context=np.stack([a.mean(-1),b.mean(-1),a.std(-1),b.std(-1)],axis=-1)
    context=np.broadcast_to(context[...,:],(*a.shape,4))
    return np.concatenate([x,np.stack([a-b,a*b],axis=-1),context],axis=-1)

class PriceGRU(nn.Module):
    def __init__(self):
        super().__init__();self.gru=nn.GRU(6,32,batch_first=True);self.head=nn.Linear(32,1)
        nn.init.zeros_(self.head.weight);nn.init.zeros_(self.head.bias)
    def forward(self,x):
        h,_=self.gru(x)
        return .5*(x[:, :, 0]+x[:, :, 1])+self.head(h).squeeze(-1)

def train_gru(x,y,origins,cutoff,seed,max_epochs=50):
    train=(origins+144<=cutoff-3*144);valid=(origins>=cutoff-3*144)
    assert train.sum()>0 and valid.sum()>0
    torch.manual_seed(seed);torch.use_deterministic_algorithms(True)
    mean=float(x[train,:, :2].mean());scale=max(float(x[train,:, :2].std()),1e-6)
    z=x.copy();z[:, :, :2]=(z[:, :, :2]-mean)/scale;target=(y-mean)/scale

    tx=torch.tensor(z[train]);ty=torch.tensor(target[train]);vx=torch.tensor(z[valid]);vy=torch.tensor(target[valid])

    model=PriceGRU();opt=torch.optim.Adam(model.parameters(),lr=.003)
    best=float('inf');weights=None;stale=0;history=[];rng=np.random.default_rng(seed)
    for epoch in range(max_epochs):
        model.train();order=rng.permutation(len(tx));losses=[]
        for left in range(0,len(order),32):
            idx=order[left:left+32];opt.zero_grad()
            loss=nn.functional.smooth_l1_loss(model(tx[idx]),ty[idx])
        loss.backward();nn.utils.clip_grad_norm_(model.parameters(),1.0);opt.step();losses.append(float(loss.detach()))
        model.eval()
```

```

        with torch.no_grad():score=float((model(vx)-vy).abs().mean())*scale
history.append({'epoch':epoch+1,'training_huber':float(np.mean(losses)), 'inner_validation_mae':score})
    if score<best-1e-7:best=score;weights=copy.deepcopy(model.state_dict());stale=0
    else:stale+=1
    if stale>=6:break
model.load_state_dict(weights);model.eval()
return model,mean,scale,history,int(train.sum()),int(valid.sum())

def same_day_errors(pred,actual,start,stop):
    values=[]
    for day in range(start,stop):
        for v in range(4):values.extend(pred[day,v,:144-v*36]-actual[day,v*36:])
    a=np.asarray(values)
    return {'mae':float(np.abs(a).mean()), 'rmse':float(np.sqrt(np.mean(a*a))), 'targets':len(a)}

def main():
    p=argparse.ArgumentParser();p.add_argument('--data-root',required=True)
    p.add_argument('--out',type=Path,default=Path('artifacts/q34'));p.add_argument('--
smoke',action='store_true')

args=p.parse_args();out=args.out;out.mkdir(parents=True,exist_ok=True);(out/'models').mkdir(exist_ok=True)
    data,_=read_extended(args.data_root);flat=data['actual_price'].ravel();started=time.perf_counter()
    stop=38 if args.smoke else 365
    origins=np.arange(1008,len(flat)-143,36)
    x=np.stack([features_at(flat,t) for t in origins]);y=np.stack([flat[t:t+144] for t in
origins]).astype('float32')
    # Fit on Jan 15/22 then each month boundary; never use targets ending after fit time.
    fit_days=[d for d in range(14,stop) if d in [14,21] or data['dates'][d].day==1]
    result={name:np.full((stop,4,144),np.nan,dtype='float64') for name in
['seasonal','ridge1','ridge10','ridge100','gru17','gru42','gru2026']}
    all_logs=[]
    for day in range(7,stop):
        for v in range(4):
            sample=features_at(flat,day*144+v*36)
            result['seasonal'][day,v]=sample[:,2].mean(-1)
    for fi,day in enumerate(fit_days):
        cutoff=day*144;end=fit_days[fi+1] if fi+1<len(fit_days) else stop
        choose=(origins+144<=cutoff)&(origins>=max(1008,cutoff-60*144))
        xx=x[choose];yy=y[choose];oo=origins[choose]
        assert (oo+144<=cutoff).all()
        future_x=np.stack([features_at(flat,d*144+v*36) for d in range(day,end) for v in range(4)])
        for alpha in [1.,10.,100.]:
            a=ridge_features(xx).reshape(-1,12);b=yy.ravel()
            model=ridge_fit(a,b,alpha);mean,scale,coef,intercept=model
            forecasts=((ridge_features(future_x)-mean)/scale)@coef+intercept
            result[f'ridge{alpha:g}'][day:end]=np.maximum(forecasts.reshape(end-day,4,144),.001)
        for seed in [17,42,2026]:
t=time.perf_counter();model,mean,scale,history,nt,nv=train_gru(xx,yy,oo,cutoff,seed+day,max_epochs=12 if
args.smoke else 50)
            z=future_x.copy();z[:,2,:]=z[:,2,:]-mean/scale
            with torch.no_grad():values=model(torch.tensor(z)).numpy()*scale+mean
            result[f'gru{seed}'][day:end]=np.maximum(values.reshape(end-day,4,144),.001)
            checkpoint=out/'models'/f'gru{seed}_fit{day:03d}.pt'
            torch.save({'state_dict':model.state_dict(),'mean':mean,'scale':scale,'fit_day':day,'seed':seed,
'training_last_target_exclusive':cutoff,'architecture':'GRU(6,32)+Linear(32,1)+seasonal
residual'},checkpoint)

log={'fit_day':day,'fit_date':str(data['dates'][day].date()),'seed':seed,'actual_rng_seed':seed+day,
'training_windows':nt,'inner_validation_windows':nv,'last_target_exclusive':int((oo+144).max()),
'seconds':time.perf_counter()-t,'history':history,'checkpoint':str(checkpoint)}
    all_logs.append(log)
    print(f'price GRU seed={seed} fit={day}: {len(history)} epochs, {log["seconds"]:.1f}s, best
inner MAE={min(r["inner_validation_mae"] for r in history):.4f}',flush=True)
    np.savez_compressed(out/'price_forecasts_checkpoint.npz',**result)
    dump_json(out/'price_fit_log.json',all_logs)
    for name in result:
        if name!='seasonal':result[name][7:14]=result['seasonal'][7:14]
        result['gru_mean']=np.mean([result[f'gru{s}'] for s in [17,42,2026]],axis=0)
        validation={name:same_day_errors(values,data['actual_price'],21,31) for name,values in result.items()}
        ridge_name=min(['ridge1','ridge10','ridge100'],key=lambda n:validation[n]['mae'])
        result['ridge']=result[ridge_name]
        chosen=min(['seasonal','ridge','gru_mean'],key=lambda n:validation[ridge_name if n=='ridge' else

```



```

n]['mae'])
    metrics={name:same_day_errors(result[name],data['actual_price'],31,stop) for name in
['seasonal','ridge','gru17','gru42','gru2026','gru_mean']}
    for name in ['seasonal','ridge','gru17','gru42','gru2026','gru_mean']:assert
np.isfinite(result[name][21:]).all()
    np.savez_compressed(out/'price_forecasts.npz',**result)

dump_json(out/'price_forecast_summary.json',{'validation':validation,'evaluation':metrics,'ridge_selected':r
idge_name,
        'selected_by_validation_price_mae':chosen,'seeds':[17,42,2026],'torch_version':torch.__version__,
        'epochs_max':12 if args.smoke else 50,'fit_days':fit_days,'elapsed_seconds':time.perf_counter()-
started,
        'source_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
        'evaluation_note':'Same-day suffix only, at each issue time; repeated targets at different origins
counted separately.',
        'training_rule':'Whole 24h targets end before fit; last 3 days internal validation; refit monthly
from scratch, last 60 days; Jan15/22 initialization.',
        'clip_floor':.001,'architecture':'Direct aligned day/week lag sequences plus known calendar;
residual GRU(6,32), no future teacher forcing'})

if __name__=='__main__':main()

```

src/v3_model.py

```
"""Exact LP reformulation and causal forecast-corrected dispatch.

Energy in kWh. Same settlement, time axis and realtime regulation as V1.
No training or tuning is performed here; inputs explicitly carry decisions.
"""

from dataclasses import dataclass, asdict
from functools import lru_cache
import time
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from src.q12 import B, TOL, audit_trace, execute_day

@lru_cache(maxsize=32)
def structure(n, revision):
    # g,c,d,w,S[,increase,decrease]; state has n+1 boundary values.
    size = (7 if revision else 5)*n+1
    a = lil_matrix(((3 if revision else 2)*n, size))
    for t in range(n):
        a[t,t], a[t,n+t], a[t,2*n+t], a[t,3*n+t] = 1,-1,1,-1
        a[n+t,4*n+t+1], a[n+t,4*n+t] = 1,-1
        a[n+t,n+t], a[n+t,2*n+t] = -B.eta_c,1/B.eta_d
        if revision:
            a[2*n+t,t], a[2*n+t,5*n+1+t], a[2*n+t,6*n+1+t] = 1,-1,1
    return a.tocsr()

def remove_cycles(charge, discharge, spill):
    """Preserve grid purchases and ALL SOC values, eliminate simultaneous flow.

    x=min(c,d/(eta_c*eta_d)); c'=c-x; d'=d-eta_c*eta_d*x;
    w'=w+(1-eta_c*eta_d)*x. Requires free unbounded curtailment.
    """
    rho=B.eta_c*B.eta_d
    x=np.minimum(charge,discharge/rho)
    return charge-x,discharge-rho*x,spill+(1-rho)*x

def solve(load,pv,price,initial,terminal=6000.,old=None,refund=True,equal=False):
    load,pv,price=map(lambda x:np.asarray(x,dtype=float),(load,pv,price))
    n=len(price);revision=old is not None;size=(7 if revision else 5)*n+1
    if not all(x.shape==(n,) and np.isfinite(x).all() for x in (load,pv,price)):
        raise ValueError('Nonfinite or misaligned dispatch inputs')
    if np.any(price<=0) or not B.minimum<=terminal<=B.maximum:
        raise ValueError('Positive prices and admissible terminal storage required')
    bounds=[(0,None)]*size
    bounds[n:3*n]=[(0,B.limit)]*(2*n)
    bounds[4*n:5*n+1]=[(B.minimum,B.maximum)]*(n+1)
    bounds[4*n]=(initial,initial);bounds[5*n]=(terminal,terminal if equal else B.maximum)
    cost=np.zeros(size)
    rhs=np.r_[load-pv,np.zeros(n)]
    if revision:
        old=np.asarray(old,dtype=float)
        assert old.shape==(n,) and np.isfinite(old).all() and old.min()>=-TOL
        cost[5*n+1:6*n+1]=1.5*price
        cost[6*n+1:]=(-.5 if refund else .5)*price
        bounds[6*n+1:]=[(0,max(float(v),0)) for v in old]
        rhs=np.r_[rhs,old]
    else:cost[:n]=price
    t=time.perf_counter()
    fit=linprog(cost,A_eq=structure(n,revision),b_eq=rhs,bounds=bounds,method='highs',
        options={'time_limit':30,'dual_feasibility_tolerance':1e-
8,'primal_feasibility_tolerance':1e-8})
    if not fit.success:raise RuntimeError(f'LP status={fit.status}: {fit.message}')
    x=fit.x;c,d,w=remove_cycles(x[n:2*n],x[2*n:3*n],x[3*n:4*n])
    f=pd.DataFrame(dict(plan_kwh=x[:n],charge_kwh=c,discharge_kwh=d,spill_kwh=w,
        soc_start_kwh=x[4*n:5*n],soc_end_kwh=x[4*n+1:5*n+1],load_kwh=load,pv_kwh=pv,
        emergency_kwh=np.zeros(n),price=price))
    physical=audit_trace(f,initial=initial)
    return f,dict(objective=float(fit.fun),seconds=time.perf_counter()-t,status=int(fit.status),
```

```

iterations=int(fit.nit),physical=physical)

@dataclass(frozen=True)
class Policy:
    quantile:float=.7
    terminal:float=6000.
    mask:int=0
    beta:float=0.
    refund:bool=True
    lookback:int=18
    decay:int=36

def corrected_load(data,pred,beta,lookback=18,decay=36):
    """At each release, use only the just-completed <=3h load residuals.

    No observed PV is used to correct load; no current/future interval enters.
    Correction decays with lead, preventing all-day propagation of a short shock.
    """
    out=np.repeat(pred['load'][:,None,:],4,axis=1).copy()
    for v in range(1,4):
        start=36*v;left=max(0,start-lookback)
        err=(data['load'][:,left:start]-pred['load'][:,left:start]).mean(axis=1)
        lead=np.arange(144-start)+.5
        out[:,v,start:]=np.maximum(0,out[:,v,start:]+beta*err[:,None]*np.exp(-lead[None,:]/decay))
    return out

def risk_buffers(data,load_pred,pv_pred,q):
    out=np.zeros_like(load_pred)
    if q==0:return out
    for day in range(7,len(load_pred)):
        left=max(1,day-28)
        error=data['load'][:,left:day,None,:]-data['pv'][:,left:day,None,:]-load_pred[left:day]+pv_pred[left:day]
        for v in range(4):
            start=36*v
            # Historical same-release residuals, not errors from unreleased versions.
            for slot in range(start,144):
                out[day,v,slot]=max(0,float(np.quantile(error[:,v,max(start,slot-3):min(144,slot+4)],q)))
    return out

def run(data,load_pred,pv_pred,buffers,policy,*,start,stop,initial,name,prices=None,retain=True):
    state=float(initial);frames=[];days=[];ledger=[];solves=[]
    for day in range(start,stop):
        date=str(data['dates'][day].date());s0=state
        realized=data['price'] if prices is None else data['actual_price'][day]
        p0=data['price'] if prices is None else prices[day,0,:144]
        sol,meta=solve(load_pred[day,0]+buffers[day,0],pv_pred[day,0],p0,state,policy.terminal)
        solves.append({'date':date,'version':0,**{k:meta[k] for k in
['seconds','status','iterations','objective']})})
        plan=np.maximum(sol.plan_kwh.to_numpy(),0);original=plan.copy()
        up=np.zeros(144);down=np.zeros(144);blocks=[]
        for v in range(4):
            first=v*36;last=first+36
            if v and policy.mask & (1<<(v-1)):
                p=data['price'][first:] if prices is None else prices[day,v,:144-first]
                old=plan[first:].copy()
                sol,meta=solve(load_pred[day,v,first:]+buffers[day,v,first:],pv_pred[day,v,first:],
                    p,state,policy.terminal,original[first:],policy.refund)
                solves.append({'date':date,'version':v,**{k:meta[k] for k in
['seconds','status','iterations','objective']})})
                new=np.maximum(sol.plan_kwh.to_numpy(),0);delta=new-old
                up[first:]+=np.maximum(delta,0);down[first:]+=np.maximum(-delta,0);plan[first:]=new
                if retain:
                    for j,slot in enumerate(range(first,144)):
                        ledger.append([date,v,first,slot,float(old[j]),float(new[j]),max(float(delta[j]),0),
                            max(-float(delta[j]),0),float(p[j]),float(load_pred[day,v,slot]),
                                float(pv_pred[day,v,slot]),float(buffers[day,v,slot]),state)])
    f=execute_day(plan[first:last],data['load'][day,first:last],data['pv'][day,first:last],state,realized[first:
last])
    state=float(f.soc_end_kwh.iloc[-1]);blocks.append(f)

```

```

f=pd.concat(blocks,ignore_index=True);f.insert(0,'date',date);f.insert(1,'slot',np.arange(144))
# Publication versions control execution; the final bill is relative to
# the midnight contract, not the sum of intermediate revision movements.
up=np.maximum(plan-original,0);down=np.maximum(original-plan,0)
f['original_plan_kwh']=original;f['increase_kwh']=up;f['decrease_kwh']=down
f['original_cost_yuan']=realized*original;f['increase_cost_yuan']=realized*1.5*up
f['decrease_cost_yuan']=realized*(-.5 if policy.refund else .5)*down
f['emergency_cost_yuan']=realized*5*f.emergency_kwh

f['total_cost_yuan']=f[['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan'
]].sum(axis=1)
np.testing.assert_allclose(f.plan_kwh,original+up-down,atol=TOL,rtol=0)
item={'date':date,'strategy':name,'soc_start_kwh':s0,'soc_end_kwh':state}
for c in f:
    if c.endswith('_yuan') or c in ['original_plan_kwh','plan_kwh','increase_kwh','decrease_kwh',
        'emergency_kwh','spill_kwh','charge_kwh','discharge_kwh']:item[c]=float(f[c].sum())
    item['emergency_intervals']=int((f.emergency_kwh>TOL).sum());days.append(item)
    if retain:frames.append(f)
daily=pd.DataFrame(days);trace=pd.concat(frames,ignore_index=True) if retain else None
versions=pd.DataFrame(ledger,columns=['date','version','effective_slot','slot','old_kwh','new_kwh',
    'increase_kwh','decrease_kwh','forecast_price','forecast_load_kwh','forecast_pv_kwh','reserve_kwh','decision_
_soc_kwh'])
summary={'strategy':name,'policy':asdict(policy),'days':stop-
start,'initial_soc':initial,'final_soc':state,
    'solver_seconds':sum(x['seconds'] for x in solves),'solver_calls':len(solves),
    'physical':audit_trace(trace,initial=initial) if retain else None,
    **{c:float(daily[c].sum()) for c in daily if c.endswith('_yuan') or c in
    ['emergency_kwh','spill_kwh']}}
return daily,trace,versions,summary,pd.DataFrame(solves)

```

src/v3_q1.py

```
"""Q1 template-start sampling with explicit periodic midnight closure."""
from pathlib import Path
from dataclasses import replace
import argparse
import numpy as np
import pandas as pd
from src.q12 import read_inputs, optimize_day, B, dump_json, interval_label, audit_trace
from src.v3_model import solve

def main():
    ap=argparse.ArgumentParser(); ap.add_argument('--data-root', required=True); a=ap.parse_args()
    data,_=read_inputs(a.data_root); out=Path('artifacts/v3'); out.mkdir(parents=True, exist_ok=True)
    L,V,p=[np.roll(data[k],1) for k in ['q1_load','q1_pv','price']]
    f,meta=solve(L,V,p,6000,equal=True);_,milp=optimize_day(L,V,p,6000,(6000,'equal'))
    assert abs(meta['objective']-milp['cost'])<1e-6
    f.insert(0,'slot',range(144)); f.insert(1,'interval',[interval_label(i) for i in range(144)])
    f.to_csv(out/'q1_intervals.csv',index=False,float_format='%.17g')
    sensitivity=[]
    for eta in [.8,.85,.9,np.sqrt(.9),1.]:
        b=replace(B,eta_c=eta,eta_d=eta); ff,mm=optimize_day(L,V,p,6000,(6000,'equal'),battery=b)
        audit_trace(ff,battery=b,initial=6000)
        sensitivity.append({'single_efficiency':eta,'roundtrip_efficiency':eta*eta,'cost':mm['cost']})
    pd.DataFrame(sensitivity).to_csv(out/'q1_efficiency.csv',index=False)
    dump_json(out/'q1_summary.json',{'cost':meta['objective'],'no_storage_cost':float(p@np.maximum(L-V,0)),
    'plan_kwh':float(f.plan_kwh.sum()),'charge_kwh':float(f.charge_kwh.sum()),'discharge_kwh':float(f.discharge_
    kwh.sum()),
    'milp':milp,'lp':meta,'specified':{interval_label(i):float(f.plan_kwh.iloc[i]) for i in
    [60,72,84,96,108,120]}},
    'alignment':'Template start timestamps; Q1 periodic 24:00 sample fills 00:00 and all signals are
    jointly rotated; solve again with actual 00:00=24:00 SOC=6000.',
    'historical_series':'Q2-Q4 retained interval-average/right-end convention; no same-day final sample
    copied into their midnight histories.')}
    print('Q1',meta['objective'],milp['cost'])

if __name__=='__main__':main()
```

src/v3_experiments.py

```

"""Final-versus-midnight settlement and paired price-information experiments."""
from pathlib import Path
import argparse, json, time, hashlib
import numpy as np
import pandas as pd
from src.q12 import forecasts, dump_json
from src.q34_data import read_extended
from src.v3_model import Policy, corrected_load, risk_buffers, run, solve

OUT=Path('artifacts/v3')
def main():
    ap=argparse.ArgumentParser(); ap.add_argument('--data-root', required=True); a=ap.parse_args()
    OUT.mkdir(parents=True, exist_ok=True); beg=time.perf_counter()
    data, audit=read_extended(a.data_root); pred,_=forecasts(data, 'ridge', (1., 1.))
    pp=np.load('artifacts/q34/price_forecasts.npz'); ll=corrected_load(data, pred, 0)
    pv2=np.repeat(pred['pv'][:, None, :], 4, axis=1); pv3=data['pv_issued']
    cache={}
    def calc(mode, policy, start, stop, label, prices=None, save=False):
        pv=pv2 if mode==2 else pv3; k=(mode, policy.quantile, stop)
        if k not in cache: cache[k]=risk_buffers(data, ll[:stop], pv[:stop], policy.quantile)
        r=run(data, ll, pv, cache[k], policy, start=start, stop=stop,
            initial=6244.256891388887 if start==21 else 9902.811287870369,
            name=label, prices=prices, retain=save)
        if save:
            for obj, suffix in
zip([r[0], r[1], r[2], r[4]], ['daily.csv', 'intervals.csv.gz', 'versions.csv.gz', 'solvers.csv.gz']):
                if len(obj): obj.to_csv(OUT/f'{label}_{suffix}', index=False, float_format='%.17g')

dump_json(OUT/f'{label}_summary.json', r[3]); print(label, round(r[3]['total_cost_yuan'], 2), flush=True)
        return r
    # Fixed design preceding the new evaluation: exact Q3 mechanism, three risks,
    # all update subsets; Q4 fair price comparison with unchanged physical policy.
    design={'q3_quantiles':[0., .5, .7], 'q3_masks':list(range(8)), 'q4_models':['seasonal', 'ridge', 'gru_mean'],
        'q4_2_policy':{'quantile':.7, 'terminal':6000}, 'settlement':'final vs midnight; no fees for
intermediate reversals',
        'scope':'same-year revision; no claim of untouched external test set'}
    dump_json(OUT/'design.json', design); rows=[]
    for q in design['q3_quantiles']:
        for mask in range(8):
            r=calc(3, Policy(quantile=q, mask=mask), 21, 31, 'validation')
            rows.append({'q':q, 'mask':mask, 'cost':r[3]['total_cost_yuan']})
    pd.DataFrame(rows).to_csv(OUT/'q3_validation.csv', index=False)
    best=min(rows, key=lambda x:(x['cost'], x['mask'], x['q'])); pol=Policy(quantile=best['q'], mask=best['mask'])
    pvals=[]
    for mode in [2, 3]:
        for m in design['q4_models']:
            r=calc(mode, Policy() if mode==2 else pol, 21, 31, 'validation', pp[m])
            pvals.append({'mode':mode, 'model':m, 'cost':r[3]['total_cost_yuan']})
    selected={str(mode):min([r for r in pvals if r['mode']==mode], key=lambda x:x['cost'])['model'] for mode
in [2, 3]}

dump_json(OUT/'selection.json', {'q3':best, 'prices':selected}); pd.DataFrame(pvals).to_csv(OUT/'q4_validation.
csv', index=False)
    summaries=[]
    for mask in
range(8): summaries.append(calc(3, Policy(quantile=pol.quantile, mask=mask), 31, 365, f'q3_m{mask}', save=True)[3])
        for mode in [2, 3]:
            for m in design['q4_models']:
                summaries.append(calc(mode, Policy() if mode==2 else pol, 31, 365, f'q4_{mode}_{m}', pp[m], True)[3])
    # Actual execution comparison with clairvoyant prices only (NOT a bound on
    # an unknown global stochastic optimum): future demand/PV remain forecasts.
    oracle=np.full((365, 4, 144), np.nan)
    for v in range(4): oracle[:, v, :144-v*36]=data['actual_price'][:, v*36:]
    for mode in [2, 3]: summaries.append(calc(mode, Policy() if mode==2 else
pol, 31, 365, f'q4_{mode}_oracle', oracle, True)[3])
    dump_json(OUT/'summary.json', summaries)
    # Provable nominal-stage bound: identical 334 forecast feasible sets and
    # initial states for every price predictor, evaluated with true prices.
    states=pd.read_csv(OUT/f'q4_2_{selected["2"]}daily.csv').soc_start_kwh.to_numpy()
    rr=cache[(2, .7, 365)]; regrets=[]
    for i, day in enumerate(range(31, 365)):
        ptrue=data['actual_price'][day]; L=ll[day, 0]+rr[day, 0]; V=pv2[day, 0]
        f, meta=solve(L, V, ptrue, states[i]); lower=float(ptrue@f.plan_kwh)
        for model in ['seasonal', 'ridge', 'gru_mean']:
            plan,_=solve(L, V, pp[model][day, 0], states[i]); cost=float(ptrue@plan.plan_kwh)

```

```

        assert cost>=lower-1e-5

regrets.append({'date':str(data['dates'][day].date()),'model':model,'nominal_cost':cost,'oracle_lower':lower
,'regret':cost-lower})
    pd.DataFrame(regrets).to_csv(OUT/'nominal_price_bound.csv',index=False,float_format='%.17g')
    dump_json(OUT/'run_manifest.json',{'seconds':time.perf_counter()-beg,'input_audit':audit,
        'sources':{f:hashlib.sha256(Path(f).read_bytes().replace(b'\r\n',b'\n')).hexdigest() for f in
['src/v3_model.py','src/v3_experiments.py']},
        'bound_scope':'fixed common nominal demand/PV forecasts, risk allowance, initial SOC and terminal
constraints; no emergency execution claim'})
    print('COMPLETE',round(time.perf_counter()-beg,2),flush=True)

if __name__=='__main__':main()

```

src/v3_verify.py

```
"""Reconstruct final contracts independently and compare all physical traces."""
from pathlib import Path
import argparse, json, hashlib
import numpy as np
import pandas as pd
from src.q12 import audit_trace, dump_json, TOL
from src.q34_data import read_extended

def main():
    ap=argparse.ArgumentParser(); ap.add_argument('--data-root', required=True); a=ap.parse_args()
    out=Path('artifacts/v3'); data,_=read_extended(a.data_root); checks={}
    manifest=json.loads((out/'run_manifest.json').read_text(encoding='utf8'))
    for f,h in manifest['sources'].items(): assert
    hashlib.sha256(Path(f).read_bytes().replace(b'\r\n',b'\n')).hexdigest()==h
    summaries=json.loads((out/'summary.json').read_text(encoding='utf8'))
    for s in summaries:

name=s['strategy']; f=pd.read_csv(out/f'{name}_intervals.csv.gz'); daily=pd.read_csv(out/f'{name}_daily.csv')
    assert len(f)==334*144 and len(daily)==334
    prices=np.tile(data['price'],334) if name.startswith('q3') else data['actual_price'][:31:].reshape(-1)
    for col,expected in [(('load_kwh',data['load'][:31:].reshape(-1)),('pv_kwh',data['pv'][:31:].reshape(-1))),('price',prices)]:
        np.testing.assert_allclose(f[col],expected,atol=1e-10,rtol=0)
        physical=audit_trace(f,initial=9902.811287870369)
        vp=out/f'{name}_versions.csv.gz'; vs=pd.read_csv(vp) if vp.exists() else pd.DataFrame()
        groups={d:g for d,g in vs.groupby('date')} if len(vs) else {}
        for date,g in f.groupby('date',sort=False):
            plan=g.original_plan_kwh.to_numpy().copy()
            if date in groups:
                for ver,v in groups[date].groupby('version',sort=True):

slots=v.slot.to_numpy(int); np.testing.assert_array_equal(slots,np.arange(int(ver)*36,144))
        np.testing.assert_allclose(v.old_kwh,plan[slots],atol=TOL,rtol=0)

np.testing.assert_allclose(v.decision_soc_kwh,g.soc_start_kwh.iloc[int(ver)*36],atol=TOL,rtol=0)
        plan[slots]=v.new_kwh
        np.testing.assert_allclose(plan,g.plan_kwh,atol=TOL,rtol=0)
        # Independent formula uses retained quantity + positive penalties, not
        # the solver's original +/- accounting expression.
        x=f.original_plan_kwh.to_numpy(); y=f.plan_kwh.to_numpy(); u=np.maximum(x-y,0); v=np.maximum(y-x,0)

np.testing.assert_allclose(f.decrease_kwh,u,atol=TOL,rtol=0); np.testing.assert_allclose(f.increase_kwh,v,atol=TOL,rtol=0)
        total=prices*(np.minimum(x,y)+.5*u+.5*v+.5*f.emergency_kwh.to_numpy())
        np.testing.assert_allclose(total,f.total_cost_yuan,atol=TOL,rtol=0)
        np.testing.assert_allclose(total.reshape(334,144).sum(1),daily.total_cost_yuan,atol=1e-6,rtol=0)
        assert abs(total.sum()-s['total_cost_yuan'])<1e-5
        for col in
['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','total_cost_yuan']:
            assert abs(f[col].sum()-s[col])<1e-5
        solver=pd.read_csv(out/f'{name}_solvers.csv.gz'); assert (solver.status==0).all()

checks[name]={ 'physical':physical, 'cost':float(total.sum()), 'versions':len(vs), 'solver_calls':len(solver)}
    assert len(checks)==16
    q1=pd.read_csv(out/'q1_intervals.csv'); audit_trace(q1,initial=6000)
    for col,key in
[(('load_kwh','q1_load'),('pv_kwh','q1_pv'),('price','price')): np.testing.assert_allclose(q1[col],np.roll(data[key],1),atol=1e-10)
a[key],1),atol=1e-10)
        b=pd.read_csv(out/'nominal_price_bound.csv'); assert len(b)==1002 and b.regret.min()>-1e-5
        # Paired GRU vs seasonal actual savings with temporal block dependence.
        df0=pd.read_csv(out/'q4_3_seasonal_daily.csv'); df1=pd.read_csv(out/'q4_3_gru_mean_daily.csv')
        diff=(df0.total_cost_yuan-df1.total_cost_yuan).to_numpy(); rng=np.random.default_rng(20260911); cis={}
        for length in [7,14,28]:
            starts=rng.integers(0,len(diff),(2000,int(np.ceil(len(diff)/length))))
            ids=((starts[:,:,:None]+np.arange(length))%len(diff)).reshape(2000,-1)[:,:len(diff)]
            cis[str(length)]=np.quantile(diff[ids].sum(1),[.025,.975]).tolist()

dump_json(out/'verification.json',{'pass':True,'traces':checks,'q1_pass':True,'nominal_bound_cases':len(b),
'gru_saving_yuan':float(diff.sum()),'gru_saving_block95':cis,'daily_series_exact_order':True})
    print('PASS:16 full-year traces, final settlements, Q1 alignment, 1002 paired nominal cases',cis)

if __name__=='__main__':main()
```


src/deliver_q12.py

"""Build the Q1/Q2 manuscript, plotting inputs, figures and Excel payload.

This never refits forecasts or changes dispatch decisions. Inputs are saved results.

```
"""
from __future__ import annotations
import argparse
from dataclasses import replace
import hashlib
import json
from pathlib import Path
import sys
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from src.q12 import B, TOL, audit_trace, dump_json, interval_label, optimize_day

LABELS = {"seasonal_q0": "同期预测 无余量", "ridge_q0": "岭回归 无余量",
          "seasonal_q0.7": "同期预测 70%余量", "ridge_q0.7": "岭回归 70%余量"}
COLORS = {"load": "#3E608D", "pv": "#E8A33D", "pv_fill": "#F0C284", "grid": "#4D779B",
          "soc": "#8074C8", "emergency": "#992224", "forecast": "#7895C1", "actual": "#CD3B42", "base": "#9D9EA3"}
DATES = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]

def md_table(headers, rows):
    def fmt(x):
        return f"{x:,.2f}" if isinstance(x, (float, np.floating)) else str(x)
    return "\n".join([" | ".join(headers) + " |", " | ".join(["---"] * len(headers)) + " |"] +
                     [" | ".join(fmt(x) for x in r) + " |" for r in rows])

def blocks(f):
    return [[f"{h:02d}:{00}-{h+4:02d}:{00}", float(f.iloc[h*6:(h+4)*6].charge_kwh.sum()),
            float(f.iloc[h*6:(h+4)*6].discharge_kwh.sum())] for h in range(0, 24, 4)]

def emergency_runs(f):
    positive = f.emergency_kwh.to_numpy() > TOL
    result = []; i = 0
    while i < len(f):
        if not positive[i]:
            i += 1; continue
        j = i + 1
        while j < len(f) and positive[j]: j += 1
        label = interval_label(i).split("-")[0] + "-" + interval_label(j-1).split("-")[1]
        result.append([label, float(f.emergency_kwh.iloc[i:j].sum())]); i = j
    return result

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--out", type=Path, default=Path("artifacts/q12"))
    parser.add_argument("--skill-root", type=Path, default=Path.home() / ".codex/skills/math-modeling")
    args = parser.parse_args(); out = args.out
    sys.path.insert(0, str(args.skill_root / "tools/figure/scripts"))
    from export_figure import export_figure
    from visual_qa import audit_layout
    q1 = pd.read_csv(out / "q1_intervals.csv")
    a = json.loads((out / "q1_summary.json").read_text(encoding="utf-8"))
    summary = json.loads((out / "q2_summary.json").read_text(encoding="utf-8"))
    manifest = json.loads((out / "run_manifest.json").read_text(encoding="utf-8"))
    if manifest["stage"] != "full": raise ValueError("Full-year outputs required")
    selected = summary["selected_strategy"]
    f = pd.read_csv(out / f"{selected}_intervals.csv.gz")
    f["interval_start"] = pd.to_datetime(f.interval_start)
    daily = pd.read_csv(out / "q2_daily_metrics.csv")
    chosen_daily = daily[daily.strategy == selected].set_index("date")
    val = pd.read_csv(out / "policy_validation.csv")
```

```

metrics={r["strategy"]:r for r in summary["strategies"]}
m=metrics[selected];base=metrics["seasonal_q0"];r0=metrics["ridge_q0"]
audit={"q1":audit_trace(q1,initial=6000),"q2":{}},"accounting":{}}
for name,row in metrics.items():
    trace=pd.read_csv(out/f"{name}_intervals.csv.gz")
    audit["q2"][name]=audit_trace(trace,initial=manifest["evaluation_initial_soc"])
    cost=float(np.dot(trace.plan_kwh+5*trace.emergency_kwh,trace.price))
    if abs(cost-row["total_cost_yuan"])>0.02:raise AssertionError("Saved trace accounting differs")
    audit["accounting"][name]={recomputed_cost_yuan:cost,difference_yuan:cost-row["total_cost_yuan"]}
    if len(trace)!=334*144:raise AssertionError("Incomplete trace")
    dump_json(out/"saved_output_audit.json",audit)
    f[f.date.isin(DATES)].to_csv(out/"selected_dates_plot_data.csv",index=False)
    q1_sensitivity=[]
    for eta in (0.9,float(np.sqrt(0.9))):
        _,meta=optimize_day(q1.load_kwh.to_numpy(),q1.pv_kwh.to_numpy(),q1.price.to_numpy(),6000,(6000,"equal"),
            battery=replace(B,eta_c=eta,eta_d=eta))

    q1_sensitivity.append({"eta_charge":eta,"eta_discharge":eta,"round_trip":eta**2,"cost_yuan":meta["cost"]})
    pd.DataFrame(q1_sensitivity).to_csv(out/"q1_efficiency_sensitivity.csv",index=False)
    payload={"q1":{"plan":[[interval_label(i),float(g)] for i,g in enumerate(q1.plan_kwh)],
        "storage":[r+["00:00",6000] if i==0 else ["24:00",6000] if i==1 else [None,None]]
            for i,r in enumerate(blocks(q1))]},
        "q2":{"headers":["日期\时间"]+[interval_label(i) for i in range(144)]+"全天购电量","全天购电费",
            "plan":[],"storage":[],"emergency":[]}}
    for date,g in f.groupby("date",sort=True):
        s=chosen_daily.loc[date]

    payload["q2"]["plan"].append([date]+g.plan_kwh.astype(float).tolist()+[float(s.plan_kwh),float(s.total_cost_yuan)])
    for i,row in enumerate(blocks(g)):
        payload["q2"]["storage"].append([date if i==0 else None]+row+
            ([ "00:00",float(s.soc_start_kwh)] if i==0 else ["24:00",float(s.soc_end_kwh)] if i==1 else
            [None,None]))
        runs=emergency_runs(g)
        if not runs:runs=[["无紧急购电",0.0]]
        payload["q2"]["emergency"].extend([[date if i==0 else None]+row for i,row in enumerate(runs)])
    dump_json(out/"xlsx_payload.json",payload)
    pd.DataFrame({"slot":np.arange(144),"input_right_endpoint":[interval_label(i).split("-")[1] for i in
    range(144)],
        "normalized_output_interval":[interval_label(i) for i in
    range(144)]}).to_csv(out/"time_mapping.csv",index=False)
    figdir=Path("figures/q12");figdir.mkdir(parents=True,exist_ok=True)
    plt.rcParams.update({"font.sans-serif":["Microsoft YaHei","SimHei","DeJaVu
    Sans"],"axes.unicode_minus":False,
        "font.size":10,"axes.spines.top":False,"axes.spines.right":False,"svg.fonttype":"none"})
    figure_audit={}
    def save(fig,name):
        issues=audit_layout(fig)
        if any(level in ("FAIL","WARN") for level,_ in issues):raise RuntimeError(f"Figure layout: {issues}")
        figure_audit[name]=issues

    export_figure(fig,str(figdir/name),formats=["png","svg"],dpi=300,size_inches=tuple(fig.get_size_inches()),
        grayscale_preview=False,tight=False)
    svg=figdir/(name+".svg")
    svg.write_text("\n".join(line.rstrip() for line in svg.read_text(encoding="utf-
    8")).splitlines()+"\n",
        encoding="utf-8",newline="\n")
    plt.close(fig)
    x=(np.arange(144)+0.5)/6

    fig,axs=plt.subplots(2,1,figsize=(7.2,5.8),sharex=True,layout="constrained",gridspec_kw={"height_ratios":[2,
    1]})
    for key,label,color in [("load_kwh","负载",COLORS["load"]),("pv_kwh","光伏",COLORS["pv"]),("plan_kwh","计
    划购电",COLORS["grid"])]):
        axs[0].plot(x,q1[key]*6,label=label,color=color,lw=1.5,ls="--" if key=="plan_kwh" else "-")
        axs[0].set_ylim(0,1.25*q1[["load_kwh","pv_kwh","plan_kwh"]].to_numpy().max()*6)
        axs[0].set_ylabel("功率 / kW");axs[0].legend(ncol=3,loc="upper left");axs[0].set_title("(a) 单日供需与购电",loc="left",fontsize=11)
        axs[1].plot(np.arange(145)/6,np.r_[q1.soc_start_kwh.iloc[0],q1.soc_end_kwh],color=COLORS["soc"])
        axs[1].axhline(1200,color=COLORS["base"],ls=":");axs[1].axhline(10800,color=COLORS["base"],ls=":")
        axs[1].set_ylabel("储电量 / kWh");axs[1].set_xlabel("时刻 / h");axs[1].set_xticks(np.arange(0,25,4));axs[1].set_xlim(0,24)
        axs[1].set_title("(b) 储能状态",loc="left",fontsize=11);save(fig,"q1_dispatch")
    fig,axs=plt.subplots(2,1,figsize=(7.2,5.4),sharex=True,layout="constrained")
    g=f[f.date==DATES[0]]
    for ax,key,label in zip(axs,("load","pv"),("负载","光伏")):

```

```

ax.plot(x,g[key+"_kwh"]*6,color=COLORS["actual"],label="实际值",lw=1.4)
ax.plot(x,g["forecast_"+key+"_kwh"]*6,color=COLORS["forecast"],label="日前预测",ls="--",lw=1.5)
ax.set_ylabel(label+" /
kw");ax.set_ylim(0,1.25*g[[key+"_kwh"],"forecast_"+key+"_kwh"].to_numpy().max()*6)
axs[0].legend(ncol=2,loc="upper left");axs[0].set_title("2025-03-20 岭回归日前预测",loc="left",fontsize=11)
axs[1].set_xlabel("时刻 / h");axs[1].set_xticks(np.arange(0,25,4));axs[1].set_xlim(0,24);save(fig,"q2_forecast")
names=list(metrics);idx=np.arange(len(names))
fig,ax=plt.subplots(figsize=(7.2,4.5),layout="constrained")
normal=np.array([metrics[n]["normal_cost_yuan"] for n in names])/1e4
emerg=np.array([metrics[n]["emergency_cost_yuan"] for n in names])/1e4
ax.bar(idx,normal,color=COLORS["grid"],label="计划购电费",width=.55)
ax.bar(idx,emerg,bottom=normal,color=COLORS["emergency"],label="紧急购电费",width=.55)
ax.set_xticks(idx,[LABELS[n].replace(" ","\n") for n in names]);ax.set_ylabel("2-12月费用 / 万元")
ax.set_ylim(0,max(normal+emerg)*1.23);ax.legend(ncol=2,loc="upper right")
for i,total in enumerate(normal+emerg):ax.text(i,total+20,f"{total:,.1f}",ha="center",fontsize=10)
save(fig,"q2_cost_comparison")
fig,axs=plt.subplots(1,2,figsize=(7.2,3.5),layout="constrained")
for method,color in [{"seasonal",COLORS["base"]},("ridge",COLORS["load"])]):
    v=val[val.forecast==method]
    label="同期预测" if method=="seasonal" else "岭回归"
    axs[0].plot(range(4),v.cost_yuan/1e4,color=color,marker="o",ms=4,label=label)
    axs[1].plot(range(4),v.emergency_cost_yuan/1e4,color=color,marker="s",ms=4,label=label)
for ax in axs:
    ax.set_xticks(range(4),["无余量","70%","80%","90%"]);ax.set_xlabel("历史误差分位数");ax.set_ylim(bottom=0)
axs[0].set_ylabel("验证期总费用 / 万元");axs[1].set_ylabel("验证期紧急费用 / 万元")
axs[0].legend(loc="lower left",fontsize=9);save(fig,"q2_validation")
dump_json(out/"figure_layout_audit.json",figure_audit)
q1table=md_table(["时段","计划购电量 / kWh"],[[interval_label(h*6),float(q1.plan_kwh.iloc[h*6])] for h in (10,12,14,16,18,20)])
q1storage=md_table(["时段","充电量 / kWh","放电量 / kWh"],blocks(q1))
comparison=md_table(["策略","计划费 / 万元","紧急费 / 万元","总费用 / 万元","紧急购电 / kWh"],
[[LABELS[n],metrics[n]["normal_cost_yuan"]/1e4,metrics[n]["emergency_cost_yuan"]/1e4,metrics[n]["total_cost_yuan"]/1e4,metrics[n]["emergency_kwh"] for n in names])
forecast_table=md_table(["预测器","负载 MAE / kW","负载 RMSE / kW","光伏 MAE / kW","光伏 RMSE / kW"],
[["同期预测",base["load_mae_kw"],base["load_rmse_kw"],base["pv_mae_kw"],base["pv_rmse_kw"]],
["岭回归",m["load_mae_kw"],m["load_rmse_kw"],m["pv_mae_kw"],m["pv_rmse_kw"]]])
date_summary=md_table(["日期","全天计划购电 / kWh","全天总购电费 / 元","紧急购电 / kWh","日初 SOC / kWh","日末 SOC / kWh"],
[[d,*[float(chosen_daily.loc[d,k]) for k in ["plan_kwh","total_cost_yuan","emergency_kwh","soc_start_kwh","soc_end_kwh"]] for d in DATES])
# Build text from computed evidence. Mathematics stays editable in Markdown.
text=rfr'# 考虑预测误差的微网日前购电与储能调度

```

第一问与第二问初稿

摘要

针对小区光伏发电、负载需求与分时电价共同影响的微网购电问题，建立统一的能量平衡与储能状态模型。在给定单日曲线时，以计划购电费最小为目标，采用混合整数线性规划约束充放电互斥，并要求日初日末储能电量相等。对于未来供需未知的情形，比较同期预测和岭回归，使用历史净负载误差的分位数构造非负余量，再通过实际运行仿真评估计划费用与五倍电价的紧急购电费用。

在充电、放电效率分别为 90%，输入功率视为十分钟区间平均值的假设下，第一问全天计划购电 $\{a['plan_kwh'] : .2f\}$ kWh，购电费 $\{a['cost'] : .2f\}$ 元，相对无储能基线降低 $\{a['saving_percent'] : .2f\}\%$ 。第二问利用一月份验证数据选择岭回归与 70% 分位数余量，随后在 2-12 月 334 天进行顺序回溯。该策略总费用为 $\{m['total_cost_yuan']/1e4 : .2f\}$ 万元，相对同期预测无余量基线下降 $\{100*(1-m['total_cost_yuan']/base['total_cost_yuan']) : .2f\}\%$ ；相对岭回归无余量策略下降 $\{100*(1-m['total_cost_yuan']/r0['total_cost_yuan']) : .2f\}\%$ 。结果表明，预测改进与适量风险余量均能降低费用，但更高余量虽可进一步减少紧急购电，也可能增加总成本。本稿仅回答前两问，其结论限于所列信息、时间、效率与运行假设。

关键词：微网调度；混合整数线性规划；岭回归；风险余量；时间顺序回溯

1 问题分析

题面及附件提供了微网的物理参数、供需曲线与结果模板 [1]。第一问的电价、负载和光伏预测曲线已给定，需决定各时段的购电与储能安排，属于确定性优化问题，不需要拟合预测模型。储能的经济作用来自跨时段转移电量，必须同时考虑功率上限、容量边界和能量损失。

第二问要求每天午夜制定整天计划。实际用电和发电变化后，普通购电仍按计划量付费，剩余供电缺口通过五倍电价紧急补购。因此，

少买与多买均存在代价。模型分为日前预测、计划优化和实时执行三个部分；历史真实值用于过去信息与事后评价，不能用于提前决定当天购电量。

2 数据处理与模型假设

附件 1 含 144 个十分钟时刻，附件 2 的负载与光伏各含 $365 \times 144 = 52,560$ 个观测。实际读取未发现缺失值、负数或日期重复，保留全部样本，不对波峰进行无依据的删除或平滑。附件 1 部分时间单元格为字符串、部分为时间类型，读取后统一转换为分钟数，检查为 10、20、...、1440 的完整序列。

采用下列假设，并将它们与题面直接给定的参数区分：

1. 每个功率值表示以其时间标签为右端点的十分钟区间平均功率。电量为功率乘 $1/6$ 小时。模板的区间标签存在疑似错位，本稿按内部区间起止时间汇总，导出副本同步采用规范标签，原始附件保持不变。
2. 充电和放电效率各取 0.9，往返效率为 0.81；功率上限定义在微网母线侧。另检验总往返效率 0.9 的解释。
3. 不设置题面未给出的售电收入，允许无法利用的剩余电量被弃用。计划富余电量仍付费。
4. 实时控制采用每十分钟内功率恒定且可即时平衡的近似。当前区间缺口可以被电池和紧急购电补足；这不是提前知道整天真实曲线。
5. 第二问实际 SOC 跨日连续，日前预测轨迹的日末 SOC 下限取 6000 kWh，作为避免耗空电池的策略设置，不强制实际日末回到此值。评价期初值由统一的一月份因果热身策略得到，不从二月起每天重置。

记区间负载与光伏电量为 L_t, V_t ，计划购电为 g_t ，紧急购电为 e_t ，母线侧充电与放电为 c_t, d_t ，未利用电量为 w_t ，区间起点储电量为 S_t 。上述电量单位均为 kWh，电价 p_t 单位为元/kWh。

3 第一问的确定性优化模型

3.1 目标与约束

最小化全天购电费用：

$$\min C_1 = \sum_{t=1}^{144} p_t g_t.$$

第一问不设置紧急购电，能量平衡与储能状态满足：

$$g_t + V_t + d_t = L_t + c_t + w_t, \quad w_t \geq 0,$$

$$S_{t+1} = S_t + 0.9c_t - \frac{d_t}{0.9}, \quad 1200 \leq S_t \leq 10800.$$

令 $z_t \in \{0, 1\}$ 表示充电状态，母线侧每区间最大充放电量为 $M = 5000/6$ kWh，规定

$$0 \leq c_t \leq M z_t, \quad 0 \leq d_t \leq M(1 - z_t), \quad g_t \geq 0.$$

取 $S_1 = S_{145} = 6000$ kWh。该等式避免利用初始存量净放电制造虚假的节省。模型共有 865 个变量，其中 144 个为二元变量；利用 SciPy 的 MILP 接口调用 HiGHS 求解 [2]。

3.2 求解结果

全天计划购电量为 $\{a['plan_kwh']\}$ kWh，购电费为 $\{a['cost']\}$ 元。无储能基线按正净负载直接购电，费用为 $\{a['no_storage_cost']\}$ 元，采用储能后降低 $\{a['saving_percent']\}$ %。表 1 给出题目指定区间，表 2 给出四小时充放电汇总。图 1 展示供需、购电与储能轨迹。

表 1 第一问指定区间计划购电量

{q1table}

表 2 第一问储能运行汇总

{q1storage}

00:00 与 24:00 的储电量均为 6000.00 kWh。充电量记录进入电池前的母线电量，放电量记录电池返回母线的电量，二者不应直接按无损储能作差。

![图 1 第一问单日调度与储电量](../figures/q12/q1_dispatch.png)

图 1 第一问单日供需、购电与 SOC 轨迹

3.3 结果检验与效率敏感性

整数模型的目标值与 LP 松弛下界之差为 $\{a['lp_gap_yuan']:.8f\}$ 元，求解器报告相对间隙为 $\{a['mip_gap']:.2g\}$ 。因此，在当前输入与约束口径下，第一问已获得数值精度内的全局最优解。最大能量平衡残差约 $\{a['audit']['max_balance_residual_kwh']:.2e\}$ kWh，未出现同时充放电。

若“效率 90%”解释为总往返效率，将两侧效率均设为 $\sqrt{0.9}$ ，第一问费用变为 $\{q1_sensitivity[1]['cost_yuan']:.2f\}$ 元。由此可见，效率定义会影响数值结论，后续整篇论文应保持同一口径，不混用两组结果。

4 第二问的预测与风险余量模型

4.1 时间顺序与预测器

以 1 月 22–31 日为验证区间，模型只用各预测时点之前已结束的日期训练。岭回归从积累足够历史后开始，每七天重新拟合一次，最多使用最近 60 天。样本不足时采用简单历史曲线。验证期选择完成后，将规则固定并应用到 2–12 月，后续已观测数据可以进入下一次更新，但不回写历史预测。

同期基线使用昨日和上周同日相同时段的均值。岭回归使用昨日、前日、上周同日的同槽电量，最近三日与七日同槽均值、七日标准差、过去日均值，三阶日内正余弦项以及星期指示变量，共 21 个特征。所有标准化参数均由当前训练段计算，预测当日的真实负载与光伏不进入特征。

对负载和光伏分别拟合带截距的岭回归 [3]：

$$\min_{\{\beta, b\}} \sum_i (y_i - b - x_i^T \beta)^2 + \lambda \|\beta\|_2^2.$$

截距不参与惩罚。验证比较 $\lambda \in \{1, 10, 100\}$ ，两种变量均选择 $\lambda = 1$ 。预测值截断为非负；如果某时槽过去七天的光伏均为零，则其预测置零。该规则只利用历史，但对日出日落变化的适应存在滞后。

4.2 风险余量

令净负载预测误差为

$$\varepsilon_{d,t} = (L_{d,t} - V_{d,t}) - (\hat{L}_{d,t} - \hat{V}_{d,t}).$$

对决策日之前最近 28 天、目标时槽前后三个十分钟槽内的历史误差取经验分位数，并截断为非负，得到余量 $r_{d,t}(q)$ 。不足 28 天时只使用已有历史，时槽池化不跨越日界。初稿比较无余量以及 $q = 0.7, 0.8, 0.9$ 。

将 $\hat{L} + r(q)$ 与 $\hat{V}$ 代入第一问的物理模型，以计划购电费最小求得日前计划，同时将预测轨迹日末储电量约束为至少 6000 kWh。这里的分位数余量是一种可解释的风险控制近似，**不是精确求解全年随机最优控制，也不等于以概率 q 保证全日无缺口**。

4.3 实时执行与费用

当天计划确定后不再倒改。每一十分钟区间，令计划购电与实际光伏减去实际负载的差额为 u_t 。当 $u_t \geq 0$ 时，在功率和容量允许范围内充电，剩余部分记为未利用电量；当 $u_t < 0$ 时，先按功率和 SOC 限制尽可能放电，剩余缺口记为紧急购电。所有区间均满足供电平衡，实际 SOC 连续传递至次日。

实际评价费用为

$$C_2 = \sum_t p_t g_t^{\text{plan}} + \sum_t p_t e_t.$$

普通购电按计划量收费，未利用的计划电量不退款。余量参数按验证期的该项总费用选择，而不是只按预测误差或紧急购电量选择。

5 第二问结果与分析

5.1 验证期的费用权衡

图 2 比较不同余量的验证费用。岭回归配合 70% 分位数余量的十天总费用为 {summary['selection']['cost_yuan']:, .2f} 元，紧急费用为 {summary['selection']['emergency_cost_yuan']:, .2f} 元，在候选方案中最低。岭回归配合 90% 余量的验证期紧急费用为零，但总费用为 {float(val[(val.forecast=='ridge') & (val['quantile']==0.9)].cost_yuan.iloc[0]):, .2f} 元，更高的计划支出抵消了避免紧急购电的收益。

![图 2 验证期余量与费用](../figures/q12/q2_validation.png)

图 2 一月验证期风险余量与费用的关系

四种策略在评价期采用相同初始 SOC: {manifest['evaluation_initial_soc']:, .2f} kWh，来源于从 1 月 1 日 6000 kWh 出发的共同热身运行。不同策略在后续产生各自的 SOC 轨迹，没有按天重置。验证期与全年比较均报告期末存量，避免忽略存量差异。

5.2 样本外预测与调度效果

表 3 为 334 天共 48,096 个时段上的预测误差。相对同期基线，岭回归对负载的改善较明显，光伏改善较小。图 3 展示题目指定的 3 月 20 日预测结果，单日图用于说明误差形态，整体评价以全期指标为准。

表 3 第二问样本外预测误差

{forecast_table}

![图 3 指定日期的负载与光伏预测](../figures/q12/q2_forecast.png)

图 3 2025 年 3 月 20 日的日前预测与实际观测

表 4 与图 4 给出各策略的费用。岭回归无余量相对同期无余量已降低成本；在相同岭回归预测上增加 70% 余量，普通购电费增加，但紧急费用下降更多，最终总费用进一步降低 {100*(1-m['total_cost_yuan']/r0['total_cost_yuan']):, .2f}%。这说明本题需要评价预测与调度的组合，而非只评价预测器。

表 4 第二问 2-12 月策略比较

{comparison}

![图 4 全期费用分解](../figures/q12/q2_cost_comparison.png)

图 4 不同预测与余量策略的全期费用分解

选定策略累计紧急购电 {m['emergency_kwh']:, .2f} kWh，涉及 {m['emergency_intervals']} 个十分钟区间。未利用电量为 {m['spill_kwh']:, .2f} kWh，其中可能同时包含光伏富余和未吸收计划电量，当前账本不将其全部解释为弃光。该结果没有实现全年零紧急购电，也没有证明其为所有可行策略中的全局最优。

岭回归无余量与带余量策略在 12 月 31 日结束时均为 {m['final_soc_kwh']:, .2f} kWh，因此二者的主要费用差异不来自不同的期末电池存量。同期两组的期末 SOC 分别为 {base['final_soc_kwh']:, .2f} 与 {metrics['seasonal_q0.7']['final_soc_kwh']:, .2f} kWh；跨预测器的费用比较仍需注意这种存量差异。

5.3 指定日期结果

表 5 为题目指定四日的汇总。“全天购电量”在本稿及导出表中指普通计划购电量；紧急购电另列。“全天购电费”包含普通与紧急两部分。后续各表给出指定十分钟槽与四小时充放电量。

表 5 第二问指定日期汇总

{date_summary}

...

table_number=6

```

for date in DATES:
    g=f[f.date==date]
    purchases=[[interval_label(h*6),float(g.plan_kwh.iloc[h*6])] for h in (10,12,14,16,18,20)]
    text+=f"### {date}\n\n 表 {table_number} 给出指定时段购电量, 表 {table_number+1} 为储能汇总, 表
{table_number+2} 为连续紧急购电区间.\n\n"
    text+=f"表 {table_number} {date} 指定区间计划购电\n\n"+md_table(["时段","计划购电 /
kWh"],purchases)+"\n\n"
    text+=f"表 {table_number+1} {date} 四小时充放电\n\n"+md_table(["时段","充电 / kWh","放电 /
kWh"],blocks(g))+"\n\n"
    text+=f"表 {table_number+2} {date} 紧急购电\n\n"+md_table(["时段","紧急购电 / kWh"],emergency_runs(g)
or [{"无",0.0}])+"\n\n"
    table_number+=3
    text+=rf"'''## 6 模型评价与后续改进

```

第一问的混合整数模型可直接对应题面物理约束, 并通过 LP 下界和数值残差检验最优性与可行性。第二问将预测、风险余量和实际执行分开, 保持历史信息边界, 并以真实账本评价经济性, 具有可解释和易复现的优点。

当前局限包括: 一月份验证区间较短, 无法充分代表全部季节; 经验分位数没有给出联合概率保证; 贪心实时平衡不一定是最优电池控制; 名义日末 6000 kWh 的目标尚未系统调优; 效率、时间标签及实时观测近似会影响结果。较优的全年表现来自本次实际回测, 不应推广为任意未来数据下的保证。

在本机的一次运行中, 包括预测候选验证、共同热身、策略筛选和四组全年回测的计算约为 {manifest['total_seconds']:.1f} 秒。每组全年共求解 334 个小规模日模型; 该时间不包括代码开发、排错、写作与图表导出, 也不代表第三、四问或更大场景模型的运行耗时。

后续可在不改变费用和物理口径的前提下, 比较更充分的季节验证、终端储能目标、分位数预测与场景优化, 并将第三问的日内预报更新接入同一调度框架。本稿不包含第三、四问结果。

参考文献

[1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 C 题 微网与外部电网电力调控策略. 用户提供题面及附件 1、2、5.

[2] SciPy Developers. scipy.optimize.milp. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html> . 本文实际运行版本 1.17.1; 在线文档用于接口与方法说明。

[3] scikit-learn Developers. Ridge. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html . 本文按带截距的 L2 正则化目标以 NumPy 实现, 未调用 scikit-learn 软件包。

```

'''
    # The abstract already identifies scope; keep project control notes outside manuscript.
    reportdir=Path("reports");reportdir.mkdir(exist_ok=True)
    (reportdir/"Q1_Q2 初稿.md").write_text(text,encoding="utf-8")
    dump_json(out/"delivery_manifest.json",{ "manuscript": "reports/Q1_Q2 初稿.md", "selected_strategy": selected,
    "figure_count": 4, "table_count": table_number-
1, "builder_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
    "skill_repo": "XiaoMaColtAI/math-modeling-
skill", "skill_commit": "3527fad922660397834a6167fc2b4c29ee64ba17"})
    print("Manuscript and plotting evidence built.",flush=True)

```

```

if __name__=="__main__":main()

```

src/v3_payload.py

```
"""Exact result-cell payloads; preserve populated V2 workbook layouts."""
from pathlib import Path
import json
import pandas as pd
from src.q12 import dump_json
from src.deliver_q12 import blocks, emergency_runs

def main():
    out=Path('artifacts/v3');sel=json.loads((out/'selection.json').read_text());payload={}
    chosen={'3':f'q3_m{sel["q3"]["mask"]}', '4-2':f'q4_2_{sel["prices"]["2"]}', '4-3':f'q4_3_{sel["prices"]["3"]}' }
    for key,name in chosen.items():

f=pd.read_csv(out/f'{name}_intervals.csv.gz');plans=[];adjusted=[];storage=[];emergency=[];ledger=[];version_rows=[]
        for date,g in f.groupby('date',sort=False):

plans.append([date,*g.original_plan_kwh.to_list(),float(g.original_plan_kwh.sum()),float(g.total_cost_yuan.sum() if key=='4-2' else g.original_cost_yuan.sum())])

adjusted.append([date,*g.plan_kwh.to_list(),float(g.plan_kwh.sum()),float(g.total_cost_yuan.sum())])
        for i,b in enumerate(blocks(g)):
            storage.append([date if i==0 else None,*b,'00:00' if i==0 else '24:00' if i==1 else None,float(g.soc_start_kwh.iloc[0]) if i==0 else float(g.soc_end_kwh.iloc[-1]) if i==1 else None])
            for i,b in enumerate(emergency_runs(g) or [['无',0]]):emergency.append([date if i==0 else None,*b])
            ledger.append([date,*[float(g[c].sum()) for c in ['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','total_cost_yuan']]])
        vp=out/f'{name}_versions.csv.gz'
        if vp.exists():
            v=pd.read_csv(vp)
            for (date,ver),g in v.groupby(['date','version'],sort=True):
                values=[None]*144
                for r in g.itertuples():values[int(r.slot)]=float(r.new_kwh)
                version_rows.append([date,f'{int(ver)*6:02d}:00',*values])

payload[key]={ 'plan':plans, 'adjusted':adjusted, 'storage':storage, 'emergency':emergency, 'ledger':ledger, 'versions':version_rows}
        q1=pd.read_csv(out/'q1_intervals.csv');b=blocks(q1)
        # Official Q1 template is ordered 00:10,...,24:00. Its last slot is the
        # first slot of the repeated day. Keep every original template label.
        payload['1']={ 'plan':[[float(q1.plan_kwh.iloc[i%144])] for i in range(1,145)],
            'storage':[[*b[i][1:],float(q1.soc_start_kwh.iloc[0]) if i==0 else float(q1.soc_end_kwh.iloc[-1]) if i==1 else None] for i in range(6)]}

dump_json(out/'xlsx_payload.json',payload);dump_json(out/'delivery_selection.json',{'1':'q1_intervals.csv','2':'../v2/result2.xlsx',**chosen})

if __name__=='__main__':main()
```


scripts/export_v3_xlsx.mjs

```
import fs from 'node:fs/promises';
import path from 'node:path';
import {FileBlob, SpreadsheetFile} from '@oai/artifact-tool';
const root=process.argv[2];if(!root)throw Error('Original data root required');
const out='artifacts/v3',p=JSON.parse(await fs.readFile(`${out}/xlsx_payload.json`,`utf8`));
const dates=rows=>rows.map(r=>[r[0]?new Date(`${r[0]}T00:00:00Z`):null,...r.slice(1)]);
await fs.mkdir(`${out}/v3/xlsx',{recursive:true});const audit=[];
for(const key of ['1','3','4-2','4-3']){
  const source=key==='1'?path.join(root,'附件/附件 5/result1.xlsx'):`artifacts/v2/result${key}.xlsx`;
  const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(source));
  if(key==='1'){
    wb.worksheets.getItem('计划购电量').getRange('B2:B145').values=p[key].plan;
    const s=wb.worksheets.getItem('充放电量');s.getRange('B2:C7').values=p[key].storage.map(r=>r.slice(0,2));
    s.getRange('E2:E3').values=p[key].storage.slice(0,2).map(r=>[r[2]]);
  }else{
    for(const [sheet,field,last] of [['计划购电量','plan','EQ'],['调整购电量','adjusted','EQ'],['充放电量','storage','F'],['紧急购电量','emergency','C'],['费用分解','ledger','F'],['计划版本','versions','EP']]){
      if((sheet==='调整购电量' || sheet==='计划版本')&&key==='4-2')continue;
      const s=wb.worksheets.getItem(sheet),rows=p[key][field];
      // Only data cells are overwritten. Header text, interval labels in the
      // existing V2 filled templates, widths and formatting remain untouched.
      if(field==='storage' || field==='emergency' || field==='versions'){
        const previous=(await s.getUsedRange().values).length;
        if(previous>1)s.getRange(`A2:${last}${previous}`).clear({applyTo:'contents'});
      }
      s.getRange(`A2:${last}${rows.length+1}`).values=dates(rows);
    }
    const note=wb.worksheets.getItem('口径说明');
    if(key!=='4-2')note.getRange('B4:B5').values=[['午夜原计划与最终有效量结算：保留量 1 倍、取消量 0.5 倍、新增量 1.5 倍、紧急 5 倍；中途版本不重复收费'],['最终有效普通计划：每次已执行区间冻结，版本只用于追溯']];
  }
  wb.recalculate();await (await SpreadsheetFile.exportXlsx(wb)).save(`${out}/result${key}.xlsx`);
  const img=await wb.render({sheetName:'计划购电量',range:key==='1'?`A59:B65`:`A1:F5`,scale:1.5,format:'png'});
  await fs.writeFile(`${out}/v3/xlsx/${key}.png`,new Uint8Array(await img.arrayBuffer()));
  audit.push({key,source,template_headers_written:false});console.log(`Exported ${key}`);
}
await fs.copyFile('artifacts/v2/result2.xlsx',`${out}/result2.xlsx`);
await fs.writeFile(`${out}/xlsx_build.json`,JSON.stringify(audit,null,2));
```

src/v5_model.py

```
"""Past-only, lead-specific online load residual calibration (energy in kWh)."""
import numpy as np
```

```
def adaptive_load(data, midnight, window=28, shrink=.2):
    """Fit one nonnegative slope per release/lead-hour using completed days.

    x: last three observed hours' mean midnight-forecast error.
    y: error in the target hour, on historical days only.
    Ridge penalty = shrink * sum(x**2); no future-day normalization.
    Within each target hour the correction is constant. Midnight is unchanged.
    """
    truth=data['load']; count=len(midnight)
    out=np.repeat(midnight[:,None,:],4,axis=1); records=[]
    error=truth[:count]-midnight
    for day in range(14,count):
        left=max(7,day-window)
        for v in range(1,4):
            start=v*36
            x=error[left:day,start-18:start].mean(axis=1)
            current=float(error[day,start-18:start].mean())
            denom=float(x*x)*(1+shrink)
            for slot in range(start,144,6):
                y=error[left:day,slot:slot+6].mean(axis=1)
                beta=float(np.clip(x@y/denom,0,1.5)) if denom>1e-12 else 0.
                out[day,v,slot:slot+6]=np.maximum(0,midnight[day,slot:slot+6]+beta*current)
                records.append(dict(day=day,issue_hour=v*6,target_slot=slot,train_first_day=left,
                                    train_last_day=day-1,n_train=day-left,beta=beta,observed_residual_kwh=current))
    return out,records
```

src/v5_experiments.py

```

"""Frozen two-candidate audit experiment; no full-year strategy reselection."""
from pathlib import Path
import argparse,hashlib,time,json
import numpy as np
import pandas as pd
from src.q12 import forecasts,dump_json
from src.q34_data import read_extended
from src.v3_model import Policy,run,risk_buffers
from src.v5_model import adaptive_load

OUT=Path('artifacts/v5')
def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root',required=True);a=ap.parse_args()
    OUT.mkdir(parents=True,exist_ok=True);tick=time.perf_counter()
    design={'candidates':['baseline','adaptive'],'validation_days':[21,31],'evaluation_days':[31,365],
            'window_days':28,'shrink':.2,'slope_bounds':[0,1.5],'observation_slots':18,'target_block_slots':6,
            'quantile':.5,'mask':7,'terminal':6000,'price_model':'seasonal',
            'selection':'January 22-31 realized cost separately for Q3 and Q4 rolling; baseline wins ties',
            'scope':'Same-dataset development; previous annual results known; no untouched-test claim'}
    dump_json(OUT/'design_before_evaluation.json',design)
    data,audit=read_extended(a.data_root);pred,fits=forecasts(data,'ridge',(1.,1.))
    base=np.repeat(pred['load'][:None,:],4,axis=1)
    adaptive,records=adaptive_load(data,pred['load']);loads={'baseline':base,'adaptive':adaptive}
    pd.DataFrame(records).to_csv(OUT/'load_calibration.csv.gz',index=False,float_format='%.17g')
    prices=np.load('artifacts/q34/price_forecasts.npz')['seasonal'];pol=Policy(quantile=.5,mask=7)
    # Complete validation and record selections BEFORE building evaluation buffers.
    validation=[];chosen={}
    for mode in [3,4]:
        for name,ll in loads.items():
            rr=risk_buffers(data,ll[:31],data['pv_issued'][:31],.5)
            r=run(data,ll,data['pv_issued'],rr,pol,start=21,stop=31,initial=6244.256891388887,
                name=f'q{mode}_{name}',prices=prices if mode==4 else None,retain=False)
    validation.append(dict(mode=mode,candidate=name,cost=r[3]['total_cost_yuan'],final_soc=r[3]['final_soc']))
    chosen[str(mode)]=min([x for x in validation if x['mode']==mode],key=lambda
x:(x['cost'],x['candidate']!= 'baseline'))['candidate']
    pd.DataFrame(validation).to_csv(OUT/'validation.csv',index=False)
    dump_json(OUT/'selection.json',chosen);print('January selection',chosen,validation,flush=True)
    summaries=[];daily_results={}
    for name,ll in loads.items():
        rr=risk_buffers(data,ll,data['pv_issued'],.5)
        for mode in [3,4]:
            label=f'q{mode}_{name}'
            r=run(data,ll,data['pv_issued'],rr,pol,start=31,stop=365,initial=9902.811287870369,
                name=label,prices=prices if mode==4 else None,retain=True)
            for obj,suffix in
zip([r[0],r[1],r[2],r[4]],['daily.csv','intervals.csv.gz','versions.csv.gz','solvers.csv.gz']):
                obj.to_csv(OUT/f'{label}_{suffix}',index=False,float_format='%.17g')
            dump_json(OUT/f'{label}_summary.json',r[3]);summaries.append(r[3]);daily_results[label]=r[0]
            print(label,round(r[3]['total_cost_yuan'],2),flush=True)
    dump_json(OUT/'summary.json',summaries)
    metrics=[]
    for name,ll in loads.items():
        for v in [1,2,3]:
            e=ll[31:,v,36*v:]-data['load'][31:,36*v:]
    metrics.append(dict(candidate=name,issue_hour=6*v,mae_kwh=float(abs(e).mean()),rmse_kwh=float(np.sqrt((e*e).
mean()))),n_pairs=e.size))
    pd.DataFrame(metrics).to_csv(OUT/'load_errors.csv',index=False)
    pairs=[];rng=np.random.default_rng(20260912)
    for mode in [3,4]:
b=daily_results[f'q{mode}_baseline'];c=daily_results[f'q{mode}_adaptive'];diff=b.total_cost_yuan.to_numpy()-
c.total_cost_yuan.to_numpy()
        for block in [7,14,28]:
            starts=rng.integers(0,len(diff),(2000,int(np.ceil(len(diff)/block))))
            indices=((starts[:,None]+np.arange(block))%len(diff)).reshape(2000,-1)[:,:len(diff)]
            lo,hi=np.quantile(diff[indices].sum(axis=1),[.025,.975])
    pairs.append(dict(mode=mode,block_days=block,saving_yuan=float(diff.sum()),ci_low=float(lo),ci_high=float(hi)
),
        final_soc_difference_kwh=float(c.soc_end_kwh.iloc[-1]-b.soc_end_kwh.iloc[-1]))
    pd.DataFrame(pairs).to_csv(OUT/'paired_comparison.csv',index=False)

```

```

    dump_json(OUT/'run_manifest.json',dict(seconds=time.perf_counter()-
tick,input_audit=audit,forecast_fits=fits,
    sources={f:hashlib.sha256(Path(f).read_bytes().replace(b'\r\n',b'\n')).hexdigest() for f in
        ['src/v5_model.py','src/v5_experiments.py','src/v3_model.py','src/q12.py','src/q34_data.py']},
    price_cache_sha256=hashlib.sha256(Path('artifacts/q34/price_forecasts.npz').read_bytes()).hexdigest()))
if __name__=='__main__':main()

```

src/v5_online.py

```

"""Monthly past-only validation of load correction, with continuous live SOC."""
from pathlib import Path
import argparse,json
import numpy as np
import pandas as pd
from src.q12 import dump_json,forecasts
from src.q34_data import read_extended
from src.v3_model import Policy,run,risk_buffers
from src.v5_model import adaptive_load

OUT=Path('artifacts/v5')
def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root',required=True);a=ap.parse_args()
    dump_json(OUT/'online_design_before_evaluation.json',{'update':'first day of every
month','validation_days':28,
    'candidates':['baseline','adaptive'],'risk':.5,'terminal':6000,'mask':7,
    'selection':'common initial SOC from live state at start of historical validation window; realized
historical costs; baseline wins ties',
    'february':'January selection, unchanged','price_model':'seasonal',
    'scope':'Predefined online selection extension; not an untouched external test'})

    data,_=read_extended(a.data_root);pred,_=forecasts(data,'ridge',(1.,1.));prices=np.load('artifacts/q34/price
_forecasts.npz')['seasonal']

    loads={'baseline':np.repeat(pred['load'][:,None,:],4,axis=1),'adaptive':adaptive_load(data,pred['load'])[0]}
    buffers={k:risk_buffers(data,v,data['pv_issued'],.5) for k,v in
loads.items()};policy=Policy(quantile=.5,mask=7)
    choices=[];summaries=[]
    for mode in [3,4]:
        state=9902.811287870369;states={31:state};pieces=[[],[],[],[]]
        for month in range(2,13):
            indices=np.flatnonzero(data['dates'].month==month);start=int(indices[0]);stop=int(indices[-1]+1)
            name='baseline'
            if month>2:
                left=start-28;vals=[]
                for candidate in ['baseline','adaptive']:
                    r=run(data,loads[candidate],data['pv_issued'],buffers[candidate],policy,start=left,stop=start,
                        initial=states[left],name='validation',prices=prices if mode==4 else
None,retain=False)
                    vals.append(dict(mode=mode,month=month,candidate=candidate,validation_first_day=left,
                        validation_last_day=start-
1,initial_soc=states[left],final_soc=r[3]['final_soc'],cost=r[3]['total_cost_yuan']))
                    name=min(vals,key=lambda x:(x['cost'],x['candidate']!='baseline'))['candidate']
                    choices.extend([{'*v','selected':v['candidate']==name} for v in vals])

            r=run(data,loads[name],data['pv_issued'],buffers[name],policy,start=start,stop=stop,initial=state,
                name=f'q{mode}_online',prices=prices if mode==4 else None,retain=True)
            state=r[3]['final_soc']
            for day,val in zip(range(start,stop),r[0].soc_start_kwh):states[day]=float(val)
            for dest,obj in zip(pieces,[r[0],r[1],r[2],r[4]]):dest.append(obj)
            print('online',mode,month,name,round(r[3]['total_cost_yuan'],2),flush=True)
            combined=[pd.concat(p,ignore_index=True) for p in pieces];daily=combined[0]
            for obj,suffix in zip(combined,['daily.csv','intervals.csv.gz','versions.csv.gz','solvers.csv.gz']):
                obj.to_csv(OUT/f'q{mode}_online_{suffix}',index=False,float_format='%17g')
            from src.q12 import audit_trace
            summary=dict(strategy=f'q{mode}_online',days=334,initial_soc=9902.811287870369,final_soc=state,

total_cost_yuan=float(daily.total_cost_yuan.sum()),emergency_kwh=float(daily.emergency_kwh.sum()),

emergency_cost_yuan=float(daily.emergency_cost_yuan.sum()),physical=audit_trace(combined[1],initial=9902.811
287870369))
            dump_json(OUT/f'q{mode}_online_summary.json',summary);summaries.append(summary)
            pd.DataFrame(choices).to_csv(OUT/'online_selection.csv',index=False,float_format='%17g')
            dump_json(OUT/'online_summary.json',summaries)
if __name__=='__main__':main()

```

src/v5_audit.py

```

"""Independent workbook, cumulative-inventory LP, dual certificate and trace audit."""
from pathlib import Path
import argparse,hashlib,json
import numpy as np
import pandas as pd
import openpyxl
from scipy.optimize import linprog
from src.q12 import B,forecasts,optimize_day,dump_json,audit_trace
from src.q34_data import read_extended
from src.v3_model import solve
from src.v5_model import adaptive_load

OUT=Path('artifacts/v5')
def independent_lp(load,pv,price,initial,terminal=6000,old=None,equal=False):
    """Eliminate state variables; revision uses a two-line convex epigraph."""
    n=len(price);rev=old is not None;k=(5 if rev else
4)*n;I=np.eye(n);Z=np.zeros((n,n));T=np.tril(np.ones((n,n)))
    eq=np.hstack([I,-I,I,-I]+([Z] if rev else []));rhs=np.asarray(load)-pv
    soc=np.hstack([Z,B.eta_c*T,-T/B.eta_d,Z]+([Z] if rev else []))
    ub=[soc,-soc];br=[np.full(n,B.maximum-initial),np.full(n,initial-B.minimum)]
    if equal:eq=np.vstack([eq,soc[-1:]];rhs=np.r_[rhs,terminal-initial]
    else:ub.append(-soc[-1:]);br.append(np.array([initial-terminal]))
    c=np.zeros(k);bounds=[(0,None)]*k;bounds[n:3*n]=[(0,B.limit)]*(2*n)
    if rev:
        c[4*n:]=1;bounds[4*n:]=[(None,None)]*n
        for slope in [.5,1.5]:
            row=np.zeros((n,k));row[:,n]=np.diag(slope*price);row[:,4*n:]=-I
            ub.append(row);br.append(slope*price*old)
        else:c[:n]=price
        A=np.vstack(ub);b=np.concatenate(br)
        fit=linprog(c,A_ub=A,b_ub=b,A_eq=eq,b_eq=rhs,bounds=bounds,method='highs')
        assert fit.success,fit.message
        stationarity=c-eq.T@fit.eqlin.marginals-A.T@fit.ineqlin.marginals-fit.lower.marginals-
fit.upper.marginals
        dual=float(rhs@fit.eqlin.marginals+b@fit.ineqlin.marginals)
        comp=[np.max(abs(fit.ineqlin.residual*fit.ineqlin.marginals))]
        for label,idx in [('lower',0),('upper',1)]:
            valid=np.array([v[idx] is not None for v in bounds]);v=np.array([x[idx] or 0 for x in
bounds]);m=getattr(fit,label).marginals
            dual+=float(v[valid]@m[valid]);comp.append(float(np.max(abs((fit.x[valid]-v[valid])*m[valid]))) if
valid.any() else 0)
            sign_violation=float(max(0,fit.ineqlin.marginals.max(),-
fit.lower.marginals.min(),fit.upper.marginals.max()))
            cert=dict(objective=float(fit.fun),dual_objective=dual,gap=abs(float(fit.fun)-
dual),stationarity=float(abs(stationarity).max()),dual_sign_violation=sign_violation,
            complementarity=max(comp),equality_residual=float(abs(eq@fit.x-
rhs).max()),inequality_violation=float(max(0,np.max(A@fit.x-b))))
            assert max(cert[x] for x in
['gap','stationarity','complementarity','equality_residual','inequality_violation','dual_sign_violation'])<1
e-5
        return cert

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-
root',required=True);a=ap.parse_args();root=Path(a.data_root)
    OUT.mkdir(parents=True,exist_ok=True);data,source=read_extended(root);books=[]
    for index in range(1,5):
        path=root/'附件'/f'附件{index}.xlsx';wb=openpyxl.load_workbook(path,data_only=False)
        sheets=[]
        for ws in wb:
            cells=list(ws.values);formula=sum(isinstance(v,str) and v.startswith('=') for row in cells for v
in row)
            sheets.append(dict(name=ws.title,rows=ws.max_row,columns=ws.max_column,state=ws.sheet_state,
            formulas=formula,hidden_rows=sum(bool(v.hidden) for v in
ws.row_dimensions.values()),hidden_columns=sum(bool(v.hidden) for v in ws.column_dimensions.values()),
            merges=[str(v) for v in ws.merged_cells.ranges]))
    books.append(dict(file=path.name,sha256=hashlib.sha256(path.read_bytes()).hexdigest(),sheets=sheets));wb.clo
se()
    # Independent openpyxl extraction compared against the production pandas parser.
    wb=openpyxl.load_workbook(root/'附件/附件 2.xlsx',data_only=True,read_only=True)
    for name,ws in zip(['load','pv'],wb):
        vals=np.array([r[1:] for r in list(ws.values)[1:]],float)/6
        np.testing.assert_allclose(vals,data[name],atol=3e-13,rtol=0)
    wb.close()

```

```

pred,_=forecasts(data,'ridge',(1.,1.));l1,records=adaptive_load(data,pred['load']);changed=dict(data)
altered=data['load'].copy();altered[20,72:]+=12345;altered[21:]+=98765;changed['load']=altered
l12,_=adaptive_load(changed,pred['load'])
np.testing.assert_array_equal(l1[:20],l12[:20]);np.testing.assert_array_equal(l1[20,:3],l12[20,:3])
certificates=[];rng=np.random.default_rng(20260912)
# Randomized and adverse cases, including low prices and redundant PV.
for case in range(36):
    n=[6,24,144][case%3];L=rng.uniform(0,800,n);V=rng.uniform(0,1000,n);p=rng.uniform(.001,2,n)
    s0=6000.;old=rng.uniform(0,900,n) if case%2 else None
    ref=independent_lp(L,V,p,s0,old=old);f,m=solve(L,V,p,s0,old=old)
    assert abs(ref['objective']-m['objective'])<1e-5
    ref.update(case=f'random_{case}',difference=abs(ref['objective']-
m['objective']));certificates.append(ref)
    if old is None:
        _,mm=optimize_day(L,V,p,s0,integer=True);assert abs(mm['cost']-m['objective'])<1e-5
    L,V,p=[np.roll(data[k],1) for k in ['q1_load','q1_pv','price']]
    cert=independent_lp(L,V,p,6000,equal=True);f,m=solve(L,V,p,6000,equal=True)
    cert.update(case='q1',difference=abs(cert['objective']-m['objective']));certificates.append(cert)
# Actual published revision instances, not only synthetic examples.
for name in ['q3_baseline','q4_baseline','q3_adaptive','q4_adaptive','q3_online','q4_online']:
    versions=pd.read_csv(OUT/f'{name}_versions.csv.gz');trace=pd.read_csv(OUT/f'{name}_intervals.csv.gz')
    for date in ['2025-03-20','2025-06-21','2025-09-23','2025-12-21']:
        for ver in [1,2,3]:
            v=versions[(versions.date==date)&(versions.version==ver)];g=trace[(trace.date==date)&(trace.slot>=ver*36)]

            L=(v.forecast_load_kwh+v.reserve_kwh).to_numpy();V=v.forecast_pv_kwh.to_numpy();p=v.forecast_price.to_numpy(
);old=g.original_plan_kwh.to_numpy();state=float(v.decision_soc_kwh.iloc[0])
            cert=independent_lp(L,V,p,state,old=old);_,m=solve(L,V,p,state,old=old)
            cert.update(case=f'{name}_{date}_{ver}',difference=abs(cert['objective']-
m['objective']));assert cert['difference']<1e-5;certificates.append(cert)
            pd.DataFrame(certificates).to_csv(OUT/'solver_certificates.csv',index=False)
            traces=[]
            for name in ['q3_baseline','q4_baseline','q3_adaptive','q4_adaptive','q3_online','q4_online']:
                f=pd.read_csv(OUT/f'{name}_intervals.csv.gz');d=pd.read_csv(OUT/f'{name}_daily.csv');v=pd.read_csv(OUT/f'{na
me}_versions.csv.gz')
                audit=audit_trace(f,initial=9902.811287870369)
                for col,key in
[('load_kwh','load'),('pv_kwh','pv')]:np.testing.assert_allclose(f[col],data[key][31:].reshape(-1),atol=1e-
10,rtol=0)
                expected_price=np.tile(data['price'],334) if name.startswith('q3') else
data['actual_price'][31:].reshape(-1)
                np.testing.assert_allclose(f.price,expected_price,atol=1e-12,rtol=0)

            independent_cost=f.price*(np.minimum(f.original_plan_kwh,f.plan_kwh)+.5*np.maximum(f.original_plan_kwh-
f.plan_kwh,0)+1.5*np.maximum(f.plan_kwh-f.original_plan_kwh,0)+5*f.emergency_kwh)
            np.testing.assert_allclose(independent_cost,f.total_cost_yuan,atol=1e-7,rtol=0)
            np.testing.assert_allclose(independent_cost.groupby(f.date).sum(),d.total_cost_yuan,atol=1e-6,rtol=0)
            for date,g in f.groupby('date',sort=False):
                plan=g.original_plan_kwh.to_numpy().copy()
                for ver,vg in v[v.date==date].groupby('version',sort=True):
                    slots=vg.slot.to_numpy(int);np.testing.assert_array_equal(slots,np.arange(int(ver)*36,144));np.testing.asser
t_allclose(vg.old_kwh,plan[slots],atol=1e-7)
                    np.testing.assert_allclose(vg.decision_soc_kwh,g.soc_start_kwh.iloc[int(ver)*36],atol=1e-
7);plan[slots]=vg.new_kwh
                    np.testing.assert_allclose(plan,g.plan_kwh,atol=1e-7)

            traces.append(dict(strategy=name,intervals=len(f),versions=len(v),cost=float(independent_cost.sum()),**audit
))
            for new,old in [('q3_baseline','q3_m7'),('q4_baseline','q4_3_seasonal')]:
                f=pd.read_csv(OUT/f'{new}_daily.csv');g=pd.read_csv(f'artifacts/v3/{old}_daily.csv')
                np.testing.assert_allclose(f.total_cost_yuan,g.total_cost_yuan,atol=1e-6,rtol=0)
                q2=pd.read_csv(f'artifacts/q12/ridge_q0.7_intervals.csv.gz')
                q2_physical=audit_trace(q2,initial=9902.811287870369)
                for col,key in
[('load_kwh','load'),('pv_kwh','pv')]:np.testing.assert_allclose(q2[col],data[key][31:].reshape(-1),atol=1e-
10,rtol=0)
                np.testing.assert_allclose(q2.price,np.tile(data['price'],334),atol=1e-12,rtol=0)
                q2_cost=float((q2.price*(q2.plan_kwh+5*q2.emergency_kwh)).sum());assert abs(q2_cost-
14132612.15924625)<1e-5
                q1=pd.read_csv(f'artifacts/v3/q1_intervals.csv');q1_physical=audit_trace(q1,initial=6000)
                for col,key in
[('load_kwh','q1_load'),('pv_kwh','q1_pv'),('price','price')]:np.testing.assert_allclose(q1[col],np.roll(dat
a[key],1),atol=1e-10,rtol=0)
                assert abs(float(q1.price@q1.plan_kwh)-certificates[36]['objective'])<1e-5
                choices=pd.read_csv(OUT/'online_selection.csv')
                for (mode,month),g in choices.groupby(['mode','month']):

```

```

        start=int(np.flatnonzero(data['dates'].month==month)[0]);assert (g.validation_last_day==start-
1).all() and (g.validation_first_day==start-28).all()
        live=pd.read_csv(OUT/f'q{mode}_online_daily.csv');assert abs(g.initial_soc.iloc[0]-
live.soc_start_kwh.iloc[start-28-31])<1e-7
        assert g[g.selected].cost.iloc[0]==g.cost.min()
        result=dict(status='PASS',workbooks=books,input_audit=source,independent_attachment2_values=105120,
        future_perturbation_test='PASS: future loads do not change earlier
releases',solver_cases=len(certificates),
        q1_retained_trace=q1_physical,q2_retained_trace=q2_physical,q2_independent_cost=q2_cost,
        monthly_selection_windows=20,milp_comparisons=18,max_solver_difference=max(x['difference'] for x in
certificates),
        max_duality_gap=max(x['gap'] for x in certificates),traces=traces,
        limitations=['Structural audit does not prove that sensors are error-free', 'Reference Modex is a
supplied comparison manuscript, not independently certified results'])
        dump_json(OUT/'verification.json',result);print(json.dumps({k:result[k] for k in
['status','solver_cases','milp_comparisons','max_solver_difference','max_duality_gap']}),ensure_ascii=False))
if __name__=='__main__':main()

```


src/v5_summary.py

```
from pathlib import Path
import json
import numpy as np
import pandas as pd
from src.q12 import dump_json

def main():
    out=Path('artifacts/v5');rows=[];months=[];rng=np.random.default_rng(20260912)
    for mode in [3,4]:
        b=pd.read_csv(out/f'q{mode}_baseline_daily.csv');c=pd.read_csv(out/f'q{mode}_online_daily.csv')
        diff=b.total_cost_yuan.to_numpy()-c.total_cost_yuan.to_numpy()

    row=dict(mode=mode,baseline_cost=float(b.total_cost_yuan.sum()),v5_cost=float(c.total_cost_yuan.sum()),savin
g=float(diff.sum()),

    saving_percent=float(diff.sum()/b.total_cost_yuan.sum()*100),baseline_emergency=float(b.emergency_kwh.sum()),
        v5_emergency=float(c.emergency_kwh.sum()),initial_soc_difference=float(c.soc_start_kwh.iloc[0]-
b.soc_start_kwh.iloc[0]),
        final_soc_difference=float(c.soc_end_kwh.iloc[-1]-b.soc_end_kwh.iloc[-1]),bootstrap=[]
        for block in [7,14,28]:

    start=rng.integers(0,334,(2000,int(np.ceil(334/block)))));ix=((start[:, :,None]+np.arange(block))%334).reshape
(2000,-1)[:,:334]

    lo,hi=np.quantile(diff[ix].sum(axis=1),[.025,.975]);row['bootstrap'].append(dict(block_days=block,low=float(
lo),high=float(hi)))
        rows.append(row)
        for month in range(2,13):
            mask=pd.to_datetime(b.date).dt.month==month

    months.append(dict(mode=mode,month=month,baseline=float(b.loc[mask,'total_cost_yuan'].sum()),online=float(c.
loc[mask,'total_cost_yuan'].sum()),saving=float(diff[mask].sum()))

    dump_json(out/'improvement.json',rows);pd.DataFrame(months).to_csv(out/'monthly_comparison.csv',index=False)
    from src.q12 import B
    pmax=json.loads((out/'verification.json').read_text(encoding='utf-
8'))['input_audit']['checks']['actual_price']['max']
    selections=pd.read_csv(out/'online_selection.csv');sensitivity=[]
    for (mode,month),g in selections.groupby(['mode','month']):
        a=g[g.selected].iloc[0];b=g[~g.selected].iloc[0]
        gap=float(b.cost-a.cost);value=float(5*pmax/B.eta_c*abs(b.final_soc-a.final_soc))

    sensitivity.append(dict(mode=int(mode),month=int(month),cost_gap=gap,max_inventory_adjustment=value,unchange
d=gap>value))
    pd.DataFrame(sensitivity).to_csv(out/'selection_inventory_sensitivity.csv',index=False)
    print(json.dumps(rows,ensure_ascii=False,indent=2))
if __name__=='__main__':main()
```

src/v5_delivery.py

```
"""Write V5 data cells into V3 layouts and read every written cell back."""
from pathlib import Path
from datetime import datetime
import json,shutil,math,hashlib
from copy import copy
import pandas as pd
import openpyxl
from src.q12 import dump_json
from src.deliver_q12 import blocks,emergency_runs

OUT=Path('artifacts/v5')
SOURCES={'1':'artifacts/v3/q1_intervals.csv','2':'artifacts/q12/ridge_q0.7_intervals.csv.gz',
          '3':str(OUT/'q3_online_intervals.csv.gz'),'4-2':'artifacts/v3/q4_2_ridge_intervals.csv.gz','4-3':str(OUT/'q4_online_intervals.csv.gz')}

def main():
    audit=[]
    for key in ['1','2','4-2']:shutil.copy2(f'artifacts/v3/result{key}.xlsx',OUT/f'result{key}.xlsx')
    for key,label in [(('3','q3_online'),('4-3','q4_online'))]:
        f=pd.read_csv(SOURCES[key]);v=pd.read_csv(OUT/f'{label}_versions.csv.gz');payload={k:[] for k in ['计划购电量','调整购电量','充放电量','紧急购电量','费用分解','计划版本']}
        for date,g in f.groupby('date',sort=False):
            dt=datetime.strptime(date,'%Y-%m-%d')
            payload['计划购电量'].append([dt,*g.original_plan_kwh.tolist(),float(g.original_plan_kwh.sum()),float(g.original_cost_yuan.sum())])
            payload['调整购电量'].append([dt,*g.plan_kwh.tolist(),float(g.plan_kwh.sum()),float(g.total_cost_yuan.sum())])
            for i,b in enumerate(blocks(g)):
                payload['充放电量'].append([dt if i==0 else None,*b,'00:00' if i==0 else '24:00' if i==1 else None,float(g.soc_start_kwh.iloc[0]) if i==0 else float(g.soc_end_kwh.iloc[-1]) if i==1 else None])
            for i,b in enumerate(emergency_runs(g) or [['无',0]]):payload['紧急购电量'].append([dt if i==0 else None,*b])
            payload['费用分解'].append([dt,*[float(g[c].sum()) for c in ['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','total_cost_yuan']]])
            for (date,ver),g in v.groupby(['date','version'],sort=True):
                values=[None]*144
                for r in g.itertuples():values[int(r.slot)]=float(r.new_kwh)
                payload['计划版本'].append([datetime.strptime(date,'%Y-%m-%d'),f'{int(ver)*6:02d}:00',*values])
            wb=openpyxl.load_workbook(f'artifacts/v3/result{key}.xlsx');assert not any(c.data_type=='f' for ws in wb for row in ws for c in row),'Formula-bearing template needs recalculation'
            headers={ws.title:tuple(c.value for c in ws[1]) for ws in wb};sheets=wb.sheetnames
            for name,rows in payload.items():
                ws=wb[name];end=ws.max_row;columns=len(rows[0])
                for row in ws.iter_rows(min_row=2,max_row=max(end,len(rows)+1),max_col=columns):
                    for cell in row:cell.value=None
                for i,row in enumerate(rows,2):
                    for j,value in enumerate(row,1):
                        cell=ws.cell(i,j);cell.value=value
                        if i>end:cell._style=copy(ws.cell(2,j)._style)
            note=wb['口径说明'];note['B2']='V5: 每月以前 28 天共同初始 SOC 验证, 选择基础或自适应负载预测; 实际 SOC 连续跨月。'
            dest=OUT/f'result{key}.xlsx';wb.save(dest);wb.close()
            checked=openpyxl.load_workbook(dest,read_only=True,data_only=True);count=0
            assert checked.sheetnames==sheets
            for name in sheets:assert tuple(next(checked[name].values))==headers[name]
            for name,rows in payload.items():
                for actual,expected in zip(checked[name].iter_rows(min_row=2,max_row=len(rows)+1,max_col=len(rows[0]),values_only=True),rows):
                    for a,e in zip(actual,expected):
                        count+=1
                        if isinstance(e,(int,float)):assert isinstance(a,(int,float)) and math.isclose(a,e,abs_tol=1e-7,rel_tol=1e-11),(name,a,e)
                        else:assert a==e,(name,a,e)
```

```

        for row in checked[name].iter_rows(min_row=len(rows)+2,values_only=True):assert all(v is None
for v in row)

checked.close();audit.append(dict(key=key,cells_compared=count,headers_preserved=True,formula_cells=0,sha256
=hashlib.sha256(dest.read_bytes()).hexdigest()))
    dump_json(OUT/'delivery_selection.json',{ 'sources':SOURCES,'main':'Q3 and Q4 rolling use causal monthly
selection; Q1/Q2/Q4 ahead retained','copied_byte_identical':['1','2','4-2']})

dump_json(OUT/'xlsx_audit.json',dict(status='PASS',updated=audit,copied={k:hashlib.sha256((OUT/f'result{k}.x
lsx').read_bytes()).hexdigest() for k in ['1','2','4-2']}));print(audit)
if __name__=='__main__':main()

```

src/v5_price_audit.py

```
"""Replay stored price checkpoints and verify release/target cutoffs."""
from pathlib import Path
import argparse, json, hashlib
import numpy as np
import torch
from src.q34_data import read_extended
from src.q4_forecast import features_at, PriceGRU, ridge_features
from src.q12 import dump_json, ridge_fit

def main():
    ap=argparse.ArgumentParser(); ap.add_argument('--data-root', required=True); a=ap.parse_args()

    data,_=read_extended(a.data_root); flat=data['actual_price'].ravel(); cache=np.load('artifacts/q34/price_forecasts.npz')
    logs=json.loads(Path('artifacts/q34/price_fit_log.json').read_text(encoding='utf-8')); fits=sorted({r['fit_day'] for r in logs}); records=[]
    for day in range(7,365):
        for v in range(4):
            origin=day*144+36*v; targets=np.arange(origin,origin+144)
            assert (targets-144<origin).all() and (targets-1008<origin).all()
            np.testing.assert_array_equal(features_at(flat,origin)[:,:2].mean(-1),cache['seasonal'][day,v])
            origins=np.arange(1008,len(flat)-143,36)
            for fi,day in enumerate(fits):
                end=fits[fi+1] if fi+1<len(fits) else 365
                xx=np.stack([features_at(flat,d*144+36*v) for d in range(day,end) for v in range(4)])
                # Refit the selected linear predictor from raw history, without using cache labels.
                oo=origins[(origins+144<=day*144)&(origins>=max(1008,day*144-60*144))]
                trainx=np.stack([features_at(flat,o) for o in oo]); trainy=np.stack([flat[o:o+144] for o in oo]).astype('float32')
                mean,scale,coef,intercept=ridge_fit(ridge_features(trainx).reshape(-1,12),trainy.ravel(),1.)
                pred=np.maximum(((ridge_features(xx)-mean)/scale)@coef+intercept,.001).reshape(end-day,4,144)
                np.testing.assert_allclose(pred,cache['ridge'][day:end],atol=1e-10,rtol=0)
                for seed in [17,42,2026]:
                    log=next(x for x in logs if x['fit_day']==day and x['seed']==seed)
                    assert log['last_target_exclusive']<=day*144 and log['actual_rng_seed']==day+seed
                    path=Path(f'artifacts/q34/models/gru{seed}_fit{day:03d}.pt')
                    saved=torch.load(path,map_location='cpu',weights_only=True); assert
                    saved['training_last_target_exclusive']==day*144
                    model=PriceGRU(); model.load_state_dict(saved['state_dict']); model.eval()
                    z=xx.copy(); z[:, :, :2]=(z[:, :, :2]-saved['mean'])/saved['scale']
                    with
                    torch.no_grad(): values=np.maximum(model(torch.tensor(z)).numpy()*saved['scale']+saved['mean'],.001).reshape(
                    end-day,4,144)
                    diff=float(abs(values-cache[f'gru{seed}'][day:end]).max()); assert diff<1e-6

    records.append(dict(fit_day=day,base_seed=seed,actual_seed=day+seed,max_replay_error=diff,checkpoint_sha256=
    hashlib.sha256(path.read_bytes()).hexdigest()))
    np.testing.assert_allclose(cache['gru_mean'][14:],np.mean([cache[f'gru{s}'][14:] for s in
    [17,42,2026]],axis=0),atol=1e-12,rtol=0)
    result=dict(status='PASS',seasonal_release_vectors=358*4,ridge_refits=13,gru_checkpoint_replays=39,
    seed_reporting_correction='17,42,2026 are base seeds; actual seed is base seed plus fit-day index',
    max_replay_error=max(x['max_replay_error'] for x in records),records=records,
    cache_sha256=hashlib.sha256(Path('artifacts/q34/price_forecasts.npz').read_bytes()).hexdigest())
    dump_json('artifacts/v5/price_replay_audit.json',result); print({k:v for k,v in result.items() if
    k!='records'})
    if __name__=='__main__':main()
```

src/v5_retained_audit.py

```
"""Rerun retained Q1, Q2 and Q4-ahead decisions from raw inputs."""
from pathlib import Path
import argparse
import numpy as np
import pandas as pd
from src.q12 import forecasts, backtest, optimize_day, dump_json
from src.q34_data import read_extended
from src.v3_model import run, Policy, risk_buffers

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root',required=True);a=ap.parse_args();out=Path('artifacts/v5')
    data,_=read_extended(a.data_root);pred,_=forecasts(data,'ridge',(1.,1.))
    L,V,p=[np.roll(data[k],1) for k in ['q1_load','q1_pv','price']]
    _,m=optimize_day(L,V,p,6000,(6000,'equal'));assert abs(m['cost']-35126.84858928963)<1e-7
    daily,_=state=backtest(data,pred,31,365,9902.811287870369,.7,'q2_reproduced',retain=False)
    old=pd.read_csv('artifacts/q12/ridge_q0.7_intervals.csv.gz')
    old_cost=(old.price*(old.plan_kwh+5*old.emergency_kwh)).groupby(old.date).sum().to_numpy()
    diff2=float(abs(daily.total_cost_yuan.to_numpy()-old_cost).max());assert diff2<1e-6

    ll=np.repeat(pred['load'][:,None,:],4,axis=1);pv=np.repeat(pred['pv'][:,None,:],4,axis=1);rr=risk_buffers(data,ll,pv,.7)

    r=run(data,ll,pv,rr,Policy(),start=31,stop=365,initial=9902.811287870369,name='q4_ahead_reproduced',prices=np.load('artifacts/q34/price_forecasts.npz')['ridge'],retain=False)
    old4=pd.read_csv('artifacts/v3/q4_2_ridge_daily.csv');diff4=float(abs(r[0].total_cost_yuan.to_numpy()-old4.total_cost_yuan.to_numpy()).max());assert diff4<1e-6

    result=dict(status='PASS',q1_milp_cost=m['cost'],q2_cost=float(daily.total_cost_yuan.sum()),q2_max_daily_difference=diff2,

    q4_ahead_cost=r[3]['total_cost_yuan'],q4_max_daily_difference=diff4,recomputed_days_per_stochastic_policy=334)
    dump_json(out/'retained_reproduction.json',result);print(result)
    if __name__=='__main__':main()
```

src/v5b_value.py

```
"""Continuous convex piecewise-linear storage values; no SOC discretization."""
from dataclasses import dataclass
import numpy as np
import pandas as pd
from src.q12 import B, audit_trace

@dataclass
class Value:
    x: np.ndarray
    y: np.ndarray
    def at(self, s): return np.interp(s, self.x, self.y)
    @property
    def slopes(self): return np.diff(self.y)/np.diff(self.x)

def restrict(f, lo=B.minimum, hi=B.maximum):
    lo = max(lo, float(f.x[0])); hi = min(hi, float(f.x[-1]))
    assert lo <= hi + 1e-7
    x = np.unique(np.r_[lo, f.x[(f.x > lo + 1e-8) & (f.x < hi - 1e-8)], hi])
    y = f.at(x)
    if len(x) > 2:
        slopes = np.diff(y)/np.diff(x); keep = np.r_[True, abs(np.diff(slopes)) > 1e-8, True]
        x, y = x[keep], y[keep]
    return Value(x, y)

def convolve(f, h):
    """Convex infimal convolution by merging segment slopes and lengths."""
    lengths = np.r_[np.diff(f.x), np.diff(h.x)]; slopes = np.r_[f.slopes, h.slopes]
    order = np.argsort(slopes, kind='stable'); lengths = lengths[order]; slopes = slopes[order]
    x = np.r_[f.x[0] + h.x[0], f.x[0] + h.x[0] + np.cumsum(lengths)]
    y = np.r_[f.y[0] + h.y[0], f.y[0] + h.y[0] + np.cumsum(lengths * slopes)]
    return restrict(Value(x, y))

def purchase_stage(net, p):
    # v = s_before - s_after; negative v charges, positive v discharges.
    lo = -B.eta_c * B.limit; hi = B.limit / B.eta_d
    zero = net / B.eta_d if net >= 0 else net * B.eta_c
    x = np.unique(np.r_[lo, hi, np.clip(0, lo, hi), np.clip(zero, lo, hi)])
    y = p * np.maximum(0, net + np.where(x >= 0, -B.eta_d * x, -x / B.eta_c))
    return Value(x, y)

def deterministic(load, pv, price, initial=6000, terminal=6000):
    n = len(price); values = [None] * (n + 1); values[n] = Value(np.array([float(terminal)]), np.array([0.]))
    stages = [purchase_stage(l - v, p) for l, v, p in zip(load, pv, price)]
    for t in range(n - 1, -1, -1): values[t] = convolve(values[t + 1], stages[t])
    s = float(initial); records = []
    for t, h in enumerate(stages):
        future = values[t + 1]; lo = max(future.x[0], s - h.x[-1]); hi = min(future.x[-1], s - h.x[0])
        choices = np.unique(np.clip(np.r_[lo, hi, future.x, s - h.x], lo, hi))
        costs = future.at(choices) + h.at(s - choices); end = float(choices[np.argmin(costs)])
        c = max(0, (end - s) / B.eta_c); d = max(0, (s - end) * B.eta_d); g = max(0, load[t] - pv[t] + c - d); w = g + pv[t] + d - load[t] - c
        records.append([g, c, d, max(0, w), s, end, load[t], pv[t], 0., price[t]]); s = end
    f = pd.DataFrame(records, columns=['plan_kwh', 'charge_kwh', 'discharge_kwh', 'spill_kwh', 'soc_start_kwh', 'soc_end_kwh', 'load_kwh', 'pv_kwh', 'emergency_kwh', 'price'])
    audit_trace(f, initial=initial)
    assert abs(float(price[f.plan_kwh]) - float(values[0].at(initial))) < 1e-6
    return f, values

def recourse_values(net, price, terminal_rate):
    """A historical path's perfect-information value; used only as an approximation.

    Emergency supply serves deficits only; it cannot charge storage. Surplus
    charges greedily, which is optimal for these nonincreasing future values.
```

```

"""
n=len(net);out=[None]*(n+1)
out[n]=Value(np.array([B.minimum,6000.,B.maximum]),terminal_rate*np.array([4800.,0.,0.]))
for t in range(n-1,-1,-1):
    if net[t]>0:
        top=min(float(net[t]),B.limit)/B.eta_d
        h=Value(np.array([0.,top]),5*price[t]*np.array([net[t],net[t]-B.eta_d*top]))
        out[t]=convolve(out[t+1],h)
    else:
        a=min(-float(net[t]),B.limit)*B.eta_c;f=out[t+1]
        x=np.unique(np.r_[B.minimum,np.clip(f.x-a,B.minimum,B.maximum),B.maximum])
        out[t]=restrict(Value(x,f.at(np.minimum(B.maximum,x+a))))
return out

def execute_value(plan,load,pv,initial,price,paths,terminal_rate):
    all_values=[recourse_values(net,price,terminal_rate) for net in paths]
    s=float(initial);rows=[]
    for t,(g,l,v,p) in enumerate(zip(plan,load,pv,price)):
        net=float(1-v-g);c=d=e=w=0.
        if net<=0:
            c=min(-net,B.limit,(B.maximum-s)/B.eta_c);w=-net-c;end=s+B.eta_c*c
        else:
            lo=max(B.minimum,s-min(net,B.limit)/B.eta_d);hi=s
            choices=np.unique(np.r_[lo,hi,*[f[t+1].x[(f[t+1].x>lo)&(f[t+1].x<hi)] for f in all_values]])
            costs=5*p*(net-B.eta_d*(s-choices))+np.mean([f[t+1].at(choices) for f in all_values],axis=0)
            end=float(choices[np.argmin(costs)]);d=B.eta_d*(s-end);e=max(0,net-d)
            rows.append([g,l,v,c,d,e,w,s,end,p]);s=end
f=pd.DataFrame(rows,columns=['plan_kwh','load_kwh','pv_kwh','charge_kwh','discharge_kwh','emergency_kwh','spill_kwh','soc_start_kwh','soc_end_kwh','price'])
audit_trace(f,initial=initial);return f

```

src/v5b_experiments.py

```
"""Screenshot-inspired tests: exact deterministic DP and causal Q2 recourse."""
from pathlib import Path
import argparse,time,json,hashlib
import numpy as np
import pandas as pd
from src.q12 import forecasts,optimize_day,reserve,execute_day,summarize_day,dump_json,audit_trace,B
from src.q34_data import read_extended
from src.v5b_value import deterministic,execute_value

OUT=Path('artifacts/v5b')
def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root',required=True);a=ap.parse_args();OUT.mkdir(parents=True,exist_ok=True)
    design=dict(candidates=['greedy','value_zero','value_night'],scenario_days=28,risk_quantile=.7,
        terminal_rates={'value_zero':0,'value_night':'first-slot fixed price / charging efficiency'},
        validation=[21,31],evaluation=[31,365],selection='January realized fee; greedy wins ties',
        scenario='Paired historical load and PV residuals from completed days; nonnegative reconstructed
levels',
        note='Mean historical perfect-information path values approximate continuation cost; causal action
is not exact stochastic DP')
    dump_json(OUT/'design_before_evaluation.json',design)
    data,_=read_extended(a.data_root);pred,_=forecasts(data,'ridge',(1.,1.))
    L,V,p=[np.roll(data[k],1) for k in ['q1_load','q1_pv','price']]
    f,vals=deterministic(L,V,p);f.to_csv(OUT/'q1_dp_intervals.csv',index=False,float_format='%.17g')
    _,m=optimize_day(L,V,p,6000,(6000,'equal'));assert abs(m['cost']-p@f.plan_kwh)<1e-6
    checks=[]
    for s in [1200.,2400.,3600.,4800.,6000.,7200.,8400.,9600.,10800.]:
        _,ref=optimize_day(L,V,p,s,(6000,'equal'))
        val=float(vals[0].at(s));assert abs(val-ref['cost'])<1e-5
        checks.append(dict(initial_soc=s,dp_cost=val,milp_cost=ref['cost']))
    pd.DataFrame(checks).to_csv(OUT/'q1_state_value_check.csv',index=False)

pd.DataFrame({'soc_kwh':vals[0].x,'future_cost_yuan':vals[0].y}).to_csv(OUT/'q1_initial_value.csv',index=False)
print('Q1 continuous DP',float(p@f.plan_kwh),len(vals[0].x),'breakpoints',flush=True)
def calc(name,start,stop,retain):
    s=6244.256891388887 if start==21 else 9902.811287870369;days=[];frames=[];tick=time.perf_counter()
    for day in range(start,stop):
        r=reserve(data,pred,day,.7);nominal,_=optimize_day(pred['load'][day]+r,pred['pv'][day],data['price'],s)
        plan=nominal.plan_kwh.to_numpy()
        if name=='greedy':f=execute_day(plan,data['load'][day],data['pv'][day],s,data['price'])
        else:
            left=max(7,day-28);l=np.maximum(0,pred['load'][day]+data['load'][left:day]-pred['load'][left:day]);v=np.maximum(0,pred['pv'][day]+data['pv'][left:day]-pred['pv'][left:day])
            paths=l-v-plan
            rate=0. if name=='value_zero' else float(data['price'][0]/B.eta_c)
            f=execute_value(plan,data['load'][day],data['pv'][day],s,data['price'],paths,rate)
            s=float(f.soc_end_kwh.iloc[-1]);days.append(summarize_day(f,data['dates'][day],name,0))
        retain=f.insert(0,'date',str(data['dates'][day].date()));f.insert(1,'slot',range(144));frames.append(f)
        if retain and data['dates'][day].day==1:print(name,str(data['dates'][day].date()),flush=True)

daily=pd.DataFrame(days);summary=dict(candidate=name,cost=float(daily.total_cost_yuan.sum()),emergency_kwh=float(daily.emergency_kwh.sum()),final_soc=s,seconds=time.perf_counter()-tick)
    if retain:
        trace=pd.concat(frames,ignore_index=True);summary['physical']=audit_trace(trace,initial=9902.811287870369)

daily.to_csv(OUT/f'q2_{name}_daily.csv',index=False,float_format='%.17g');trace.to_csv(OUT/f'q2_{name}_intervals.csv',index=False,float_format='%.17g')
    return summary
    validation=[calc(name,21,31,False) for name in design['candidates']];best=min(validation,key=lambda r:(r['cost'],r['candidate']!= 'greedy'))['candidate']

dump_json(OUT/'validation.json',validation);dump_json(OUT/'selection.json',{'candidate':best});print('January',validation,best,flush=True)
    results=[calc(name,31,365,True) for name in design['candidates']];dump_json(OUT/'summary.json',results);print(results,flush=True)

dump_json(OUT/'source_manifest.json',{str(f):hashlib.sha256(f.read_bytes().replace(b'\r\n',b'\n')).hexdigest() for f in [Path(__file__),Path('src/v5b_value.py')]})
if __name__=='__main__':main()
```


src/v5b_rolling.py

```

"""State-value control in the remaining three cases, with causal prices."""
from pathlib import Path
import argparse,json,time
import numpy as np
import pandas as pd
from src.q12 import B,forecasts,audit_trace,execute_day,dump_json
from src.q34_data import read_extended
from src.v3_model import solve,risk_buffers
from src.v5_model import adaptive_load
from src.v5b_value import recourse_values

OUT=Path('artifacts/v5b')
def execute_prefix(plan,load,pv,initial,actual_price,forecast_price,paths,steps,release_aware=False):
    """Future values use forecast prices; only executed slots read real prices."""
    continuation_price=np.array(forecast_price,copy=True)
    # Beyond the next release, additional ordinary contracts cost 1.5p rather
    # than 5p. This is a value-function approximation, not a new billing rule.
    if release_aware:continuation_price[steps:]*=.3
    values=[recourse_values(path,continuation_price,0.) for path in paths];s=float(initial);rows=[]
    for t in range(steps):
        g,l,v,p=plan[t],load[t],pv[t],actual_price[t];net=float(1-v-g);c=d=e=w=0.
        if net<=0:c=min(-net,B.limit,(B.maximum-s)/B.eta_c);w=-net-c;end=s+B.eta_c*c
        else:
            lo=max(B.minimum,s-
min(net,B.limit)/B.eta_d);choices=np.unique(np.r_[lo,s,*[f[t+1].x[(f[t+1].x>lo)&(f[t+1].x<s)] for f in
values]])
            costs=5*p*(net-B.eta_d*(s-choices))+np.mean([f[t+1].at(choices) for f in values],axis=0)
            end=float(choices[np.argmin(costs)]);d=B.eta_d*(s-end);e=max(0,net-d)
            rows.append([g,l,v,c,d,e,w,s,end,p]);s=end

f=pd.DataFrame(rows,columns=['plan_kwh','load_kwh','pv_kwh','charge_kwh','discharge_kwh','emergency_kwh','sp
ill_kwh','soc_start_kwh','soc_end_kwh','price'])
    audit_trace(f,initial=initial);return f

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root',required=True);ap.add_argument('--
case',choices=['q3','q42','q43'],required=True);ap.add_argument('--extra-
only',action='store_true');a=ap.parse_args()
    case=a.case;rolling=case!='q42';q=.5 if rolling else .7
    controls=['greedy','value','value_release'] if rolling else ['greedy','value']
    dump_json(OUT/f'{case}_design.json',dict(candidates=controls,selection='January cost; greedy wins ties',
    quantile=q,terminal=6000,scenario_days=28,terminal_value_rate=0.,
    load_selector='Retain earlier V5 causal monthly load selections as a shared forecast input',
    price_information='Only the executed slot sees actual prices; future value uses current-release
price forecast',
    release_extension='Added after January frozen-contract value performed poorly; assumes future
shortage can enter a 1.5p adjusted contract after next issue. Actual bills unchanged.))

data,_=read_extended(a.data_root);pred,_=forecasts(data,'ridge',(1,1));base=np.repeat(pred['load'][:,None,
:],4,axis=1)
    loads={'baseline':base,'adaptive':adaptive_load(data,pred['load'])[0]}
    pv=data['pv_issued'] if rolling else np.repeat(pred['pv'][:,None,:],4,axis=1)
    rr={k:risk_buffers(data,ll,pv,q) for k,ll in loads.items()}
    pp=None if case=='q3' else np.load('artifacts/q34/price_forecasts.npz')['ridge' if case=='q42' else
'seasonal']
    selections=pd.read_csv('artifacts/v5/online_selection.csv');mode=3 if case=='q3' else 4
    choices={int(r.month):r.candidate for r in
selections[(selections['mode']==mode)&selections.selected].itertuples())
    def run(control,start,stop,retain):
        state=6244.25689138887 if start==21 else
9902.811287870369;frames=[];daily=[];versions=[];tick=time.perf_counter()
        for day in range(start,stop):
            date=str(data['dates'][day].date());month=int(data['dates'][day].month);name=choices.get(month,'baseline')
            if rolling else 'baseline'
                ll=loads[name];buffer=rr[name];p0=data['price'] if pp is None else
pp[day,0,:144];actual=data['price'] if pp is None else data['actual_price'][day]

nominal,_=solve(ll[day,0]+buffer[day,0],pv[day,0],p0,state);original=nominal.plan_kwh.to_numpy().copy();plan
=original.copy();blocks=[]
                for ver in (range(4) if rolling else [0]):
                    first=ver*36;steps=36 if rolling else 144;p=data['price'][first:] if pp is None else
pp[day,ver,:144-first]

```

```

        if ver:

old=plan[first:].copy();nominal,_=solve(ll[day,ver,first:]+buffer[day,ver,first:],pv[day,ver,first:],p,state
,old=original[first:]);plan[first:]=np.maximum(nominal.plan_kwh,0)
        if retain:
            for j,t in
enumerate(range(first,144)):versions.append([date,ver,first,t,float(old[j]),float(plan[t]),float(p[j]),float
(ll[day,ver,t]),float(pv[day,ver,t]),float(buffer[day,ver,t]),state))
            if
control=='greedy':f=execute_day(plan[first:first+steps],data['load'][day,first:first+steps],data['pv'][day,f
irst:first+steps],state,actual[first:first+steps])
            else:
                left=max(7,day-28)
                L=np.maximum(0,ll[day,ver,first:]+data['load'][left:day,first:]-ll[left:day,ver,first:])
                V=np.maximum(0,pv[day,ver,first:]+data['pv'][left:day,first:]-pv[left:day,ver,first:])
f=execute_prefix(plan[first:],data['load'][day,first:],data['pv'][day,first:],state,actual[first:],p,L-V-
plan[first:],steps,control=='value_release')
                state=float(f.soc_end_kwh.iloc[-1]);blocks.append(f)

f=pd.concat(blocks,ignore_index=True);f.insert(0,'date',date);f.insert(1,'slot',range(144));f['original_plan
_kwh']=original
                f['increase_kwh']=np.maximum(plan-original,0);f['decrease_kwh']=np.maximum(original-plan,0)

f['original_cost_yuan']=actual*original;f['increase_cost_yuan']=1.5*actual*f.increase_kwh;f['decrease_cost_y
uan']=-.5*actual*f.decrease_kwh;f['emergency_cost_yuan']=5*actual*f.emergency_kwh

f['total_cost_yuan']=f[['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan'
]].sum(axis=1)
                daily.append(dict(date=date,soc_start_kwh=float(f.soc_start_kwh.iloc[0]),soc_end_kwh=state,
                **{k:float(f[k].sum()) for k in
['total_cost_yuan','original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','pla
n_kwh','emergency_kwh','charge_kwh','discharge_kwh','spill_kwh']}))
                if retain:frames.append(f)
                if retain and data['dates'][day].day==1:print(case,control,date,flush=True)

d=pd.DataFrame(daily);s=dict(case=case,control=control,cost=float(d.total_cost_yuan.sum()),final_soc=state,e
mergency_kwh=float(d.emergency_kwh.sum()),seconds=time.perf_counter()-tick)
                if retain:

f=pd.concat(frames,ignore_index=True);s['physical']=audit_trace(f,initial=9902.811287870369);d.to_csv(OUT/f'
{case}_{control}_daily.csv',index=False,float_format='%.17g');f.to_csv(OUT/f'{case}_{control}_intervals.csv.
gz',index=False,float_format='%.17g')
                if
versions=pd.DataFrame(versions,columns=['date','version','effective_slot','slot','old_kwh','new_kwh','foreca
st_price','forecast_load_kwh','forecast_pv_kwh','reserve_kwh','decision_soc_kwh']).to_csv(OUT/f'{case}_{cont
rol}_versions.csv.gz',index=False,float_format='%.17g')
                return s
                prior=json.loads((OUT/f'{case}_validation.json').read_text(encoding='utf-8'))['validation'] if
a.extra_only else []
                selected_controls=['value_release'] if a.extra_only else controls
                val=prior+[run(c,21,31,False) for c in selected_controls];chosen=min(val,key=lambda
x:(x['cost'],x['control']!='greedy'))['control']

dump_json(OUT/f'{case}_validation.json',{'validation':val,'selected':chosen});print(case,'January',val,chose
n,flush=True)
                prior=json.loads((OUT/f'{case}_summary.json').read_text(encoding='utf-8')) if a.extra_only else []
                results=prior+[run(c,31,365,True) for c in
selected_controls];dump_json(OUT/f'{case}_summary.json',results);print(case,results,flush=True)
if __name__=='__main__':main()

```

src/v5b_windows.py

```
"""Training-window sensitivity, separated from policy selection."""
from pathlib import Path
import inspect, argparse
import numpy as np
import pandas as pd
import src.q12 as q12
from src.q34_data import read_extended

def window_forecasts(data, window):
    # Reuse the full checked forecaster, changing exactly one training horizon.
    source=inspect.getsource(q12.forecasts).replace('d-60', 'd-'+str(int(window)))
    namespace=dict(vars(q12));exec(compile(source, '<window_forecasts>', 'exec'), namespace)
    return namespace['forecasts'](data, 'ridge', (1., 1.))[0]

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root', required=True);a=ap.parse_args()
    out=Path('artifacts/v5b');data,_=read_extended(a.data_root);rows=[]
    q12.dump_json(out/'window_design.json', {'windows_days': [28, 56, 60, 84], 'same_model': 'ridge alpha 1; weekly
refit; all other features unchanged',
    'interpretation': 'Sensitivity only; January cannot distinguish these windows because training starts
on day 7. Do not pick lowest annual result as a validated policy.'})
    for window in [28, 56, 60, 84]:
        pred=window_forecasts(data, window)
        val=q12.backtest(data, pred, 21, 31, 6244.256891388887, .7, 'validation', retain=False)[0]
        daily=q12.backtest(data, pred, 31, 365, 9902.811287870369, .7, 'window', retain=False)[0]

    row=dict(window_days=window, validation_cost=float(val.total_cost_yuan.sum()), annual_cost=float(daily.total_c
ost_yuan.sum()),
        load_mae_kwh=float(abs(pred['load'])[31:]-
data['load'])[31:]).mean()), pv_mae_kwh=float(abs(pred['pv'])[31:]-data['pv'])[31:]).mean()))
        rows.append(row);print(row, flush=True)
    pd.DataFrame(rows).to_csv(out/'window_sensitivity.csv', index=False)
if __name__=='__main__':main()
```

src/v5b_analysis.py

```

"""Recompute bills from raw inputs, audit revisions and quantify paired gains."""
from pathlib import Path
import argparse, json, hashlib
import numpy as np
import pandas as pd
from src.q12 import audit_trace, dump_json
from src.q34_data import read_extended
from src.v5_audit import independent_lp
OUT=Path('artifacts/v5b')
SELECTED={'2':'q2_value_zero', '3':'q3_value_release', '4-2':'q42_value', '4-3':'q43_value_release'}
def canonical(name):
    f=pd.read_csv(OUT/f'{name}_intervals.csv.gz')
    if 'original_plan_kwh' not in f:
        f['original_plan_kwh']=f.plan_kwh
        f['original_cost_yuan']=f.price*f.plan_kwh
        f['increase_cost_yuan']=0.; f['decrease_cost_yuan']=0.
        f['emergency_cost_yuan']=5*f.price*f.emergency_kwh
        f['total_cost_yuan']=f.original_cost_yuan+f.emergency_cost_yuan
    return f
def main():
    ap=argparse.ArgumentParser(); ap.add_argument('--data-root', required=True); a=ap.parse_args()
    data,_=read_extended(a.data_root); audits=[]; certs=[]
    names=[p.name.removesuffix('_intervals.csv.gz') for p in OUT.glob('*_intervals.csv.gz')]
    for name in sorted(names):
        if name.endswith('_selected'): continue
        f=canonical(name); assert len(f)==334*144
        assert not f[['date', 'slot']].duplicated().any()
        assert f.date.tolist()==np.repeat(data['dates'][31:].strftime('%Y-%m-%d'), 144).tolist()
        np.testing.assert_array_equal(f.slot, np.tile(np.arange(144), 334))
        for col, key in
            [ ('load_kwh', 'load'), ('pv_kwh', 'pv') ]:
            np.testing.assert_allclose(f[col], data[key][31:].reshape(-1), rtol=0, atol=1e-10)
        p=np.tile(data['price'], 334) if name.startswith(('q2_', 'q3_')) else
        data['actual_price'][31:].reshape(-1)
        np.testing.assert_allclose(f.price, p, atol=1e-12, rtol=0)
        fee=p*(np.minimum(f.original_plan_kwh, f.plan_kwh)+.5*np.maximum(f.original_plan_kwh-
        f.plan_kwh, 0)+1.5*np.maximum(f.plan_kwh-f.original_plan_kwh, 0)+5*f.emergency_kwh)
        np.testing.assert_allclose(fee, f.total_cost_yuan, atol=1e-7, rtol=0)

    d=pd.read_csv(OUT/f'{name}_daily.csv'); np.testing.assert_allclose(fee.groupby(f.date).sum(), d.total_cost_yua
    n, atol=1e-6, rtol=0)
    physical=audit_trace(f, initial=9902.811287870369)
    vp=OUT/f'{name}_versions.csv.gz'
    if vp.exists():
        v=pd.read_csv(vp)
        for date, g in f.groupby('date', sort=False):
            plan=g.original_plan_kwh.to_numpy().copy()
            for ver in [1, 2, 3]:
                z=v[(v.date==date)&(v.version==ver)]; slots=z.slot.to_numpy(int)
                np.testing.assert_array_equal(slots, np.arange(ver*36, 144))
                np.testing.assert_allclose(plan[slots], z.old_kwh, atol=1e-8, rtol=0)
                assert abs(z.decision_soc_kwh.iloc[0]-g.soc_start_kwh.iloc[ver*36])<1e-8
                plan[slots]=z.new_kwh
                np.testing.assert_allclose(plan, g.plan_kwh, atol=1e-8, rtol=0)
            if name in SELECTED.values():
                for date in ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']:
                    for ver in [1, 2, 3]:
                        z=v[(v.date==date)&(v.version==ver)]; g=f[(f.date==date)&(f.slot>=ver*36)]
                        c=independent_lp((z.forecast_load_kwh+z.reserve_kwh).to_numpy(), z.forecast_pv_kwh.to_numpy(), z.forecast_pric
                        e.to_numpy(), float(z.decision_soc_kwh.iloc[0]), old=g.original_plan_kwh.to_numpy())
                        # The recorded future plan is a nominal optimizer; subsequent
                        # execution and reoptimizations need not follow its SOC path.
                        delta=z.new_kwh.to_numpy()-g.original_plan_kwh.to_numpy()
                        recorded=float(z.forecast_price.to_numpy())@(1.5*np.maximum(delta, 0)-.5*np.maximum(-
                        delta, 0)))
                        assert abs(recorded-c['objective'])<1e-5
                        c.update(case=name, date=date, version=ver, recorded_difference=abs(recorded-
                        c['objective'])); certs.append(c)
                    audits.append(dict(name=name, cost=float(fee.sum()), intervals=len(f), physical=physical))
            gains=[]; monthly=[]; rng=np.random.default_rng(20260912)
            for key, name in SELECTED.items():
                base=canonical(name.split('_')[0]+'_greedy'); f=canonical(name)
                assert abs(f.soc_end_kwh.iloc[-1]-base.soc_end_kwh.iloc[-1])<1e-7
                diff=(base.total_cost_yuan-f.total_cost_yuan).groupby(f.date).sum().to_numpy(); boots=[]
                for block in [7, 14, 28]:

```

```

idx=(rng.integers(0,len(diff),(2000,int(np.ceil(len(diff)/block))))[:,None]+np.arange(block))%len(diff)
sums=diff[idx.reshape(2000,-1)[:,len(diff)]]sum(axis=1)
lo,hi=np.quantile(sums,[.025,.975]);boots.append(dict(block=block,low=float(lo),high=float(hi)))

record=dict(question=key,selected=name,baseline_cost=float(base.total_cost_yuan.sum()),cost=float(f.total_cost_yuan.sum()),saving=float(diff.sum()),percent=float(diff.sum()/base.total_cost_yuan.sum()*100),bootstrap=boots,initial_soc=float(f.soc_start_kwh.iloc[0]),final_soc=float(f.soc_end_kwh.iloc[-1]))
for label,frame in [('before',base),('after',f)]:
    record[label]={c:float(frame[c].sum()) for c in
['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','emergency_kwh','spill_kwh']}
    gains.append(record)
    for month in range(2,13):
        mask=pd.to_datetime(f.date).dt.month==month
        monthly.append(dict(question=key,month=month,saving=float((base.total_cost_yuan-f.total_cost_yuan)[mask].sum()))
        f.to_csv(OUT/f'{name}_selected_intervals.csv.gz',index=False,float_format='%.17g')
        periodic=[]
        for key in ['load','pv']:
            x=data[key]
            for lag in
[1,7,14,28]:periodic.append(dict(series=key,lag_days=lag,slot_correlation=float(np.corrcoef(x[lag:].ravel(),x[:-lag].ravel())[0,1]),daily_energy_correlation=float(np.corrcoef(x[lag:].sum(1),x[:-lag].sum(1))[0,1]))
        pd.DataFrame(periodic).to_csv(OUT/'periodicity.csv',index=False)

pd.DataFrame(monthly).to_csv(OUT/'monthly_savings.csv',index=False);pd.DataFrame(certs).to_csv(OUT/'new_solver_certificates.csv',index=False)
dump_json(OUT/'improvement.json',gains)
dump_json(OUT/'verification.json',dict(status='PASS',traces=audits,total_intervals=sum(x['intervals']
for x in audits),new_independent_lp_cases=len(certs),max_duality_gap=max(x['gap'] for x in
certs),inputs={str(p):hashlib.sha256(p.read_bytes()).hexdigest() for p in Path(a.data_root,'附件').glob('附件*.xlsx')},sources={str(p):hashlib.sha256(p.read_bytes().replace(b'\r\n',b'\n')).hexdigest() for p in
Path('src').glob('v5b_*.py')}))
print('PASS',len(audits),'full traces;',len(certs),'independent LP
certificates');print([(r['question'],r['cost'],r['saving']) for r in gains])
if __name__=='__main__':main()

```

src/v5b_delivery.py

```
"""Write V5 data cells into V3 layouts and read every written cell back."""
from pathlib import Path
from datetime import datetime
import json,shutil,math,hashlib
from copy import copy
import pandas as pd
import openpyxl
from src.q12 import dump_json
from src.deliver_q12 import blocks,emergency_runs

OUT=Path('artifacts/v5b')
from src.v5b_analysis import SELECTED
SOURCES={'1':'artifacts/v3/q1_intervals.csv','**{k:str(OUT/f'{v}_selected_intervals.csv.gz') for k,v in
SELECTED.items()}}

def main():
    audit=[]
    for key in ['1']:shutil.copy2(f'artifacts/v3/result{key}.xlsx',OUT/f'result{key}.xlsx')
    for key,label in SELECTED.items():
        f=pd.read_csv(SOURCES[key]);v=pd.read_csv(OUT/f'{label}_versions.csv.gz') if key in ['3','4-3'] else
pd.DataFrame();payload={k:[] for k in (['计划购电量','调整购电量','充放电量','紧急购电量','费用分解','计划版本']
if key in ['3','4-3'] else ['计划购电量','充放电量','紧急购电量']+(['费用分解'] if key=='4-2' else []))}
        for date,g in f.groupby('date',sort=False):
            dt=datetime.strptime(date,'%Y-%m-%d')
            payload['计划购电量']
            '.append([dt,*g.original_plan_kwh.tolist(),float(g.original_plan_kwh.sum()),float(g.original_cost_yuan.sum(
)) if key in ['3','4-3'] else float(g.total_cost_yuan.sum())])
            if '调整购电量' in payload:payload['调整购电量']
            '.append([dt,*g.plan_kwh.tolist(),float(g.plan_kwh.sum()),float(g.total_cost_yuan.sum())])
            for i,b in enumerate(blocks(g)):
                payload['充放电量'].append([dt if i==0 else None,*b,'00:00' if i==0 else '24:00' if i==1 else
None,float(g.soc_start_kwh.iloc[0]) if i==0 else float(g.soc_end_kwh.iloc[-1]) if i==1 else None])
                for i,b in enumerate(emergency_runs(g) or [['无',0]]):payload['紧急购电量'].append([dt if i==0
else None,*b])
                if '费用分解' in payload:payload['费用分解'].append([dt,*[float(g[c].sum()) for c in
['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','total_cost_yuan']]])
                for (date,ver),g in (v.groupby(['date','version'],sort=True) if len(v) else []):
                    values=[None]*144
                    for r in g.itertuples():values[int(r.slot)]=float(r.new_kwh)
                    payload['计划版本'].append([datetime.strptime(date,'%Y-%m-%d'),f'{int(ver)*6:02d}:00',*values])
                    wb=openpyxl.load_workbook(f'artifacts/v5/result{key}.xlsx');assert not any(c.data_type=='f' for ws
in wb for row in ws for c in row),'Formula-bearing template needs recalculation'
                    headers={ws.title:tuple(c.value for c in ws[1]) for ws in wb};sheets=wb.sheetnames
                    for name,rows in payload.items():
                        ws=wb[name];end=ws.max_row;columns=len(rows[0])
                        for row in ws.iter_rows(min_row=2,max_row=max(end,len(rows)+1),max_col=columns):
                            for cell in row:cell.value=None
                        for i,row in enumerate(rows,2):
                            for j,value in enumerate(row,1):
                                cell=ws.cell(i,j);cell.value=value
                                if i>end:cell._style=copy(ws.cell(2,j)._style)
                    if '口径说明' in wb:wb['口径说明']['B2']='V5 复核更新: 过去 28 日配对残差价值控制; 日前按固定合同, 滚动按后续
可调单成本估计库存价值; 实际费用规则不变。'
                    dest=OUT/f'result{key}.xlsx';wb.save(dest);wb.close()
                    checked=openpyxl.load_workbook(dest,read_only=True,data_only=True);count=0
                    assert checked.sheetnames==sheets
                    for name in sheets:assert tuple(next(checked[name].values))==headers[name]
                    for name,rows in payload.items():
                        for actual,expected in
zip(checked[name].iter_rows(min_row=2,max_row=len(rows)+1,max_col=len(rows[0]),values_only=True),rows):
                            for a,e in zip(actual,expected):
                                count+=1
                                if isinstance(e,(int,float)):assert isinstance(a,(int,float)) and
math.isclose(a,e,abs_tol=1e-7,rel_tol=1e-11),(name,a,e)
                                else:assert a==e,(name,a,e)
                            for row in checked[name].iter_rows(min_row=len(rows)+2,values_only=True):assert all(v is None
for v in row)

checked.close();audit.append(dict(key=key,cells_compared=count,headers_preserved=True,formula_cells=0,sha256
=hashlib.sha256(dest.read_bytes()).hexdigest()))
    dump_json(OUT/'delivery_selection.json',{'sources':SOURCES,'main':'January-selected historical-path
value controllers; Q1 unchanged','copied_byte_identical':['1']})

dump_json(OUT/'xlsx_audit.json',dict(status='PASS',updated=audit,copied={k:hashlib.sha256((OUT/f'result{k}.x
```

```
lsx').read_bytes()).hexdigest() for k in ['1']});print(audit)
if __name__=='__main__':main()
```